
Data Science Core Skills

Mathematics

September 2026

Table of contents

Data Science Core Skills	2
Reading mathematics	3
Functions, exponentials and logarithms	11
Rates of change	19
The differentiation toolkit	29
Finding the best value	36
Several variables and the gradient	44
Accumulation and area	55
Data as vectors	64
Matrices as operations on data	73
Solving $Ax = b$	83
When there is no solution: least squares	92
Directions in data: eigenvectors and the SVD	102

Data Science Core Skills

The mathematics, statistics and Python for your MSc, one unit at a time

This booklet is the **Mathematics** track of *Data Science Core Skills*, a self-study course for students on an MSc in Data Science. The whole course — all three subjects, with runnable code and revealable solutions — is at uh-pam.github.io/bootcamp.

Twelve lessons covering the calculus and linear algebra that a data science degree assumes you have met: rates of change, optimisation, integration, vectors, matrices, least squares and the SVD.

Two lessons are needed elsewhere: **M7** (integration) before S5, and **M11** (least squares) before S11.

Every lesson ends by checking its own claim in Python. If you cannot yet read that code, the Python track is eleven short lessons and worth the detour.

On the site, the lessons of all three subjects are arranged into ten units of about six to seven hours each. The line at the top of each lesson says which unit it belongs to.

Every worked example uses one small table describing a fictional previous cohort: their prior background, hours of practice and final scores. **The data are invented.** They describe no real student and no real cohort.

Reading mathematics

Unit 2 · Lesson 3 of 3 · Mathematics · M1 Reading: 5 pages With practice: ⌚ 1.5 hours Assumes: nothing — start here

Why this lesson is here

Most people who find mathematics hard do not find the *ideas* hard. They find the **notation** hard, and then never get to the ideas.

A formula is a compressed sentence. Once you can read it aloud and say what every symbol stands for, the rest is arithmetic you already know. This lesson teaches the reading, and nothing else — there is no new mathematics in it at all.

Before you start

This lesson assumes no university mathematics, only school algebra. You should be able to rearrange something like $2x + 3 = 11$, work with fractions, and read a pair of coordinates off a graph. That is genuinely all.

Quick check. Solve $3x - 4 = 11$.

💡 Show the answer

$x = 5$. Add 4 to both sides to get $3x = 15$, then divide both sides by 3.

If you found 5 by trying numbers, check you can also do the two steps: trial gets slow once equations get harder. If that felt slow rather than wrong, you will be fine — it comes back quickly. If it felt genuinely unfamiliar, spend an hour on a GCSE algebra refresher before going on, because everything after this assumes it.

The idea

Take a formula that will appear later in these notes:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

It looks like four unfamiliar things. It is one familiar thing: **add up all the values and divide by how many there are**. It is the average.

Read it aloud, left to right:

- $\bar{x}$ — “x bar”, a single number, the average of the x values.
- $=$ — is defined to be.
- $\frac{1}{n}$ — one over n , where n is how many values there are.
- $\sum$ — “sum”, meaning add up what follows.
- $i = 1$ underneath and n on top — starting at the first value and going to the last.
- x_i — “x sub i”, the i -th value.

$\bar{x}$

↓

mean

=

$\frac{1}{n}$

/ len(x)

$\sum_{i=1}^n$

sum(...)

x_i

each x

the whole thing

mean = sum(x) / len(x)

Figure 1: The same instruction written twice. Every part of the formula corresponds to a part of the code; neither is more real than the other.

```
values = [68.4, 55.1, 72.0, 61.3]
mean = sum(values) / len(values)
print(mean)
```

64.2

The formula and the code say the same thing. Neither is the “real” version — they are two notations for one instruction, and being fluent in both is the point of this course.

The mechanics

Sets of numbers

Symbol	Name	Means
$\mathbb{N}$	naturals	1, 2, 3, ...
$\mathbb{Z}$	integers	..., -2, -1, 0, 1, 2, ...
$\mathbb{R}$	reals	every number on the line, including $\pi \approx 3.14159$ and $\sqrt{2} \approx 1.41421$

$x \in \mathbb{R}$ reads “ x is a real number”. The $\in$ means “is in”.

Intervals

Round brackets exclude the endpoint; square brackets include it.

- $[0, 1]$ — from 0 to 1, **including** both.
- $(0, 1)$ — from 0 to 1, **excluding** both.
- $[0, 1)$ — includes 0, excludes 1.

The distinction matters when the endpoint is impossible: a probability lies in $[0, 1]$, but a standard deviation you are about to divide by must lie in $(0, \infty)$ — zero excluded, deliberately. The symbol ∞ , *infinity*, is not a number: it says the interval has no upper end.

Summation

$$\sum_{i=1}^4 x_i = x_1 + x_2 + x_3 + x_4$$

The subscript says where to start, the superscript where to stop, and the body says what to add. It is a for loop:

```
x = [10, 20, 30, 40]

total = 0
for i in range(len(x)):      # range(4) is 0, 1, 2, 3, the same as range(0, 4)
    total = total + x[i]

print(total, sum(x))
```

```
100 100
```

Two conventions worth knowing. Mathematics counts from 1 and includes the last index; Python counts from 0 and excludes the stop value. They describe the same four numbers by different book-keeping, and mixing them up is the commonest off-by-one error there is.

A sum with something more elaborate inside works identically:

$$\sum_{i=1}^n (x_i - \bar{x})^2$$

“For each value, subtract the mean, square it, and add all of those up.” You will meet this exact expression in [S2](#) — it is the heart of the variance.

```
import numpy as np

x = np.array([68.4, 55.1, 72.0, 61.3])
print(sum((value - x.mean())**2 for value in x))
print((x - x.mean())**2).sum()
```

```
169.70000000000005
169.70000000000005
```

The second line uses a NumPy array, which [P6](#) covers: `x - x.mean()` subtracts the mean from *every* value at once, so the whole sum needs no loop. Both lines compute the same thing.

Subscripts and function notation

x_i is not x multiplied by i . The subscript is a **label** — the i -th item, like `x[i]` — except that x_1 is `x[0]`, by the counting convention above.

$f(x)$ is not f times x either. It is “the function f , applied to x ”, and it is `f(x)` in code too.

Powers, roots and logarithms

$$x^n = \underbrace{x \times x \times \cdots \times x}_{n \text{ times}} \quad x^{1/2} = \sqrt{x} \quad x^{-1} = \frac{1}{x}$$

```
print(2**10, 9**0.5, 2**-1)
```

```
1024 3.0 0.5
```

The three rules you will use constantly:

$$x^a x^b = x^{a+b} \quad \frac{x^a}{x^b} = x^{a-b} \quad (x^a)^b = x^{ab}$$

Greek letters

They are just letters, and the convention is stable enough to be worth learning:

	Name	Usually means
μ	mu	a population mean
σ	sigma	a population standard deviation
$\bar{x}$	x-bar	a sample mean
α, β	alpha, beta	parameters: unknown numbers in a model, estimated from data (not Python's function parameters)
λ	lambda	a rate
Δ	delta	a change in something

A *population* is the whole group you want to describe; a *sample* is the part you observed. S1 makes both precise.

Note σ and Σ : lower case is a standard deviation, upper case is the summation sign. Same letter, different jobs, decided by case alone.

Worked example

Reading a formula you have not met, entirely from its parts.

Here is the sample standard deviation, which arrives properly in S2:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Work outward from the middle, which is how these are built.

Innermost: $x_i - \bar{x}$ — each value minus the mean. How far this observation sits from the centre.

Then: $(x_i - \bar{x})^2$ — squared. Squaring makes everything positive, so distances above and below the mean do not cancel out.

Then: $\sum_{i=1}^n$ — add all those squared distances up.

Then: $\frac{1}{n-1}$ — divide by one less than the count. (Why $n - 1$ rather than n is a real question with a real answer, and it is S2's, not this lesson's.)

Finally: $\sqrt{}$ — square root, which undoes the squaring and returns the answer to the original units. If x was in marks, s is in marks.

Now build it up in code, one layer at a time:

```
x = np.array([68.4, 55.1, 72.0, 61.3])

deviations = x - x.mean()
squared = deviations**2
total = squared.sum()
divided = total / (len(x) - 1)
s = divided**0.5

print("deviations", deviations.round(2))
print("squared    ", squared.round(2))
print("sum        ", round(total, 2))
print("/(n-1)     ", round(divided, 2))
print("sqrt       ", round(s, 3))
print("numpy      ", round(x.std(ddof=1), 3))
```

```
deviations [ 4.2 -9.1  7.8 -2.9]
squared    [17.64 82.81 60.84  8.41]
sum         169.7
/(n-1)      56.57
sqrt        7.521
numpy       7.521
```

Five lines, five layers, and the last one confirms it against NumPy. A formula you can take apart like this is a formula you can check — and checking is what makes it yours rather than something you copied.

Try it yourself

Exercise 1 Write $\sum_{i=1}^5 i^2$ out in full, then compute it.

 Show the solution

$$1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55.$$

```
sum(i**2 for i in range(1, 6))    # range stops before 6
```

Note the `range(1, 6)`: mathematics says “to 5 inclusive”, Python says “stop before 6”. Same five numbers.

Exercise 2 Say in words what each means, and give a value that is in the first but not the second.

- a. $x \in [0, 10]$ b. $x \in (0, 10)$

💡 Show the solution

- a. x is a real number from 0 to 10, **including** both ends.
- b. x is strictly between 0 and 10, **excluding** both.

Either 0 or 10 is in the first and not the second.

Where this bites: if x is about to appear in a denominator, you need $(0, 10]$, not $[0, 10]$ — and the bracket is the only thing saying so.

Exercise 3 Translate into a single line of Python:

$$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})$$

Then predict its value before running it, for any x at all.

💡 Show the solution

```
(x - x.mean()).mean()
```

It is **zero**, always, for any set of numbers.

The mean is the balance point: the distances above it exactly cancel the distances below. That is precisely why the variance squares the deviations first — without squaring, the whole thing collapses to nothing and tells you no measure of spread at all.

Try it and you will get something like 10^{-15} rather than exactly 0 — the floating-point noise from P2.

Check it in Python

The habit this lesson is really teaching: **translate a formula into code, then check it against a library that already implements it**. Agreement is strong evidence you read the notation correctly.

```
x = np.array([68.4, 55.1, 72.0, 61.3])

by_hand = sum(x) / len(x)
print(f"by hand {by_hand:.6f}    numpy {x.mean():.6f}    agree:
      {np.isclose(by_hand, x.mean())}")
```

```
by hand 64.200000    numpy 64.200000    agree: True
```

And the deviations-sum-to-zero fact from Exercise 3, which is worth seeing:

```
print("sum of deviations:", (x - x.mean()).sum())
print("close to zero:     ", np.isclose((x - x.mean()).sum(), 0))
```

```
sum of deviations: -7.105427357601002e-15
close to zero:     True
```

Not exactly zero, and it should not be — that residue is floating-point arithmetic, not a mistake. `np.isclose` is the right question to ask.

Common wrong turns

Where this usually goes wrong

Reading x_i as x times i The subscript is a label — the i -th value, like $x[i - 1]$ in Python, which counts from 0. Likewise $f(x)$ is a function applied to x , not f multiplied by x .

Mixing the two counting conventions Mathematics counts from 1 and includes the last index; Python counts from 0 and excludes the stop. $\sum_{i=1}^n$ is `range(0, n)` or just `range(n)`.

Confusing σ with Σ Lower case is a standard deviation, upper case is a summation. The case is the only difference and it changes everything.

Ignoring which bracket an interval uses $[0, 1]$ and $(0, 1)$ differ exactly where it matters — at the end-points, which is usually where a formula would break.

Trying to absorb a formula whole Work outward from the innermost bracket. Every formula in these notes was built in layers and comes apart the same way.

Skipping the check because the algebra “looks right” Three lines of code will tell you whether you read the notation correctly, and it takes less time than staring at it.

Check yourself

Answers are at the foot of the page.

- What is $\sum_{i=1}^3 2i$?
 - 6
 - 12
 - 8
 - 2
- What is the difference between σ and Σ ?
- Is 0 in the interval $(0, 5]$?
- Write $\sqrt{x}$ using a power.
- Why does the variance square the deviations before adding them?

Show the answers

- (b) 12.** $2(1) + 2(2) + 2(3) = 2 + 4 + 6$.
- Lower-case σ is a standard deviation — a number. Upper-case Σ is the instruction to add up. Case is the only distinction.
- No.** The round bracket excludes 0. The interval is everything strictly greater than 0, up to and including 5.
- $x^{1/2}$, or equivalently $x^{0.5}$.
- Because without squaring they sum to exactly zero, for any data at all — the mean is the balance point. Squaring makes every deviation positive so they accumulate into a genuine measure of spread.

Where this shows up next

M2 uses this notation to describe functions, and adds the handful of curves that recur everywhere. **S2**

returns to the standard deviation formula from the worked example and explains the $n - 1$. Every formula in the rest of these notes is built from the pieces in this lesson.

Functions, exponentials and logarithms

Unit 3 · Lesson 3 of 4 · Mathematics · M2 Reading: 5 pages With practice: 1.5 hours Assumes: M1

Why this lesson is here

A **function** is the object every model in data science is. A regression is a function from predictors (the inputs you know) to a prediction; a loss is a function from parameters to a number measuring error, which you want small; a probability density is a function from an outcome to a height. Each arrives properly later; for now, notice that each turns an input into one output.

And a small number of shapes recur endlessly. Recognise those five curves and you can read most of what follows — including why nearly every quantity in data science gets logged at some point.

Before you start

You should be able to:

- read $f(x)$, subscripts and Σ — M1;
- work with powers and roots — M1.

Quick check. If $f(x) = 3x + 1$, what is $f(4)$?

💡 Show the answer

13. Substitute 4 wherever x appears: $3(4) + 1$.
 $f(4)$ in code does exactly the same thing.

The idea

A function is a rule with a **domain**: what you are allowed to put in.

$$f : \mathbb{R} \rightarrow \mathbb{R}, \quad f(x) = x^2$$

Read as “ f takes a real number and gives back a real number, by squaring it”. The rule alone is not the whole story — the domain is part of the definition, and leaving it out is where errors come from:

```
import numpy as np
print(np.sqrt(-4))
```

nan

$\sqrt{x}$ is only defined for $x \geq 0$ in the reals. Its domain is $[0, \infty)$, and NumPy warns rather than pretending otherwise.

The three that will catch you:

- **division** needs a denominator that is not zero;
- **square root** needs a non-negative argument;
- **logarithm** needs a *strictly positive* argument.

The mechanics

The five shapes

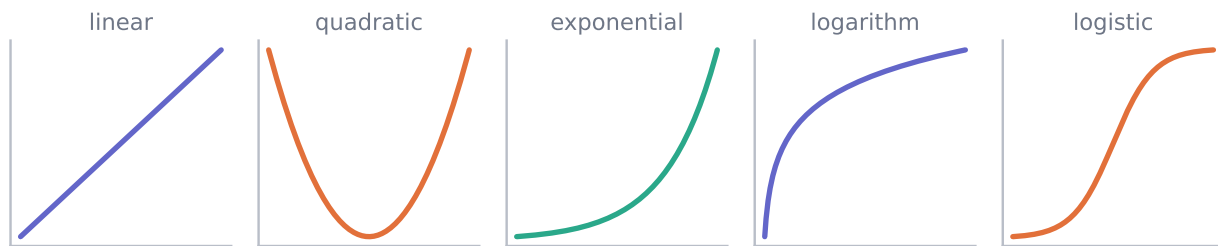


Figure 1: The five curves that recur. Each is defined by how it *changes*: constant slope, turning, accelerating, decelerating, saturating.

Linear, $f(x) = a + bx$. Constant rate of change: every extra unit of x adds the same b . The slope b is the whole content.

Quadratic, $f(x) = ax^2 + bx + c$. Turns exactly once — a single maximum or minimum. Loss functions are often quadratic near their best point, which is why they are worth recognising.

Exponential, $f(x) = e^x$, where $e \approx 2.718$ is a fixed number, like π . Constant *proportional* growth: every extra unit multiplies. Doubling times, compound interest, unchecked epidemics.

Logarithmic, $f(x) = \ln x$. The inverse of the exponential. Grows without limit but ever more slowly.

Logistic, $f(x) = 1/(1 + e^{-x})$. Starts flat, accelerates, saturates. It maps anything at all into $(0, 1)$, which is exactly what you want when the output must be a probability.

Exponentials and logs are inverses

$$\ln(e^x) = x \quad e^{\ln x} = x \quad (x > 0)$$

```
print(np.log(np.exp(3.7)))
print(np.exp(np.log(3.7)))
```

```
3.7
3.7000000000000006
```

Each undoes the other, which is what makes the logarithm the tool for getting a variable out of an exponent.

The three log rules

$$\ln(ab) = \ln a + \ln b \quad \ln \frac{a}{b} = \ln a - \ln b \quad \ln(a^b) = b \ln a$$

```
a, b = 6.0, 2.5
print(np.isclose(np.log(a*b), np.log(a) + np.log(b)))
print(np.isclose(np.log(a/b), np.log(a) - np.log(b)))
print(np.isclose(np.log(a**b), b*np.log(a)))
```

True
True
True

That third rule is the important one. It **turns a power into a multiplication**, which is how you solve for something stuck in an exponent:

$$2^x = 100 \implies x \ln 2 = \ln 100 \implies x = \frac{\ln 100}{\ln 2}$$

```
x = np.log(100) / np.log(2)
print(f"x = {x:.4f}, check: 2**x = {2**x:.4f}")
```

x = 6.6439, check: 2**x = 100.0000

i log means the **natural** logarithm, base e , in essentially all mathematics and in NumPy. Base 10 is `np.log10`, base 2 is `np.log2`. Calculators and spreadsheets often use `log` for base 10, which causes real confusion — check which you have.

Why data gets logged

Two reasons, both practical.

It turns multiplicative structure into additive structure. If a quantity grows by a constant *percentage*, its logarithm grows by a constant *amount* — a straight line, which is far easier to model and to read.

```
weeks = np.arange(0, 11)
users = 100 * 1.4**weeks          # 40% growth per week

print("users:      ", users.round(0)[:5], "...")
print("log users:", np.log(users).round(3)[:5], "...")
print("differences in log:", np.diff(np.log(users)).round(4)[:4])
```

```
users:      [100. 140. 196. 274. 384.] ...
log users: [4.605 4.942 5.278 5.615 5.951] ...
differences in log: [0.3365 0.3365 0.3365 0.3365]
```

The raw numbers accelerate away; the differences in the log are constant at $\ln 1.4 \approx 0.336$. Exponential growth has become a straight line.

It compresses a long right tail. Incomes, city sizes, file sizes and gene expression levels all have a few enormous values that dominate everything.

```
rng = np.random.default_rng(2026)
# lognormal: values whose *logs* have mean 10 and spread 1 (not an income mean of
↳ 10)
income = rng.lognormal(mean=10, sigma=1.0, size=1000)
```

```
print(f"raw:      median {np.median(income):>10,.0f}    mean
      ↳ {income.mean():>10,.0f}")
print(f"logged:  median {np.median(np.log(income)):>10.2f}    mean
      ↳ {np.log(income).mean():>10.2f}")
```

```
raw:      median      22,840    mean      38,177
logged:  median       10.04    mean       10.02
```

On the raw scale the mean sits well above the median, dragged by the tail. On the log scale they very nearly coincide.

But: a log changes what a difference *means*. A gap of 0.7 in logs is a factor of two, whatever the starting point. If you log, say so, and report back on the original scale where you can.

Worked example

Reading the cohort's diagnostic scores, and whether logging helps.

```
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
hours = cohort["hours_practice"].to_numpy()

print(f"\n {hours.size}")
print(f"min {hours.min():.1f}    median {np.median(hours):.1f}    "
      f"mean {hours.mean():.1f}    max {hours.max():.1f}")
```

```
n 240
min 1.0    median 15.2    mean 16.4    max 36.2
```

Mean and median are close, so there is no sign of a long right tail here. A close match is evidence, not proof, of symmetry, but logging is unlikely to help, and that is worth checking rather than doing reflexively:

```
ratio = hours.mean() / np.median(hours)
print(f"mean / median = {ratio:.3f}")
print("strong right tail?" , ratio > 1.2)
```

```
mean / median = 1.082
strong right tail? False
```

Now a quantity where it does help — the same lognormal incomes:

```
for name, values in [("hours_practice", hours), ("simulated income", income)]:
    r = values.mean() / np.median(values)
    print(f"{name:<18} mean/median {r:5.2f}    log-scale ratio "
          f"{np.log(values).mean()/np.median(np.log(values)):5.2f}")
```

hours_practice	mean/median	1.08	log-scale ratio	0.97
simulated income	mean/median	1.67	log-scale ratio	1.00

The income data has a mean 65% above its median; after logging, the two agree to within a couple of per cent. The hours data showed no long tail, and logging does little for it.

The lesson is the check, not the transformation: **compare mean to median before deciding**. A log is a tool for a specific shape, not a routine step.

Try it yourself

Exercise 1 State the domain of each.

a. $f(x) = \frac{1}{x-3}$ b. $g(x) = \sqrt{x+2}$ c. $h(x) = \ln(x-1)$

Show the solution

- a. Everything except 3: $\{x \in \mathbb{R} : x \neq 3\}$. Division by zero.
- b. $x \geq -2$, that is $[-2, \infty)$. The argument of the root must be non-negative.
- c. $x > 1$, that is $(1, \infty)$. **Strictly** greater — $\ln 0$ is undefined, so 1 is excluded. That is the one people get wrong; the square root allows its endpoint and the logarithm does not.

Exercise 2 Solve $3^x = 50$ for x , and check your answer.

Show the solution

Take logs of both sides and use $\ln(a^b) = b \ln a$:

$$x \ln 3 = \ln 50 \implies x = \frac{\ln 50}{\ln 3} \approx 3.561$$

```
x = np.log(50) / np.log(3)
print(x, 3**x)           # 3.5609..., 50.0000...
```

Any base works as long as you use the same one on both sides — $\log_{10}$ gives the identical answer, because the two logs differ by a constant factor that cancels in the ratio.

Exercise 3 A quantity doubles every 3 years. Write it as a function of years t , then find how long until it is ten times its starting value.

Show the solution

$$N(t) = N_0 \cdot 2^{t/3}$$

Ten times the start means $2^{t/3} = 10$, so

$$\frac{t}{3} \ln 2 = \ln 10 \implies t = \frac{3 \ln 10}{\ln 2} \approx 9.97 \text{ years}$$

```
t = 3 * np.log(10) / np.log(2)
print(t, 2**(t/3))       # 9.966..., 10.0
```

Worth noticing: ten-fold takes only about 3.3 doublings, not ten. Compounding is consistently faster than intuition expects, which is exactly why exponentials are worth recognising on sight.

Check it in Python

Check an inverse actually inverts, over a range rather than at one point:

```
values = np.array([0.1, 1.0, 3.7, 250.0])
print(np.allclose(np.log(np.exp(values)), values))
print(np.allclose(np.exp(np.log(values)), values))
```

True

True

Check a log rule rather than trusting your memory of it:

```
a, b = 6.0, 2.5
checks = {
    "ln(ab) = ln a + ln b": (np.log(a*b), np.log(a) + np.log(b)),
    "ln(a/b) = ln a - ln b": (np.log(a/b), np.log(a) - np.log(b)),
    "ln(a^b) = b ln a": (np.log(a**b), b*np.log(a)),
    "ln(a+b) = ln a + ln b": (np.log(a+b), np.log(a) + np.log(b)),
}
for name, (left, right) in checks.items():
    print(f" {name:<26} {np.isclose(left, right)}")
```

$\ln(ab) = \ln a + \ln b$	True
$\ln(a/b) = \ln a - \ln b$	True
$\ln(a^b) = b \ln a$	True
$\ln(a+b) = \ln a + \ln b$	False

The last one is **false**, and deliberately included. There is no rule for the log of a sum, and inventing one is a standard error — the check catches it in one line.

Check the domain before applying a function to real data:

```
values = np.array([12.0, 0.0, 5.5, -3.0])
loggable = values > 0
print(f"{loggable.sum()} of {values.size} values can be logged")
print("logged:", np.log(values[loggable]).round(3))
```

2 of 4 values can be logged
logged: [2.485 1.705]

Filtering first is better than logging and then hunting for the $-\infty$ and nan that appear.

Common wrong turns

Where this usually goes wrong

Inventing a rule for $\ln(a + b)$ There isn't one. The rules cover products, quotients and powers — never sums.

Forgetting that $\ln$ needs a *strictly positive argument* $\sqrt{0}$ is fine; $\ln 0$ is not. Zero is a genuine value in plenty of datasets, and it is the one that breaks a log transform.

Assuming log means base 10 In NumPy and in mathematics it is base e . Calculators and spreadsheets often differ. Use `np.log10` when you mean base 10.

Logging by reflex A log transform is for right-skewed, multiplicative quantities. Compare mean to median first; if they agree, logging is unlikely to help and costs interpretability.

Reporting results on the log scale without saying so A difference of 0.7 in logs is a factor of two, not “0.7 more”. State the scale, and back-transform where you can.

Stating a rule without its domain “ $f(x) = 1/x$ ” is incomplete. The domain is part of the function, and it is where the errors live.

Check yourself

Answers are at the foot of the page.

- What is the domain of $\ln(x - 2)$?
 - $x \geq 2$
 - $x > 2$
 - $x \neq 2$
 - all reals
- Simplify $\ln(x^3) - \ln(x)$.
- Why does logging turn constant-percentage growth into a straight line?
- A variable has mean 4,200 and median 900. What does that suggest, and what would you try?
- Is $\ln(a + b) = \ln a + \ln b$ ever true for positive a and b ?

Show the answers

- (b)** $x > 2$. Strictly greater — the logarithm is undefined at zero, so $x = 2$ is excluded.
- $\ln(x^3) - \ln x = 3 \ln x - \ln x = 2 \ln x$. (Equivalently $\ln(x^3/x) = \ln(x^2)$.) Valid for $x > 0$.
- Because constant-percentage growth means multiplying by the same factor each period, and the log of a product is a sum of logs — so equal multiplications become equal additions, which is a straight line.
- A strong right skew: a few very large values dragging the mean far above the median. Try a log transform, then check whether mean and median come into agreement on the new scale.
- Yes, but only in special cases.** It needs $a + b = ab$, which for positive numbers holds only on a curve (for instance $a = b = 2$: $\ln 4 = \ln 2 + \ln 2$). For almost every pair it is false, so “no, not in general” is also a fair answer. It is not a rule, and treating it as one is a standard error.

Where this shows up next

M3 asks how fast a function changes, which is the start of everything that follows. **M7** integrates the exponential, and **S6** meets it again inside the normal density. The logistic curve returns whenever a model must output a probability.

Rates of change

Unit 3 · Lesson 4 of 4 · Mathematics · M3 Reading: 6 pages With practice: 2 hours Assumes: M1, M2

Why this lesson is here

Every model you will ever fit is chosen by asking one question over and over: *if I nudge this number slightly, does the model get better or worse?*

That question is a derivative. It is the same question whether you are tuning a single parameter by hand or letting a neural network tune ten million of them, and it is the idea that M4 to M7 are built on. Before any of that, though, a derivative is something much more ordinary: a way of saying how quickly one quantity responds to a change in another.

Before you start

You should be able to:

- read a graph and find the gradient of a straight line — M1 ;
- substitute a number into a formula — M1 ;
- say what the domain of a function is — M2 .

Quick check. What is the gradient of the straight line through (1, 3) and (3, 11)?

💡 Show the answer

$$\text{gradient} = \frac{11 - 3}{3 - 1} = \frac{8}{2} = 4.$$

If that took real effort, reread M1 before going on. Everything below is this calculation, applied to curves instead of lines.

The idea

Throughout these notes we use one dataset: a synthetic record of a previous cohort who worked through this material. Here are three of its columns.

```
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
cohort[["background", "hours_practice", "final_score"]].head()
```

	background	hours_practice	final_score
0	Mathematics or Physics	12.9	83.6
1	Life sciences	22.8	56.6
2	Engineering	1.7	58.5
3	Computing	7.5	67.5
4	Computing	13.5	73.8

Suppose we model how a student's final score responds to hours of practice. For one particular group of students, a reasonable-looking model is

$$s(h) = 45 + 6h - 0.25h^2$$

where h is hours of practice and s is the final score in marks.

Now ask: **how fast is the score improving?**

That question has no single answer until you say *where*. Between $h = 0$ and $h = 4$ the score climbs from 45 to 65, so it is improving at

$$\frac{s(4) - s(0)}{4 - 0} = \frac{65 - 45}{4} = 5 \text{ marks per hour.}$$

Between $h = 20$ and $h = 24$ the same model gives $s(20) = 65$ and $s(24) = 45$, an *average* of -5 marks per hour. The model says the same student, practising the same amount, is now going backwards. Same curve, two opposite answers, because we asked about two different places on it.

This quantity — change in output divided by change in input — is the **average rate of change**, and it always refers to an interval, never to a point.

Shrinking the interval

To talk about the rate at a *single* value of h , take shorter and shorter intervals around it. Both sides matter, so we shrink towards $h = 4$ from above and from below:

```
def s(h):
    return 45 + 6*h - 0.25*h**2

print(f"{'interval':>9}    {'from the right':>14}    {'from the left':>13}")
for step in [4, 2, 1, 0.5, 0.1, 0.01]:
    right = (s(4 + step) - s(4)) / step
    left = (s(4 - step) - s(4)) / (-step)
    print(f"{'step':>9}    {'right':>14.4f}    {'left':>13.4f}")
```

interval	from the right	from the left
4	3.0000	5.0000
2	3.5000	4.5000
1	3.7500	4.2500
0.5	3.8750	4.1250
0.1	3.9750	4.0250
0.01	3.9975	4.0025

Both columns are settling on 4, one from below and one from above. Neither ever *reaches* 4 — every row uses a genuine interval of some width — but both get and stay as close to 4 as you like, by taking the interval short enough. That the two sides agree is not a formality; later in this lesson you will meet a function where they do not.

That settled-on value is the **derivative** of s at $h = 4$, written $s'(4)$. We say $s'(4) = 4$ marks per hour.

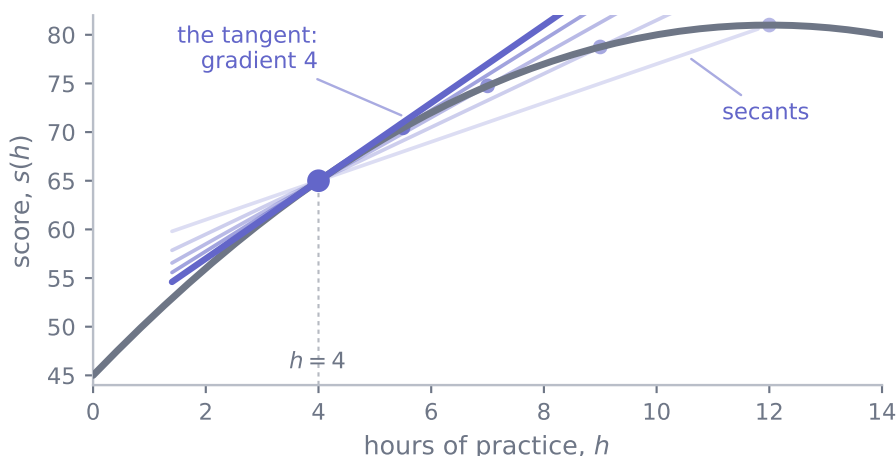


Figure 1: Each faint line joins $h = 4$ to a point further along the curve; its gradient is one row of the table above. As that second point slides in, the lines settle onto the single bold line — the tangent, whose gradient is 4.

i Two readings of the same number, both worth carrying:

Geometric. $s'(4)$ is the gradient of the *tangent* to the curve at $h = 4$. Each row of the table above is the gradient of a line through two points on the curve; as the second point slides towards the first, those lines settle into one limiting position, and that is the tangent. (It is tempting to define a tangent as the line that touches the curve once and does not cross it. That is a useful picture here but not a definition — some tangents do cross their curve at the point of contact.)

Practical. $s'(4)$ is the *sensitivity* of the score to practice at that point: one more hour buys roughly 4 more marks. This is the reading you will use constantly from M6 onwards, and it is the reason a derivative carries the units of the output divided by the units of the input — here, marks per hour.

The mechanics

The difference quotient

For a function f and a point a , the average rate of change over an interval of width h is

$$\frac{f(a + h) - f(a)}{h}$$

which is called the **difference quotient**. The derivative is the value it settles on as h shrinks towards zero, written

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a + h) - f(a)}{h}.$$

Read $\lim_{h \rightarrow 0}$ as “the value this settles on as h gets arbitrarily small”. You cannot simply put $h = 0$: that gives $0/0$, which is not a number. The whole point of the notation is to say *approach* zero without arriving.

! The symbol h is doing two different jobs on this page: it is hours of practice in the worked model, and interval width in the general definition. That collision is normal in mathematics and worth noticing rather than tripping over. In the general definition, h is always the interval width.

Notation

All of these mean the same thing:

$$f'(x) \quad \frac{df}{dx} \quad \frac{d}{dx}f(x)$$

The first is compact. The second is useful because it names the input variable, which matters when there is more than one — as there will be in [M6](#).

Doing it from first principles

Take $f(x) = x^2$. Then

$$\begin{aligned} \frac{f(x+h) - f(x)}{h} &= \frac{(x+h)^2 - x^2}{h} \\ &= \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \frac{2xh + h^2}{h} \\ &= 2x + h. \end{aligned}$$

The second line multiplies out the bracket: $(x+h)^2 = (x+h)(x+h) = x^2 + xh + hx + h^2 = x^2 + 2xh + h^2$. The last step is legitimate because h is small but *not* zero, so dividing by it is allowed. Now let h shrink: the expression settles on $2x$. So

$$f(x) = x^2 \implies f'(x) = 2x.$$

The same three moves — expand, cancel the h in the denominator, let what remains shrink — handle every first-principles derivative you will be asked for. In [M4](#) you will get rules that make this unnecessary, but doing it by hand two or three times first is what makes those rules mean something.

When there is no derivative

A derivative does not always exist. The clearest failure is a corner. Take $f(x) = |x|$, the **absolute value** (the size of x ignoring its sign, `abs(x)` in Python), at $x = 0$: approaching from the right, the difference quotient is always $+1$; from the left it is always -1 . The two sides disagree, so nothing settles, so $f'(0)$ does not exist.

```
for step in [1, 0.1, 0.001]:
    right = (abs(0 + step) - abs(0)) / step
    left = (abs(0 - step) - abs(0)) / (-step)
    print(f"step {step:<7} from the right {right:+.0f}    from the left
          {left:+.0f}")
```

step 1	from the right +1	from the left -1
step 0.1	from the right +1	from the left -1
step 0.001	from the right +1	from the left -1

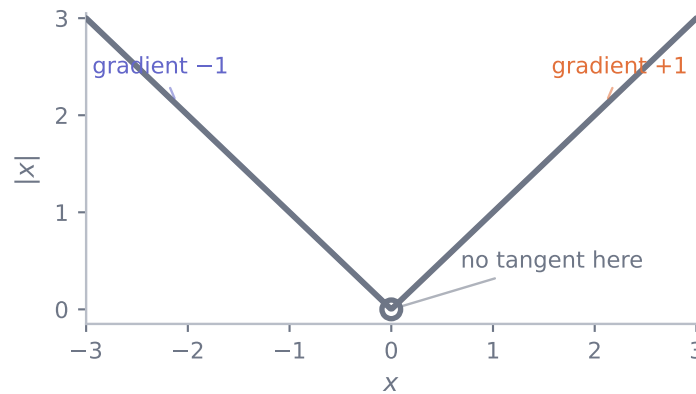


Figure 2: At every point except the origin, $|x|$ has an unambiguous tangent. At $x = 0$ the two sides disagree — one arrives with gradient -1 , the other with $+1$ — so no single line fits, and there is no derivative there.

The function is perfectly well behaved — **continuous** (its graph can be drawn without lifting the pen), defined everywhere, easy to draw. It has no single tangent at that point. Continuity is not enough for differentiability, and that gap matters later: some loss functions used in practice have exactly this kind of kink.

The tangent line

Once you have $f'(a)$, you have the straight line that best matches f near a :

$$f(x) \approx f(a) + f'(a)(x - a).$$

This is the **local linear approximation**. It says: start from the value at a , then move along at the rate you measured there. It is exact at $x = a$ and gets worse as you move away, and it is the single most useful approximation in applied mathematics — [M6](#) is essentially this idea in many variables at once.

Worked example

The model above has a best value, and the derivative finds it.

Our model was $s(h) = 45 + 6h - 0.25h^2$. Differentiate from first principles.

$$\begin{aligned} \frac{s(h+t) - s(h)}{t} &= \frac{[45 + 6(h+t) - 0.25(h+t)^2] - [45 + 6h - 0.25h^2]}{t} \\ &= \frac{6t - 0.25(2ht + t^2)}{t} \\ &= 6 - 0.5h - 0.25t. \end{aligned}$$

(Here t is the interval width, since h is already in use.) Letting t shrink gives

$$s'(h) = 6 - 0.5h.$$

Now read it:

Table 1: Sensitivity of the model at four values of h

h	$s'(h)$	what it says
4	+4.0	an extra hour is worth about 4 marks
10	+1.0	still improving, but four times more slowly
12	0.0	the model has stopped improving at this instant
20	-4.0	the model now predicts the score <i>falling</i>

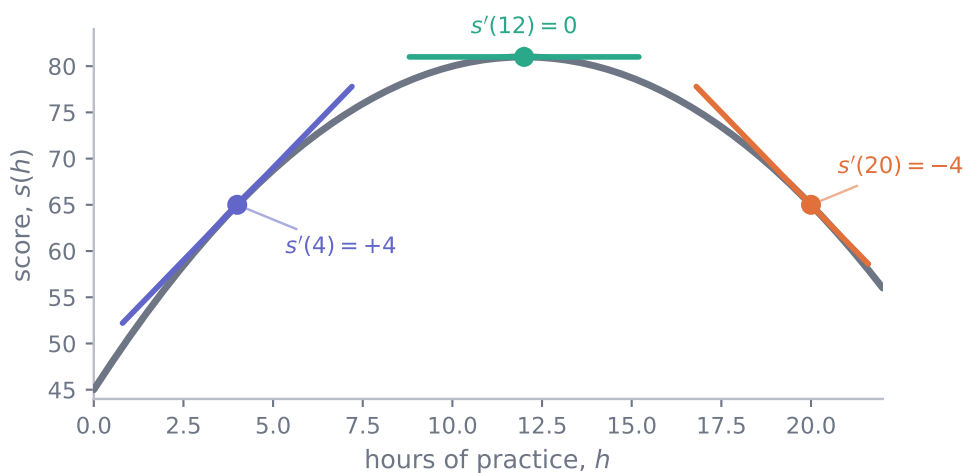


Figure 3: The same curve with its tangent drawn at three places. The gradient falls steadily, reaches zero at $h = 12$ where the curve is momentarily flat, and is negative beyond it.

Two things are worth saying about that last row, and both are habits rather than calculations.

First, solving $s'(h) = 0$ is how you find *candidates* for a best value without plotting anything. A zero derivative marks a place where the curve is momentarily flat; whether that is a maximum, a minimum or neither is a further question, and so is whether the true best value sits at the edge of the allowed range instead. Here the model is a downward parabola, so $h = 12$ really is its peak. Doing that properly is the whole of M5.

Note also what $s'(12) = 0$ does *not* say. It is an instantaneous rate, not the value of one more hour: $s(13) - s(12) = -0.25$ marks. The distinction between a rate at a point and a change over an interval is the same one this lesson opened with, and it does not go away once you have a derivative.

Second, the model predicts that practising for 20 hours is worse than practising for 12. Nothing in the data forces that; it is an artefact of having chosen a downward parabola, which must eventually come down. A derivative tells you what your *model* does. Whether the model deserves to be believed out there is a separate question, and one that S11 takes seriously.

Try it yourself

Exercise 1 From first principles, differentiate $f(x) = 5x - 2$.

 Show the solution

$$\frac{f(x+h) - f(x)}{h} = \frac{[5(x+h) - 2] - [5x - 2]}{h} = \frac{5h}{h} = 5.$$

There is no h left to shrink, so $f'(x) = 5$ for every x . That is what you should expect: the graph is a straight line of gradient 5, and its tangent is itself.

Exercise 2 From first principles, differentiate $f(x) = x^2 - 3x$.

 Show the solution

$$\begin{aligned} \frac{f(x+h) - f(x)}{h} &= \frac{[(x+h)^2 - 3(x+h)] - [x^2 - 3x]}{h} \\ &= \frac{2xh + h^2 - 3h}{h} \\ &= 2x + h - 3. \end{aligned}$$

As h shrinks this settles on $f'(x) = 2x - 3$.

Sanity check: at $x = 1.5$ the derivative is zero. Since $f(x) = x(x - 3)$, the curve crosses zero at $x = 0$ and $x = 3$, and a parabola is symmetric, so its bottom sits halfway between them — at $x = 1.5$.

Exercise 3 A university models the cost of running a module as a smooth function $C(n)$ of the number of students enrolled, in pounds. (Enrolment is a whole number, so C is a continuous stand-in for something discrete — a very common move, and one worth noticing.) You are told that $C'(100) = 42$.

Write down, in one sentence and with units, what that means. Then say what it does **not** tell you.

 Show the solution

It means that at an enrolment of 100 students, the modelled cost is rising at **£42 per additional student**. Using the local linear approximation, the 101st student is expected to add roughly £42 — *roughly*, because the derivative describes the rate at exactly $n = 100$, while the 101st student is a whole unit away.

It does not tell you the total cost $C(100)$: a rate says nothing about a level. It also says nothing about the 200th student, because the rate can change — indeed it usually does, which is the entire reason we need derivatives rather than a single number.

Check it in Python

You will rarely differentiate by hand in practice, but you should always be able to check that you did it right. The tool is a **centred difference**: evaluate a small step either side and take the gradient between them.

```
import numpy as np

def s(h):
    return 45 + 6*h - 0.25*h**2

def s_prime(h):
    # our by-hand answer
```

```

    return 6 - 0.5*h

def centred_difference(f, x, step):
    return (f(x + step) - f(x - step)) / (2 * step)

for h in [4, 10, 12, 20]:
    approx = centred_difference(s, h, 1e-6)
    exact = s_prime(h)
    print(f"h={h:>3}    numerical {approx:+.6f}    by hand {exact:+.6f}    agree:
    ↳ {np.isclose(approx, exact)}")

```

```

h= 4    numerical +4.000000    by hand +4.000000    agree: True
h= 10   numerical +1.000000    by hand +1.000000    agree: True
h= 12   numerical +0.000000    by hand +0.000000    agree: True
h= 20   numerical -4.000000    by hand -4.000000    agree: True

```

Agreement at four points is evidence, not proof. A disagreement is worth taking seriously, but it is not by itself proof that the algebra is wrong — as the next block shows, a perfectly correct derivative can disagree with a badly chosen numerical check.

One trap. It is tempting to think that a smaller step is always better. It is not:

```

for step in [1e-1, 1e-3, 1e-6, 1e-10, 1e-14]:
    approx = centred_difference(s, 4.0, step)
    print(f"step {step:<8.0e} estimate {approx:.10f}    error {abs(approx -
    ↳ 4.0):.3e}")

```

```

step 1e-01    estimate 4.0000000000    error 8.527e-14
step 1e-03    estimate 4.0000000000    error 4.889e-12
step 1e-06    estimate 3.9999999970    error 2.994e-09
step 1e-10    estimate 3.9999292767    error 7.072e-05
step 1e-14    estimate 4.2632564146    error 2.633e-01

```

The error gets *worse* as the step shrinks, and by 10^{-14} the estimate is wrong in the first decimal place. The reason is that you are asking the computer to subtract two numbers that agree to fourteen digits; almost all of the digits cancel, and what survives is mostly noise. [P2](#) explains why computers do this.

There is a second effect that this particular model happens to hide. Our s is a parabola, and for a parabola the two sides of a centred difference cancel exactly, so the *method* contributes no error at all whatever step you choose. Everything in that table is the arithmetic, not the mathematics.

On a curve that bends more sharply the cancellation is only partial, and a large step then measures the wrong thing because the curve changes direction across the interval. In general both ends are bad and the usable range sits in the middle — around 10^{-6} when x is of a size near 1. For much larger or smaller values, scale the step to x rather than using a fixed number, and check that the answer is stable across a few step sizes before trusting it.

Common wrong turns

Where this usually goes wrong

Reading $f'(a)$ as a value of f $f(4) = 65$ marks. $f'(4) = 4$ marks per hour. Different quantities, different units. The rule is that a derivative carries the units of the output divided by the units of the input — so if the input is a plain number with no units of its own, the derivative does share the output's units, and only then.

Reading the derivative off the height of the graph A curve can be very high and perfectly flat — high f , zero f' — or close to zero and plunging. The derivative is about *steepness*, and the two are unrelated.

Assuming every function has a derivative everywhere $|x|$ is continuous at 0 and has no derivative there. Continuity is necessary but not sufficient.

Dropping the units “The derivative is 4” is almost useless. “4 marks per hour of practice” can be checked against reality, argued with, and acted on. Carry the units through every line.

Believing the model because the calculus was correct $s'(20) = -4$ is a flawless statement about the parabola we chose. Whether practice really harms scores after 12 hours is a question about the world, and the derivative has no opinion on it.

Check yourself

Answers are at the foot of the page. Try to give a reason as well as an answer — if you cannot say *why*, treat it as wrong.

1. A function has $f(2) = 10$ and $f'(2) = -3$. Using the local linear approximation, what is your estimate of $f(2.1)$?
 - a. 10.3
 - b. 9.7
 - c. -3
 - d. The approximation cannot be used here
2. Water is draining from a tank. $V(t)$ is the volume in litres after t minutes. What are the units of $V'(t)$, and what sign would you expect?
3. True or false: if f is continuous at $x = 0$ then $f'(0)$ exists.
4. For $f(x) = x^2$, the difference quotient simplified to $2x + h$. Why is it wrong to say “set $h = 0$ in the original quotient” rather than “let h shrink”?
5. Your by-hand derivative and a centred difference disagree in the fourth decimal place. Name two quite different explanations.



Show the answers

1. **(b), 9.7.** The approximation gives $f(2) + f'(2)(0.1) = 10 - 0.3 = 9.7$. Option (a) gets the sign of the rate wrong; (c) confuses a rate with a value. Note that the question asks for an *estimate*: two facts about f at a single point cannot determine its value anywhere else. A sharply curving f could pass through $(2, 10)$ with gradient -3 and still be nowhere near 9.7 at $x = 2.1$.
2. **Litres per minute, and negative.** The tank is emptying, so volume is decreasing, so the rate of change is below zero.
3. **False.** $f(x) = |x|$ is the standard counterexample: continuous at 0, no derivative there.
4. Because at $h = 0$ the original quotient reads $0/0$, which is not a number — you cannot

divide by zero. The cancellation that produced $2x + h$ is only valid *because* $h \neq 0$. The limit describes where the values head, and deliberately never evaluates at the forbidden point.

5. Either the algebra is wrong, or the numerical check is: a step size too large (the curve bends across the interval) or too small (floating-point cancellation). Rerun the check at several step sizes before suspecting the algebra — the plot of error against step size will tell you which it is.

Where this shows up next

M4 gives you rules — power, product, chain — so you never have to do first principles again. **M5** sets $f'(x) = 0$ to find best values, which is the calculation behind fitting any model. **M6** does all of this with several variables at once and arrives at gradient descent. **S11** uses it to fit a straight line to the cohort data, at which point the mathematics, the statistics and the Python all turn out to be the same piece of work.

The differentiation toolkit

Unit 4 · Lesson 3 of 4 · Mathematics · M4 Reading: 3 pages With practice: ⌕ 2 hours Assumes: M3

Why this lesson is here

M3 defined the derivative and made you compute a few from first principles. That was necessary once and is unbearable as a habit.

This lesson replaces it with six rules. After them you will be able to differentiate essentially any function you meet in this course in a line or two — including the ones that appear inside loss functions, which is where all of this is heading.

Before you start

You should be able to:

- state what $f'(a)$ means, in words and in units — M3;
- compute a derivative from the difference quotient for a quadratic — M3;
- work with powers, roots and logarithms — M2.

Quick check. In M3 we found from first principles that $f(x) = x^2$ has $f'(x) = 2x$. Without doing any work, what do you expect $f'(x)$ to be for $f(x) = x^3$?

💡 Show the answer

$3x^2$. If you spotted the pattern — bring the power down, then reduce it by one — you have already guessed the first and most useful rule below.
If you did not, no matter. That is what this lesson is for.

The idea

Every complicated function is built from simple ones by a small number of moves: adding them, multiplying them, dividing them, and feeding one into another.

The rules below mirror those moves exactly. There is a rule for a sum, a rule for a product, a rule for a quotient, and a rule for a composition. So differentiating a complicated function is a matter of noticing **how it was built**, and then applying the matching rule.

That is the whole method, and it is worth saying plainly because students often try to memorise the rules as a list of unrelated facts. They are not a list. They are a set of instructions for taking a function apart.

So before differentiating anything, ask one question: *what is the outermost thing being done here?* Is this fundamentally a product of two things, or a quotient, or one function applied to another? Answer that, and the rule chooses itself.

The mechanics

The six rules

Let f and g be differentiable, and let c and n be constants.

	Rule	
Constant	$(c)' = 0$	a flat line has no slope
Power	$(x^n)' = n x^{n-1}$	any real n , wherever x^n is defined and differentiable
Sum	$(f + g)' = f' + g'$	and $(cf)' = cf'$
Product	$(fg)' = f'g + fg'$	not $f'g'$
Quotient	$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$	order matters in the numerator
Chain	$(f(g(x)))' = f'(g(x)) \cdot g'(x)$	differentiate the outside, times the inside

Two more you will need constantly, which we take as given here:

$$(e^x)' = e^x \quad (\ln x)' = \frac{1}{x} \quad (x > 0)$$

The first is the defining property of e^x : it is the function that is its own rate of change. That is the entire reason e appears everywhere in this subject.

The chain rule deserves a paragraph to itself

It is the one people get wrong, and it is by far the most used — every neural network in existence is trained by applying it repeatedly.

The idea is that a composition is a two-stage process. In $(2x + 1)^5$, the inside stage turns x into $2x + 1$, and the outside stage raises that to the fifth power. If x moves a little, the inside moves 2 times as much, and the outside moves $5(\text{inside})^4$ times as much as the inside did. The two sensitivities **multiply**:

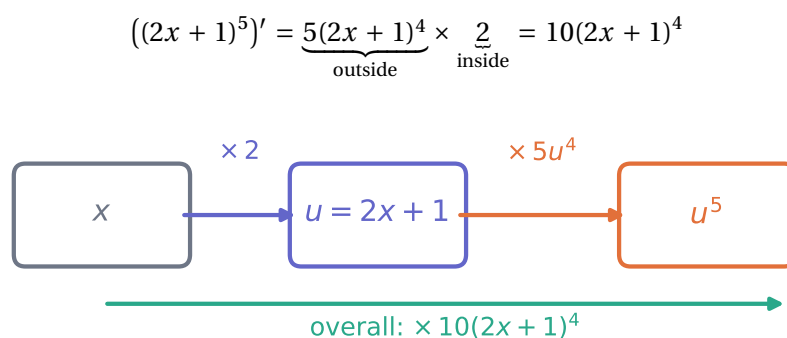


Figure 1: The chain rule as two stages in series. A nudge of 1 at the input becomes a nudge of 2 after the inside stage, and each unit of that becomes $5u^4$ after the outside stage. The sensitivities compound, so they multiply.

A reliable procedure, if you find it slippery: name the inside. Let $u = 2x + 1$. Then the function is u^5 , and

$$\frac{d}{dx} u^5 = 5u^4 \cdot \frac{du}{dx} = 5(2x+1)^4 \cdot 2.$$

Forgetting the $\cdot du/dx$ is the single commonest error in first-year calculus.

Second derivatives

Differentiating f' gives f'' , the rate at which the *rate* is changing. It measures curvature: $f'' > 0$ means the slope is increasing, so the curve bends upwards. You will use it in **M5** to tell a minimum from a maximum.

Worked example

One function, taken apart.

$$f(x) = e^{-x}(x^3 + 2x + 2)$$

Ask the question first: what is the outermost operation? It is a **product** — e^{-x} times a **polynomial** (a sum of powers of x , each times a number). So the product rule chooses itself, and we will need the chain rule to handle e^{-x} along the way.

The two pieces and their derivatives:

$$\begin{array}{ll} u = e^{-x} & u' = e^{-x} \cdot (-1) = -e^{-x} \quad (\text{chain rule: inside is } -x) \\ v = x^3 + 2x + 2 & v' = 3x^2 + 2 \quad (\text{power and sum rules}) \end{array}$$

Now assemble with $(uv)' = u'v + uv'$:

$$\begin{aligned} f'(x) &= -e^{-x}(x^3 + 2x + 2) + e^{-x}(3x^2 + 2) \\ &= e^{-x}[-(x^3 + 2x + 2) + 3x^2 + 2] \\ &= e^{-x}(-x^3 + 3x^2 - 2x) \\ &= -e^{-x}(x^3 - 3x^2 + 2x) \\ &= -e^{-x} x(x-1)(x-2). \end{aligned}$$

The last line takes out the common factor x , leaving $x^2 - 3x + 2$, which is $(x-1)(x-2)$: multiply those brackets out to check. That last factorisation is worth the extra line. It says at a glance that $f'(x) = 0$ exactly when x is 0, 1 or 2, because e^{-x} is never zero. Those three values are precisely what **M5** will need. A derivative left as an unfactored mess hides that; a factored one hands it to you.

i And the model from M3. We laboured through $s(h) = 45 + 6h - 0.25h^2$ from first principles over most of a page. With the rules it is one line: the constant goes to 0, $6h$ goes to 6, and $-0.25h^2$ goes to $-0.5h$, giving $s'(h) = 6 - 0.5h$. Same answer, about thirty seconds.

Try it yourself

Exercise 1 Differentiate $f(x) = \frac{x}{3x+2}$.

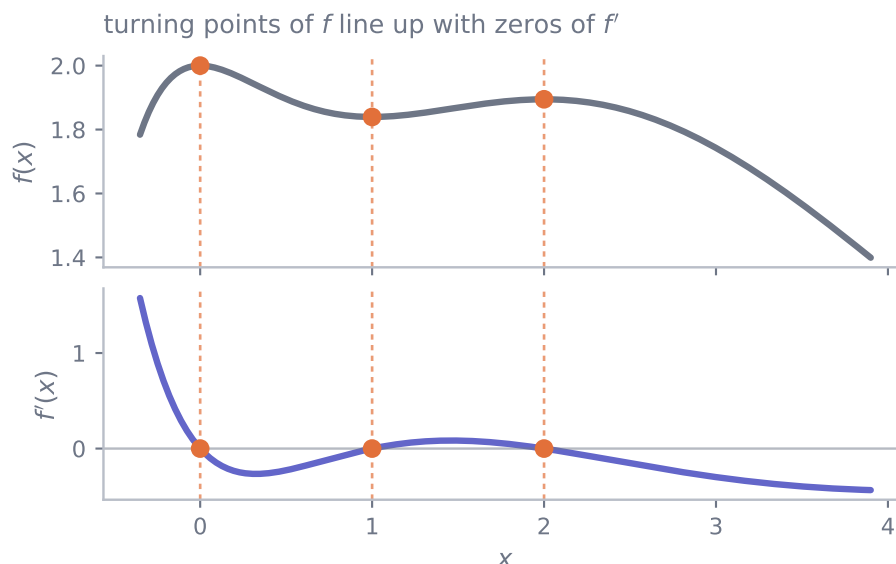


Figure 2: The function above its own derivative. Wherever f turns — at $x = 0, 1$ and 2 — the derivative crosses zero. That is the entire content of M5, visible in one picture.

💡 Show the solution

Outermost operation: a quotient. With $u = x$ and $v = 3x + 2$, so $u' = 1$ and $v' = 3$:

$$f'(x) = \frac{u'v - uv'}{v^2} = \frac{1 \cdot (3x + 2) - x \cdot 3}{(3x + 2)^2} = \frac{3x + 2 - 3x}{(3x + 2)^2} = \frac{2}{(3x + 2)^2}.$$

Worth noticing: the answer is positive everywhere it is defined. Be careful how you say that, though. The function is undefined at $x = -2/3$, so its domain comes in two separate pieces, and it is increasing *on each piece* — not across the gap. Comparing $f(-1) = 1$ with $f(0) = 0$ shows why: the value drops as x rises, because the two points sit either side of the break.

Exercise 2 Differentiate $g(x) = \ln(x^2 + 1)$.

💡 Show the solution

Outermost operation: a composition — a logarithm applied to something. Let $u = x^2 + 1$, so $g = \ln u$ and $u' = 2x$:

$$g'(x) = \frac{1}{u} \cdot u' = \frac{2x}{x^2 + 1}.$$

The commonest wrong answer is $1/(x^2 + 1)$, which drops the $\cdot u'$. If you made it, that is the error to watch for from now on — it is the chain rule's signature failure.

Exercise 3 A model's squared error for one observation is $L(a) = (a - 7)^2$, where a is a parameter you get to choose. Differentiate L with respect to a , then find where $L'(a) = 0$ and say in words what that value of a is doing.

💡 Show the solution

Chain rule, with inside $u = a - 7$ and $u' = 1$:

$$L'(a) = 2(a - 7) \cdot 1 = 2(a - 7).$$

Setting $L'(a) = 0$ gives $a = 7$.

In words: $a = 7$ is the choice of parameter that makes the squared error stop changing — and since squaring makes L non-negative and $L(7) = 0$, it is the choice that makes the error as small as it can possibly be. You have just fitted a one-parameter model by calculus, which is the whole idea that [M5](#), [M6](#) and [M11](#) develop.

Check it in Python

Differentiating by hand is error-prone, and a numerical check costs two lines. This is the same centred difference from [M3](#), now used as a routine tool.

```
import numpy as np

def centred_difference(f, x, step=1e-6):
    return (f(x + step) - f(x - step)) / (2 * step)

def f(x):
    # e^-x (x^3 + 2x + 2)
    return np.exp(-x) * (x**3 + 2*x + 2)

def f_prime(x):
    # our by-hand answer
    return -np.exp(-x) * x * (x - 1) * (x - 2)

for x in [0.5, 1.5, 3.0]:
    numeric, symbolic = centred_difference(f, x), f_prime(x)
    print(f"x={x:<4} numeric {numeric:+.8f} by hand {symbolic:+.8f} "
          f"agree: {np.isclose(numeric, symbolic)}")
```

```
x=0.5  numeric -0.22744900  by hand -0.22744900  agree: True
x=1.5  numeric +0.08367381  by hand +0.08367381  agree: True
x=3.0  numeric -0.29872241  by hand -0.29872241  agree: True
```

Now use it on the three exercises at once — the same two lines, three times:

```
# lambda x: expression is a one-line function, like def f(x): return expression
checks = [
    ("x/(3x+2)",      lambda x: x/(3*x + 2),      lambda x: 2/(3*x + 2)**2,  1.0),
    ("ln(x^2+1)",     lambda x: np.log(x**2 + 1), lambda x: 2*x/(x**2 + 1),  3.0),
    ("(a-7)^2",       lambda a: (a - 7)**2,       lambda a: 2*(a - 7),      4.0),
]

for name, fn, derivative, at in checks:
    numeric, symbolic = centred_difference(fn, at), derivative(at)
    print(f"{name:<12} at x={at}: {numeric:+.6f} vs {symbolic:+.6f} "
          f"agree: {np.isclose(numeric, symbolic)}")
```

```
x/(3x+2)      at x=1.0:  +0.080000 vs +0.080000  agree: True
```

```
ln(x^2+1)    at x=3.0:  +0.600000 vs +0.600000    agree: True
(a-7)^2      at x=4.0:  -6.000000 vs -6.000000    agree: True
```

Try deliberately breaking one. Change the $\ln(x^2+1)$ derivative to `lambda x: 1/(x**2 + 1)` — the chain-rule mistake from Exercise 2 — and watch the check fail. That is what it is for: it will not tell you the right answer, but it will tell you, in one second, that you do not yet have it.

Common wrong turns

Where this usually goes wrong

Forgetting the inside derivative $(\ln(x^2 + 1))'$ is $\frac{2x}{x^2+1}$, not $\frac{1}{x^2+1}$. Every chain-rule application ends in $\times u'$. If your answer does not contain a factor that came from the inside, you have dropped it.

Assuming $(fg)' = f'g'$ It is $f'g + fg'$. Test it on $f = g = x$: the product is x^2 with derivative $2x$, while $f'g' = 1$. Not the same.

Getting the quotient rule backwards The numerator is $f'g - fg'$, in that order. Swapping the terms flips the sign of every answer. If you struggle to keep it straight, rewrite f/g as $f \cdot g^{-1}$ and use the product and chain rules instead.

Applying the power rule to e^x $(e^x)' = e^x$, not $x e^{x-1}$. The power rule is for a *variable base and constant power*; e^x is the other way round.

Reaching for a rule before looking at the structure Ask what the outermost operation is first. Choosing the rule is most of the work; applying it is bookkeeping.

Leaving the answer unfactored $e^{-x}(-x^3 + 3x^2 - 2x)$ and $-e^{-x}x(x-1)(x-2)$ are the same function. Only the second one tells you where the derivative is zero, which is usually the reason you differentiated.

Check yourself

Answers are at the foot of the page.

1. What is the derivative of $f(x) = \sqrt{x}$?

- a. $\frac{1}{2\sqrt{x}}$
- b. $2\sqrt{x}$
- c. $\frac{1}{2}x^{1/2}$
- d. $\sqrt{x}$

2. Differentiate $y = (3x - 4)^7$.

3. A student writes $(x^2 e^x)' = 2x e^x$. What did they do wrong, and what is the correct answer?

4. Explain, in terms of *sensitivity*, why the chain rule multiplies rather than adds.

5. Your numerical check disagrees with your algebra at $x = 2$ but agrees at $x = 1$ and $x = 5$. Is that more likely to be an algebra error or a numerical one?

 Show the answers

1. **(a), for $x > 0$.** Write $\sqrt{x} = x^{1/2}$ and apply the power rule with $n = \frac{1}{2}$: $\frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}}$. The restriction matters — the expression is undefined at $x = 0$, where $\sqrt{x}$ is defined but has no finite slope. Option (c) is the common slip of forgetting to reduce the power.
2. $y' = 7(3x - 4)^6 \cdot 3 = 21(3x - 4)^6$. The $\cdot 3$ is the inside derivative.
3. They applied the power rule to x^2 and left e^x alone, as though it were a constant. It is not — it is a **product** of two functions of x , so: $(x^2 e^x)' = 2x e^x + x^2 e^x = x e^x (2 + x)$. Multiplying the two derivatives, $(x^2)'(e^x)' = 2x e^x$, lands on the same wrong answer by a different slip, and has the same fix.
4. Because the two stages compound. If the inside responds twice as fast as the input, and the outside responds five times as fast as the inside, then the whole thing responds ten times as fast as the input — not seven.
5. There is not enough evidence to say, and that is the honest answer. Algebra is the first thing to suspect: a factorisation that divided by something, or a cancelled term that happens to vanish at $x = 2$. But a numerical check can also fail at a single point — if the function is not differentiable there, if the two evaluations nearly cancel, or if $x = 2$ sits close to a domain boundary. Test several step sizes at $x = 2$ before deciding: if the disagreement shrinks as the step changes it is numerical, and if it holds steady it is algebra.

Where this shows up next

M5 sets $f'(x) = 0$ and works out which of the resulting candidates is actually a best value — the factorised derivative from this lesson's worked example is exactly what it starts from. **M6** applies the same rules one variable at a time, which is all a partial derivative is, and arrives at gradient descent. **S11** uses Exercise 3's calculation, with two parameters instead of one, to fit a line to data.

Finding the best value

Unit 5 · Lesson 1 of 3 · Mathematics · M5 Reading: 5 pages With practice: 2 hours Assumes: M4

Why this lesson is here

Fitting a model means choosing the parameter values that make it *least wrong*. That is an optimisation problem, and this lesson is where you learn to solve one.

The central fact is small and easy to misuse: at a peak or a trough of a smooth curve, the derivative is zero. What takes care is everything around it — because a zero derivative is a *candidate*, not an answer, and the best value is sometimes nowhere near one.

Before you start

You should be able to:

- differentiate using the rules, and factorise the result — M4;
- read $f'(x)$ as a rate of change — M3.

Quick check. Differentiate $f(x) = x^2 - 6x + 5$ and find where $f'(x) = 0$.

💡 Show the answer

$f'(x) = 2x - 6$, which is zero at $x = 3$.

That is the whole method in one line. The rest of this lesson is about what $x = 3$ actually *is*, and when this method misses the answer entirely.

The idea

At the top of a hill the ground is momentarily level. At the bottom of a valley it is level too. So if a smooth curve has a highest or lowest point somewhere in the middle of its range, the tangent there is horizontal — the derivative is zero.

A point where $f'(x) = 0$ is called a **stationary point**. Finding them is easy. Deciding what they are is the work:

The middle one is the warning. Its tangent is horizontal, so $f'(x) = 0$, and yet it is neither a peak nor a trough — the curve flattens and carries on in the same direction.

The mechanics

The second-derivative test

f'' measures how the slope is changing, which is exactly what distinguishes a peak from a trough. If $f'(c) = 0$ and f'' is continuous near c :

$f''(c)$	curve bends	c is a
> 0	upwards	local minimum

$f''(c)$	curve bends	c is a
< 0	downwards	local maximum
$= 0$	—	test says nothing

```
def f(x): return x**2 - 6*x + 5
def f1(x): return 2*x - 6
def f2(x): return 2.0

print(f"f'(3) = {f1(3)}, f''(3) = {f2(3)} > 0 → minimum")
print(f"f(3) = {f(3)}")
```

$f'(3) = 0$, $f''(3) = 2.0 > 0 \rightarrow \text{minimum}$
 $f(3) = -4$

When $f''(c) = 0$ the test is genuinely inconclusive — not “probably a minimum”. Both x^4 (a minimum) and x^3 (neither) have $f'(0) = f''(0) = 0$:

```
for name, g in [("x^4", lambda x: x**4), ("x^3", lambda x: x**3)]:
    left, centre, right = g(-0.1), g(0.0), g(0.1)
    print(f"{name}: f(-0.1)={left:+.5f} f(0)={centre:+.5f} f(0.1)={right:+.5f}")
```

x^4 : $f(-0.1)=+0.00010$ $f(0)=+0.00000$ $f(0.1)=+0.00010$
 x^3 : $f(-0.1)=-0.00100$ $f(0)=+0.00000$ $f(0.1)=+0.00100$

For x^4 both neighbours are above the centre — a minimum. For x^3 one is above and one below — neither. When the test fails, look at the values either side.

Local is not global

A **local** minimum is lower than its immediate neighbours. A **global** minimum is the lowest anywhere in the range you care about. They need not coincide, and the global one may sit at an **endpoint**, where the derivative is not zero at all:

The full procedure

To find the best value of f on an interval $[a, b]$, collect **every** candidate and compare:

1. stationary points — where $f'(x) = 0$;
2. points where f' does not exist — corners, like $|x|$ at 0;
3. the endpoints a and b .

Then evaluate f at all of them and pick the largest or smallest. That is the whole method, and steps 2 and 3 are the ones people skip.

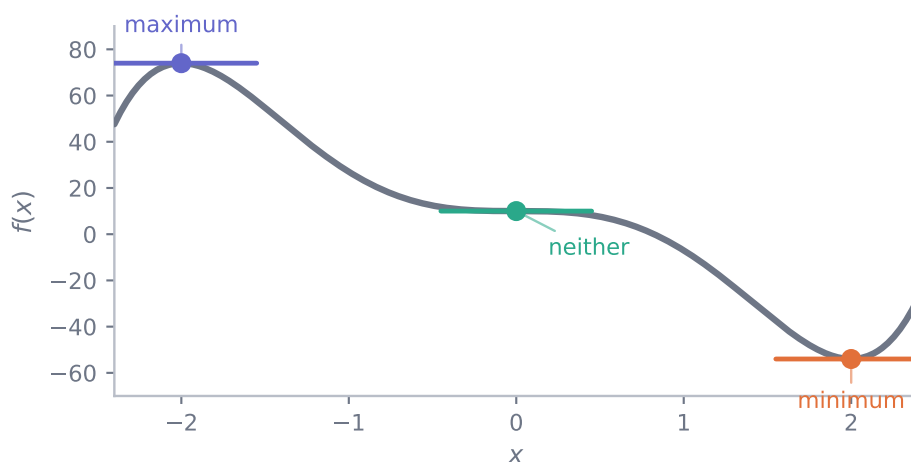


Figure 1: Three stationary points, all with $f'(x) = 0$. Only two are turning points, and the third is neither a maximum nor a minimum — which is why a zero derivative is a candidate rather than an answer.

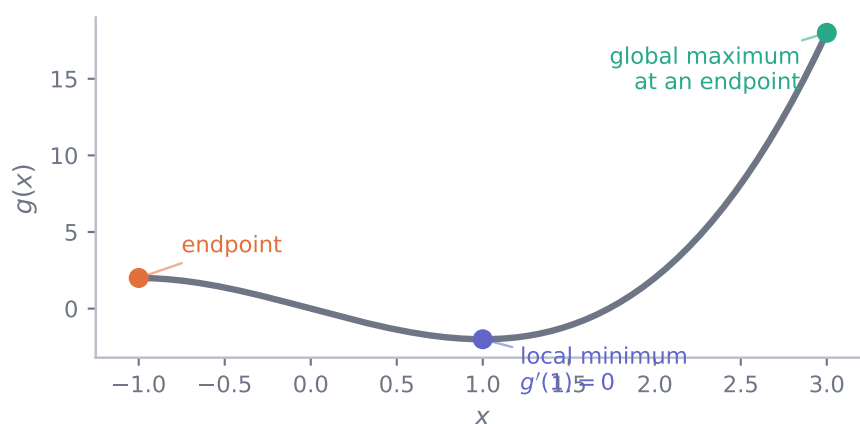


Figure 2: On $[-1, 3]$ the *highest* point is the right-hand endpoint, where the derivative is not zero at all. Checking stationary points alone would have missed it entirely — the only interior stationary point is the local minimum.

```
def g(x): return x**3 - 3*x
def g1(x): return 3*x**2 - 3

candidates = [-1.0, 1.0, 3.0]          # endpoints and the stationary point in
↳ range
for c in sorted(candidates):
    print(f" x={c:>4}   g(x)={g(c):>7.2f}   g'(x)={g1(c):>6.2f}")

best = min(candidates, key=g)
print(f"\nlowest on [-1, 3]: x={best}, g={g(best):.2f}")
```

```
x=-1.0   g(x)=   2.00   g'(x)=   0.00
x= 1.0   g(x)=  -2.00   g'(x)=   0.00
x= 3.0   g(x)=  18.00   g'(x)=  24.00
```

```
lowest on [-1, 3]: x=1.0, g=-2.00
```

A detail worth noticing. $g'(x) = 3x^2 - 3$ is zero at $x = \pm 1$, so *both* are stationary points — and $x = -1$ happens also to be the left endpoint, so it arrives on the list twice over. Only $x = 1$ is a genuinely interior stationary point, and it is not the answer: the largest value sits at $x = 3$, where the derivative is 24.

Worked example

Fitting a one-parameter model by calculus.

Suppose you must summarise a set of scores with a single number a , and you choose a to make the total squared error as small as possible:

$$L(a) = \sum_{i=1}^n (x_i - a)^2$$

Differentiate with respect to a , using the chain rule on each term:

$$L'(a) = \sum_{i=1}^n 2(x_i - a)(-1) = -2 \sum_{i=1}^n (x_i - a)$$

Set it to zero:

$$\sum_{i=1}^n (x_i - a) = 0 \implies \sum x_i = na \implies a = \frac{1}{n} \sum x_i = \bar{x}$$

The best single summary, in the squared-error sense, is **the mean**. That is not a definition — it is a result, and this is where it comes from.

Confirm it is a minimum rather than a maximum:

$$L''(a) = 2n > 0$$

Positive for any n , so it curves upwards everywhere: one stationary point, and it is the global minimum.

```

import numpy as np
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
scores = cohort["final_score"].dropna()
scores = scores[scores.between(0, 100)].to_numpy()

def loss(a):
    return ((scores - a)**2).sum()

grid = np.linspace(scores.mean() - 6, scores.mean() + 6, 400)
best = grid[np.argmin([loss(a) for a in grid])]

print(f"mean of the data      {scores.mean():.4f}")
print(f"grid search minimum    {best:.4f}")
print(f"agree to 2 dp:          {np.isclose(scores.mean(), best, atol=0.05)}")

```

```

mean of the data      65.2718
grid search minimum   65.2568
agree to 2 dp:       True

```

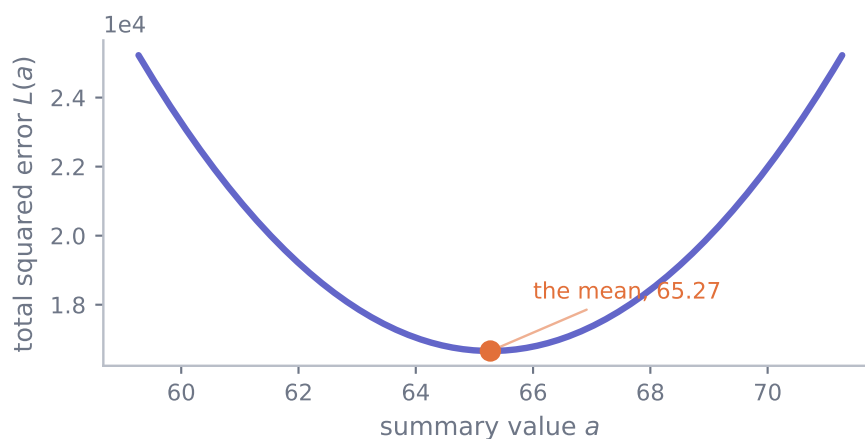


Figure 3: The squared-error loss as the summary value varies. One trough, at the mean — and $L'' > 0$ everywhere, so there is nowhere else it could be.

Change the loss and you change the answer. Minimising the sum of *absolute* errors gives the **median** instead — a different summary, from a different choice about what “wrong” means. The mean is not neutral; it is the answer to one specific question.

Try it yourself

Exercise 1 Find and classify the stationary points of $f(x) = x^3 - 3x^2 + 4$.

💡 Show the solution

$f'(x) = 3x^2 - 6x = 3x(x - 2)$, so stationary points at $x = 0$ and $x = 2$.
 $f''(x) = 6x - 6$:

- $f''(0) = -6 < 0 \rightarrow$ local **maximum**, $f(0) = 4$.
- $f''(2) = +6 > 0 \rightarrow$ local **minimum**, $f(2) = 0$.

Note neither is global over all of $\mathbb{R}$: x^3 dominates, so $f \rightarrow +\infty$ and $-\infty$ at the ends. “Global” needs an interval to be global over.

Exercise 2 Find the global minimum of $f(x) = x^2$ on $[1, 4]$. Why is the method of setting $f'(x) = 0$ misleading here?

 Show the solution

$f'(x) = 2x = 0$ gives $x = 0$ — which is **not in** $[1, 4]$, so it is not a candidate at all.

The candidates are the endpoints: $f(1) = 1$ and $f(4) = 16$. The global minimum is 1, at $x = 1$.

The trap is reporting $x = 0$ because the algebra produced it. Always check that a stationary point lies inside the interval before treating it as a candidate.

Exercise 3 $f(x) = x^4$ has $f'(0) = 0$ and $f''(0) = 0$. What kind of point is $x = 0$, and how would you decide without the second-derivative test?

 Show the solution

A **minimum** — in fact the global one, since $x^4 \geq 0$ with equality only at zero.

Two ways to decide when the test is silent:

- **Evaluate either side.** $f(-0.1) = f(0.1) = 0.0001 > 0 = f(0)$, so the centre is lower than both neighbours.
- **Check the sign of f' .** $f'(x) = 4x^3$ is negative for $x < 0$ and positive for $x > 0$: the function falls then rises, which is a minimum.

The second is more reliable, because it examines a whole neighbourhood rather than two arbitrary points.

Check it in Python

Check a stationary point numerically before trusting your algebra:

```
def f(x): return x**3 - 3*x**2 + 4
def f1(x): return 3*x**2 - 6*x

def centred_difference(g, x, h=1e-6):
    return (g(x + h) - g(x - h)) / (2*h)

for c in (0.0, 2.0):
    print(f"x={c}: symbolic f'={f1(c):+.6f}    numerical {centred_difference(f,
        ↪ c):+.6f}")
```

```
x=0.0: symbolic f'=+0.000000    numerical +0.000000
```

```
x=2.0: symbolic f'=+0.000000    numerical +0.000000
```

Check the classification by looking either side — the method that still works when the second-derivative test is silent:

```
def classify(g, c, h=1e-3):
    left, centre, right = g(c - h), g(c), g(c + h)
    if left > centre < right:
        return "minimum"
    if left < centre > right:
        return "maximum"
    return "neither"

for name, g, c in [("x^3-3x^2+4 at 0", f, 0.0), ("x^3-3x^2+4 at 2", f, 2.0),
                  ("x^4 at 0", lambda x: x**4, 0.0),
                  ("x^3 at 0", lambda x: x**3, 0.0)]:
    print(f" {name:<20} {classify(g, c)}")
```

```
x^3-3x^2+4 at 0      maximum
x^3-3x^2+4 at 2      minimum
x^4 at 0             minimum
x^3 at 0             neither
```

Both cases the second-derivative test cannot resolve are settled in one line.

And search the whole interval, so an endpoint answer cannot be missed:

```
def g(x): return x**3 - 3*x
grid = np.linspace(-1, 3, 2001)
values = g(grid)
print(f"lowest {values.min():.3f} at x={grid[values.argmin():.3f}")
print(f"highest {values.max():.3f} at x={grid[values.argmax():.3f}")
```

```
lowest -2.000 at x=1.000
highest 18.000 at x=3.000
```

A grid search is not a substitute for the calculus — it cannot prove anything and it misses features between grid points. But it is an excellent way to find out that your algebra disagrees with the function.

Common wrong turns

Where this usually goes wrong

Treating $f'(c) = 0$ as proof of a minimum It is a candidate. It may be a maximum, or neither, and the global answer may be somewhere else entirely.

Forgetting the endpoints On a closed interval the best value is very often at an end, where the derivative is not zero. It never appears in the algebra; you must add it.

Using a stationary point outside the interval $f'(x) = 0$ at $x = 0$ is irrelevant if you are working on $[1, 4]$. Check membership before treating it as a candidate.

Reading $f''(c) = 0$ as “probably a minimum” It says nothing at all. Look at the values or the sign of f' either side.

Missing points where f' does not exist $|x|$ has its minimum at a corner, where there is no derivative to set to zero. Non-differentiable points are candidates too.

Forgetting that the loss defines the answer Minimising squared error gives the mean; minimising absolute error gives the median. The summary you get is a consequence of the loss you chose.

Check yourself

Answers are at the foot of the page.

1. $f'(c) = 0$ and $f''(c) < 0$. What is c ?
 - a. local minimum
 - b. local maximum
 - c. neither
 - d. cannot tell
2. Name the three kinds of candidate for a global extremum on $[a, b]$.
3. Why does $f''(c) = 0$ leave the question open? Give two functions that show it.
4. Minimising $\sum (x_i - a)^2$ gives the mean. What does minimising $\sum |x_i - a|$ give?
5. You are told the minimum of $f(x) = x^2$ on $[2, 5]$ is at $x = 0$. What went wrong?

Show the answers

1. **(b) a local maximum.** Negative second derivative means the curve bends downwards, so the stationary point is a peak.
2. Stationary points inside the interval; points where f' does not exist; and the two endpoints.
3. Because both a genuine extremum and a non-extremum can have zero second derivative. x^4 has $f'(0) = f''(0) = 0$ and a minimum at 0; x^3 has the same derivatives at 0 and no extremum there.
4. The **median**. Different loss, different summary — which is the point: the mean is the answer to one specific question about what “wrong” means.
5. $x = 0$ is not in $[2, 5]$, so it was never a candidate. The minimum is at the endpoint $x = 2$, giving $f(2) = 4$.

Where this shows up next

M6 does all of this with several parameters at once, and arrives at gradient descent — the method used when there is no formula for the answer. **M11** minimises exactly this lesson’s squared-error loss with many parameters, and gets a closed-form solution. **S11** applies it to fit a line to the cohort data.

Several variables and the gradient

Unit 6 · Lesson 1 of 3 · Mathematics · M6 Reading: 6 pages With practice: 2.5 hours Assumes: M5

Why this lesson is here

Real models have more than one number to choose. A straight line has two; a logistic regression has one per column of your data; a language model has billions. Fitting any of them means adjusting all of those numbers *together* until the model is least wrong.

This lesson is where the one-variable calculus of M3–M5 becomes usable on real models. The tool is the **gradient**: a list of sensitivities, one per parameter, that answers “if I nudge this one, does the model get better or worse?” for every parameter at once. Following it downhill is **gradient descent**, and that loop — with no changes beyond the number of parameters — is how essentially every model in modern machine learning is trained.

Before you start

You should be able to:

- differentiate with the rules, especially the chain rule — M4;
- set a derivative to zero to find a candidate best value, and remember it is only a candidate — M5;
- read a derivative as a sensitivity carrying units — M3;
- do arithmetic on a whole NumPy array at once — P6.

Quick check. Differentiate $f(x) = (3x + 2c)^2$ with respect to x , treating c as a constant.

 Show the answer

Chain rule: outside is $()^2$, inside is $3x + 2c$ whose derivative is 3. So

$$f'(x) = 2(3x + 2c) \cdot 3 = 6(3x + 2c).$$

The $2c$ came along for the ride and contributed nothing of its own, because it does not move when x moves. Holding one symbol still while differentiating with respect to another is the entire mechanical content of this lesson; you have just done it.

The idea

In M5 you summarised the cohort’s scores with a single number a , chose it to minimise $L(a) = \sum (x_i - a)^2$, and found the mean. One knob, one derivative, one equation.

Now use practice hours as well, and predict a score with the straight line $\hat{s} = a + bh$. Two knobs. The total squared error is a function of both,

$$L(a, b) = \frac{1}{n} \sum_{i=1}^n (s_i - a - bh_i)^2,$$

and asking for “the derivative of L ” no longer has a single answer — for the same reason that in M3 asking for “the rate of change” had no answer until you said *where*. Here you must say *in which direction*.

Picture yourself on a hillside in thick fog. Your height depends on two coordinates and you want to get down, but you cannot see the valley. What you *can* do is feel the ground: step a little east and notice whether you rose or fell, step a little north and do the same. Those two numbers are the **partial derivatives**, and the fact this whole lesson rests on is that on a smooth surface they are enough — from the slope due east and the slope due north you can work out the slope in *every* direction, including the steepest. (The word *smooth* is doing real work: a surface can have both partial derivatives at a point and still misbehave in a diagonal direction there. Everything in these notes is well behaved.)

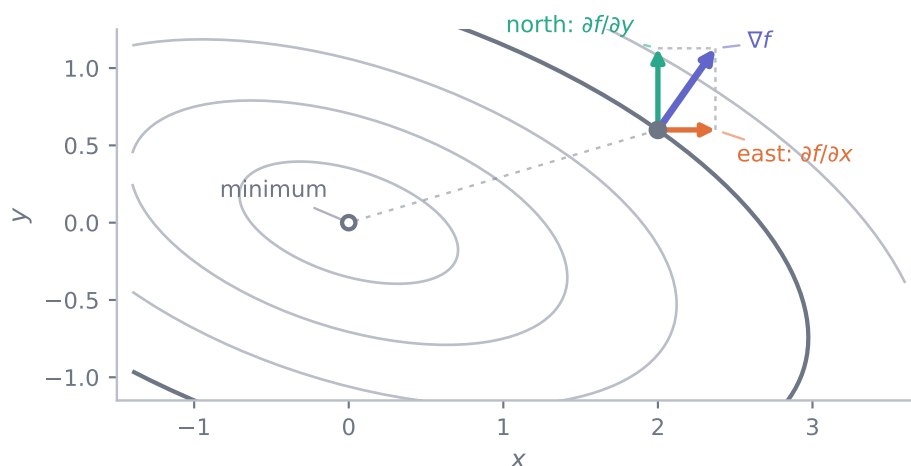


Figure 1: Contours of a two-variable function: each ring is a set of points of equal height, and the minimum is the bullseye. At the marked point, the slope due east and the slope due north combine into one arrow — the gradient — which points straight uphill and crosses the contour at right angles. Note that its reverse does *not* point at the minimum.

That arrow is the gradient. Everything else in this lesson is either how to compute it or what to do once you have it.

The mechanics

Partial derivatives

The **partial derivative** of f with respect to x is the ordinary derivative of [M3](#), taken with every other input frozen:

$$\frac{\partial f}{\partial x}(a, b) = \lim_{t \rightarrow 0} \frac{f(a + t, b) - f(a, b)}{t}.$$

Only the first slot moves: this is the difference quotient of [M3](#), with b sitting there unchanged throughout.

So the working rule is: **to differentiate with respect to x , treat every other variable as a constant** and use the rules of [M4](#) exactly as they stand. No new rules. For $f(x, y) = x^2y + 3y - 5$:

$$\frac{\partial f}{\partial x} = 2xy, \quad \frac{\partial f}{\partial y} = x^2 + 3.$$

In the first, y is a constant multiplier and $3y - 5$ is a constant, so it differentiates to zero. In the second, x^2 is the constant multiplier of y .

The curly ∂ is not decoration. It says: other variables exist, and they are being held still. Writing d here

would claim there is only one input. Where space is short the same two quantities are written f_x and f_y .

The gradient

Collect the partials into one object, in a fixed order:

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right).$$

Read ∇f as “grad f ”. It is not a number — it is a list with one entry per input, and it has a direction as well as a size. (M8 gives that kind of object a proper name and an arithmetic.) Three facts make it useful.

It gives you every direction. Move a small step $(\Delta x, \Delta y)$ from (a, b) and the height changes by roughly

$$f(a + \Delta x, b + \Delta y) \approx f(a, b) + \frac{\partial f}{\partial x} \Delta x + \frac{\partial f}{\partial y} \Delta y.$$

This is the local linear approximation of M3 with one term per input, and it is the reason two numbers suffice: the two partials, weighted by how far you step in each direction, give the slope along any step at all.

It points uphill, steepest. Among all directions of the same small length, the one that increases f fastest is the direction of ∇f itself. So $-\nabla f$ is the direction of steepest *decrease* — the way down. That claim is checked numerically at the end of this lesson rather than asserted.

Its length says how steep. $\|\nabla f\| = \sqrt{f_x^2 + f_y^2}$ is the slope in that steepest direction. Where the ground is level, it is zero.

Stationary points, and a new way to be fooled

A stationary point now needs *all* the partials to vanish at once: $\nabla f = (0, 0)$ means $\partial f / \partial x = 0$ **and** $\partial f / \partial y = 0$ at the same point. That is a system of simultaneous equations rather than a single one, and as in M5 a solution is a *candidate*, not an answer. With two variables there is a new way for a candidate to disappoint, with no one-variable analogue: a **saddle**, which is a minimum along one direction and a maximum along another.

At the origin, $\partial f / \partial x = 2x = 0$ and $\partial f / \partial y = -2y = 0$. Both partials vanish and the point is neither best nor worst. In two variables a saddle is a curiosity; with a thousand parameters it is the *typical* stationary point, and recognising one is a large part of why training a big model is hard.

Gradient descent

For the models in this lesson you could solve $\nabla L = 0$ by algebra, and M11 does exactly that. For most models you cannot: there is no formula, only a function you can evaluate and differentiate. So walk downhill instead. Repeat, from any starting guess:

$$a \leftarrow a - \eta \frac{\partial L}{\partial a}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

Both updates use the partials computed at the *old* values, and are applied together. The number η

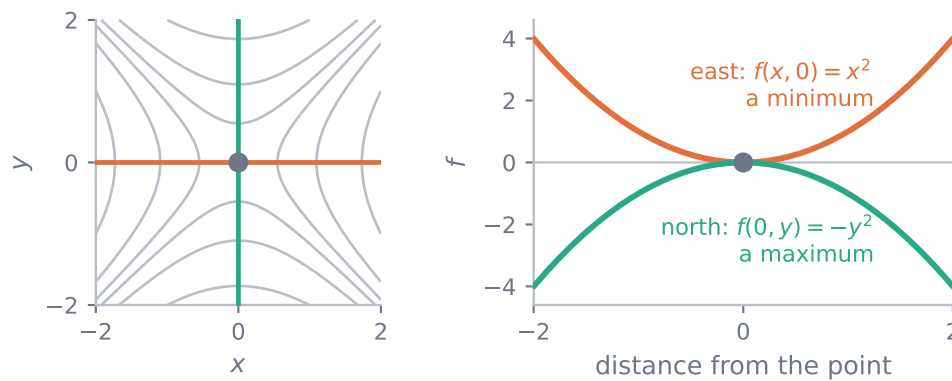


Figure 2: The saddle $f(x, y) = x^2 - y^2$ at the origin. Left: its contours, with the flat point marked. Right: the same point, seen along two directions — walk east and it is the bottom of a valley; walk north and it is the top of a ridge. Both partials are zero, and it is neither a maximum nor a minimum.

(“eta”) is the **learning rate** or step size. It is yours to choose, and the choice matters:

η	what happens
too small	every step is safe and the loss crawls down; you run out of patience
about right	the loss falls quickly and flattens out
too large	each step overshoots the bottom and lands further up the far side; the loss grows without limit

There is no universally correct η , because the safe range depends on how sharply the surface curves — and, as the next section shows, on something as mundane as the units your columns are measured in.

Worked example

Fit a line to the cohort by gradient descent.

We model final score from hours of practice, on the rows where the score is usable. That is the 238-row filter used throughout these notes.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h, s = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()
print(f"{len(usable)} rows; hours run from {h.min():.1f} to {h.max():.1f}")
```

238 rows; hours run from 1.0 to 36.2

The loss is $L(a, b) = \frac{1}{n} \sum (s_i - a - b h_i)^2$. Differentiate it with respect to each parameter in turn, treating the other as a constant and using the chain rule on the bracket. Write $r_i = s_i - a - b h_i$ for the residual:

$$\frac{\partial L}{\partial a} = \frac{1}{n} \sum 2r_i \cdot (-1) = -\frac{2}{n} \sum r_i, \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum 2r_i \cdot (-h_i) = -\frac{2}{n} \sum r_i h_i.$$

Now descend. First attempt, straight from the raw columns, starting at $a = b = 0$ with $\eta = 0.01$:

```
def gradient(a, b, x, y):
    r = y - a - b*x
    return -2*r.mean(), -2*(r*x).mean()

a, b = 0.0, 0.0
for step in range(1, 7):
    ga, gb = gradient(a, b, h, s)
    a, b = a - 0.01*ga, b - 0.01*gb
    print(f"step {step}    a = {a:>12.4g}    b = {b:>12.4g}    loss = {((s - a -
    ↪ b*h)**2).mean():>10.3g}")
```

step 1	a =	1.305	b =	21.12	loss =	1.1e+05
step 2	a =	-4.352	b =	-99.95	loss =	3.58e+06
step 3	a =	29.87	b =	593.4	loss =	1.18e+08
step 4	a =	-164.3	b =	-3378	loss =	3.86e+09
step 5	a =	949.9	b =	1.937e+04	loss =	1.27e+11
step 6	a =	-5431	b =	-1.109e+05	loss =	4.15e+12

The loss multiplies by about 32 every step. Nothing is wrong with the calculus — the partials are right and the direction is genuinely downhill. The step *length* is wrong. Practice hours run up to 36, so $\partial L / \partial b$ carries a factor of h and comes out about sixteen times the size of $\partial L / \partial a$ at the start, as the first line of output shows. The surface is a long narrow valley, and $\eta = 0.01$ is safe for a and lethal for b .

The standard fix is to put the input on a scale of its own size — subtract its mean and divide by its spread, so it runs over a few units either side of zero:

```
z = (h - h.mean()) / h.std()

a, b = 45.0, 10.0          # any starting guess will do
for step in range(1, 41):
    ga, gb = gradient(a, b, z, s)
    a, b = a - 0.3*ga, b - 0.3*gb
    if step in (1, 2, 3, 5, 40):
        print(f"step {step:>2}    a = {a:8.4f}    b = {b:8.4f}    loss = {((s - a -
        ↪ b*z)**2).mean():9.4f}")
```

step 1	a =	57.1631	b =	2.8270	loss =	154.7954
step 2	a =	62.0284	b =	-0.0423	loss =	80.3551
step 3	a =	63.9745	b =	-1.1899	loss =	68.4446
step 5	a =	65.0643	b =	-1.8327	loss =	66.2340
step 40	a =	65.2718	b =	-1.9551	loss =	66.1759

Same loss, same two partial derivatives, same descent loop — only the scale of one column changed, and $\eta = 0.3$ is now comfortable where 0.01 was fatal. (The two runs start from different points, which makes no difference here: the surface has one minimum and every starting point reaches it.)

Finally, translate back. A slope measured per standard deviation of practice becomes a slope per hour by dividing by that standard deviation, and the intercept shifts to match:

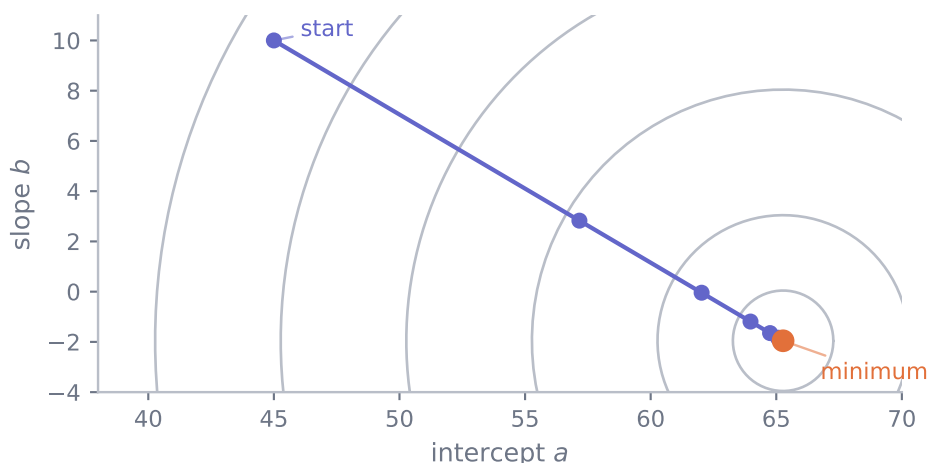


Figure 3: The descent above, drawn on the contours of $L(a, b)$, whose minimum is at $a = 65.27$, $b = -1.96$. On this scaling the contours are circles, so downhill points straight at the minimum and the path is a straight run. The steps shorten as it arrives, because the gradient itself shrinks towards zero — the method slows down on its own.

```
slope, intercept = b / h.std(), a - b*h.mean()/h.std()
exact_slope, exact_intercept = np.polyfit(h, s, 1)
print(f"gradient descent   intercept {intercept:8.4f}    slope {slope:+.4f} marks
      ↳ per hour")
print(f"closed form       intercept {exact_intercept:8.4f}    slope
      ↳ {exact_slope:+.4f} marks per hour")
print(f"agree to 4 dp: {np.allclose([slope, intercept], [exact_slope,
      ↳ exact_intercept], atol=1e-4)}")
```

```
gradient descent   intercept 69.2261    slope -0.2408 marks per hour
closed form       intercept 69.2261    slope -0.2408 marks per hour
agree to 4 dp: True
```

Two readings, and the second matters more. The mathematics: an iterative walk downhill has landed on the answer that algebra gives directly, which is the evidence that the walk is doing what it claims. The data: the fitted slope is about -0.24 marks per hour of practice — the model says practising *more* goes with scoring *less*. That is a real feature of this dataset and not an arithmetic slip, and it reverses completely once you look within each background group. [S12](#) is about why.

Try it yourself

Exercise 1 For $f(x, y) = x^2y - 4y^3 + 7$, find both partial derivatives and evaluate ∇f at $(2, 1)$. Which direction from that point increases f fastest?

💡 Show the solution

Holding y fixed, $-4y^3 + 7$ is a constant and y is a multiplier:

$$\frac{\partial f}{\partial x} = 2xy.$$

Holding x fixed, x^2 is a multiplier and 7 is a constant:

$$\frac{\partial f}{\partial y} = x^2 - 12y^2.$$

At (2, 1): $\nabla f = (2 \cdot 2 \cdot 1, 4 - 12) = (4, -8)$.

The fastest increase is in the direction of ∇f itself: for every 4 you move in x , move -8 in y . The tempting error is to answer “(4, -8)” to the question *how much does f increase* — that is a direction, not an amount. The steepest slope there is its length, $\sqrt{4^2 + 8^2} \approx 8.94$.

Exercise 2 Find the stationary point of $g(x, y) = x^2 + xy + y^2 - 3y$.

 Show the solution

Both partials must vanish at once:

$$\frac{\partial g}{\partial x} = 2x + y = 0, \quad \frac{\partial g}{\partial y} = x + 2y - 3 = 0.$$

From the first, $y = -2x$. Substituting: $x - 4x - 3 = 0$, so $x = -1$ and $y = 2$. The only stationary point is $(-1, 2)$, where $g = -3$.

It is a minimum, though nothing above proves that. Checking is easy: $g(-1.1, 2) = g(-0.9, 2) = -2.99$ and $g(-1, 1.9) = g(-1, 2.1) = -2.99$, all higher than -3 . Four neighbours are not a proof either — a saddle can hide in a diagonal direction that neither axis explores — but they are the right first move, and they are what the code in the next block automates.

Exercise 3 Take $L(a, b) = (a - 3)^2 + 4(b + 1)^2$, starting at $(0, 0)$ with $\eta = 0.1$. Do one step of gradient descent by hand. Then work out how large η can be before the b updates start to grow instead of shrink.

 Show the solution

$\nabla L = (2(a - 3), 8(b + 1))$, so at $(0, 0)$ it is $(-6, 8)$. One step gives

$$(a, b) \leftarrow (0, 0) - 0.1 \cdot (-6, 8) = (0.6, -0.8).$$

Both moved towards the minimum at $(3, -1)$.

For the second part, follow one parameter through a general step. The b update is $b \leftarrow b - 8\eta(b + 1)$, so the distance to the target $b = -1$ obeys

$$(b + 1) \leftarrow (1 - 8\eta)(b + 1).$$

The distance shrinks only when $|1 - 8\eta| < 1$, that is when $\eta < 0.25$. The same argument for a gives $\eta < 1$. So b sets the limit, and it is the more sharply curved direction — the coefficient 4 in front of $(b + 1)^2$ is exactly what makes it so. This is the worked example in miniature: one badly scaled parameter caps the step size for all of them.

Check it in Python

First, that the partials are right. A centred difference in one coordinate at a time is the two-variable version of the check from [M3](#):

```
def f(x, y):
    return x**2 * y - 4*y**3 + 7
```

```
def grad_by_hand(x, y):
    return np.array([2*x*y, x**2 - 12*y**2])

def grad_numeric(g, x, y, step=1e-6):
    return np.array([(g(x + step, y) - g(x - step, y)) / (2*step),
                     (g(x, y + step) - g(x, y - step)) / (2*step)])

print("by hand ", grad_by_hand(2.0, 1.0))
print("numeric ", grad_numeric(f, 2.0, 1.0))
print("agree: ", np.allclose(grad_by_hand(2.0, 1.0), grad_numeric(f, 2.0, 1.0)))
```

```
by hand [ 4. -8.]
numeric [ 4. -8.]
agree:  True
```

Break it on purpose and watch the check fire — a check you have never seen fail is a check you have no reason to trust:

```
def grad_wrong(x, y):
    return np.array([2*x*y, x**2 - 12*y]) # forgot to square the y

for point in [(2.0, 1.0), (2.0, 3.0)]:
    print(f"at {point}: broken {grad_wrong(*point)} numeric {grad_numeric(f,
    ↪ *point)}"
          f" agree: {np.allclose(grad_wrong(*point), grad_numeric(f,
    ↪ *point))}")
```

```
at (2.0, 1.0): broken [ 4. -8.] numeric [ 4. -8.] agree: True
at (2.0, 3.0): broken [ 12. -32.] numeric [ 12. -104.00000001] agree: False
```

The first row is the more instructive one. At $y = 1$ the mistake is invisible, because y and y^2 agree there — a check run only at that point would have passed a wrong derivative. Test where nothing is special.

Then the lesson's main claim: that the gradient points in the direction of steepest increase. Nothing so far has demonstrated this. Try every direction and see which wins:

```
angles = np.linspace(0, 2*np.pi, 3601) # every 0.1 degree
step = 1e-6
rise = np.array([(f(2 + step*np.cos(t), 1 + step*np.sin(t)) - f(2, 1)) / step
                  for t in angles])

best = angles[rise.argmax()]
gx, gy = grad_by_hand(2.0, 1.0)
predicted = np.arctan2(gy, gx) % (2*np.pi)

print(f"steepest direction found by search : {np.degrees(best):8.3f} degrees")
print(f"direction of the gradient          : {np.degrees(predicted):8.3f}
    ↪ degrees")
```

```
print(f"steepest rise found           : {rise.max():.4f}")
print(f"length of the gradient       : {np.hypot(gx, gy):.4f}")
```

```
steepest direction found by search : 296.600 degrees
direction of the gradient          : 296.565 degrees
steepest rise found                : 8.9443
length of the gradient              : 8.9443
```

Both pairs agree — the two angles to within the 0.1° resolution of the search, and the steepest rise to four decimal places against the gradient’s length. That is the claim, tested rather than asserted.

Finally, that descent converges — and when it does not. Everything above is about one point; this is about the loop:

```
def descend(eta, steps=200):
    a, b = 0.0, 0.0
    for _ in range(steps):
        ga, gb = gradient(a, b, z, s)
        a, b = a - eta*ga, b - eta*gb
    return a, b, ((s - a - b*z)**2).mean()

for eta in (0.5, 0.3, 0.05, 1.05):
    a_e, b_e, loss = descend(eta)
    verdict = "converged" if np.isfinite(loss) and loss < 67 else "FAILED"
    print(f"eta={eta:<5} a={a_e:>12.4g} b={b_e:>12.4g} loss={loss:>12.4g}
    ↳ {verdict}")
```

eta=0.5	a=	65.27	b=	-1.955	loss=	66.18	converged
eta=0.3	a=	65.27	b=	-1.955	loss=	66.18	converged
eta=0.05	a=	65.27	b=	-1.955	loss=	66.18	converged
eta=1.05	a=	-1.24e+10	b=	3.713e+08	loss=	1.538e+20	FAILED

Three converge to the same place from the same start and the fourth runs away, so “gradient descent finds the minimum” is a claim about the method *and* the step size together. Note also that $\eta = 0.5$ lands on the answer in a single step: on this scaling the surface is a perfectly circular bowl, and half the gradient is exactly the distance to the bottom. That is luck arranged by the standardising — on any real loss surface it will not happen.

Common wrong turns

Where this usually goes wrong

Treating ∇f as a number It has one component per input. “The gradient is 8.94” confuses the gradient with its length, and loses the only part you need in order to take a step — which way to go.

Letting the frozen variable thaw In $\partial(x^2y)/\partial x = 2xy$, the y is a constant multiplier. It is not a function of x , so no product rule and no chain rule apply to it. If you find yourself differentiating y with respect to x , you have stopped taking a partial derivative.

Reading $\nabla f = 0$ as “minimum” It is a candidate, exactly as in M5, plus one new possibility: a saddle, which is a minimum along one direction and a maximum along another. In many dimensions saddles outnumber genuine minima.

Expecting $-\nabla f$ to point at the minimum It points down the steepest slope *here*. On an elongated bowl that is across the valley rather than along it, which is why descent zigzags and why scaling the inputs helps so much. The gradient is local information and knows nothing about where the bottom is.

Updating the parameters one at a time Compute all the partials at the current point, *then* apply all the updates. If you overwrite a and then use the new a to compute $\partial L / \partial b$, you are no longer stepping along the gradient. It often still converges, which is what makes the bug expensive: nothing visibly breaks.

Blaming the calculus when the loss explodes A diverging loss almost always means the step size is too large for the curvature — very often because one column is measured in units hundreds of times bigger than another. Check the scaling before you check the algebra.

Check yourself

Answers are at the foot of the page. Try to give a reason as well as an answer.

- For $f(x, y) = 3x^2y$, what is $\partial f / \partial y$ at $(2, 5)$?
 - 12
 - 60
 - $30x$
 - 0
- At a point P , both partial derivatives of f are zero; walking east from P the ground rises both ways, walking north it falls both ways. What is P ?
- A loss L is measured in marks² and the parameter a in marks. What are the units of $\partial L / \partial a$, and what does that make the units of η ?
- Gradient descent on your model diverges. Name two changes that could fix it without touching a single derivative.
- Your update loop sets $a = a - \text{eta} * \text{grad_a}$, then computes grad_b from the new a , then sets $b = b - \text{eta} * \text{grad_b}$. Why is this not gradient descent, and why is the bug hard to notice?

Show the answers

- (a) 12.** Holding x fixed, $f = (3x^2)y$ is a constant times y , so $\partial f / \partial y = 3x^2 = 12$, and the $y = 5$ is never used. Option (b) is $\partial f / \partial x = 6xy$, the other partial — the most common slip. Option (c) is that same wrong partial, $6xy$, with $y = 5$ put in and x left alone; (d) treats *both* variables as constants.
- A **saddle**. East is a minimum, north is a maximum, and the point is neither — so $\nabla f = 0$ has told you nothing about whether it is a good place to be.
- Marks** — marks² divided by marks. Since $\eta \cdot \partial L / \partial a$ is subtracted from a , it must also be in marks, so here η is a plain number. That is a coincidence of this particular loss, not a general fact: in general η carries units of (parameter)² per unit of loss, which is why it is not a free-floating knob and why a rate that works on one dataset can be catastrophic on the same data rescaled.
- Reduce η ; and standardise the input columns so that no parameter's curvature is enormously larger than another's. Both change the step, not the mathematics. (A third: start

from a different point — occasionally decisive, but it will not rescue a step size that is simply too big.)

5. Gradient descent steps along ∇L evaluated at **one** point. Mixing an updated a with an old b means the two components come from two different points, so the step is along no gradient at all. It is hard to notice because the mixed step is usually still downhill, so the loss still falls and the fit still looks fine — it just is not the algorithm you wrote down, and it will diverge under conditions the real one survives.

Where this shows up next

M11 minimises this lesson's loss with many parameters and finds that, for a linear model, $\nabla L = 0$ can be solved exactly — no descent, no learning rate. **M12** explains the long narrow valley properly: its shape is set by directions in the data, and that is what standardising was fixing. **S11** fits this same line to the cohort and asks the question this lesson deliberately did not: whether to believe it. **S12** takes up the negative slope, and why it turns positive inside every background group.

Accumulation and area

Unit 6 · Lesson 2 of 3 · Mathematics · M7 Reading: 5 pages With practice: ⌚ 2 hours Assumes: M4

Why this lesson is here

A derivative answers *how fast is this changing?* This lesson answers the opposite question: *how much has accumulated?* Given a rate, what is the total; given a curve, what is the area underneath it.

You need this for one reason above all others. In the Statistics track a continuous quantity is described by a **density**, and a density on its own is not a probability — you get a probability by taking the area under it between two values. Every statement of the form “about 43% of the cohort scored between 60 and 70” is an area, and from S5 onwards you will be reading them constantly. The other half of the lesson is the fact that makes areas computable at all: accumulation and differentiation are inverse operations.

Before you start

You should be able to:

- differentiate with the rules — M4;
- read a derivative as a rate carrying units — M3;
- work with e^x and $\ln x$ — M2;
- add up a NumPy array with `.sum()` — P7.

Quick check. Differentiate $F(x) = \frac{1}{3}x^3 - 2x$.

💡 Show the answer

$$F'(x) = x^2 - 2.$$

Now read that backwards: you have just found a function whose derivative is $x^2 - 2$. Going in that direction — from a rate back to the thing that accumulated it — is what this lesson is about, and by the end of it you will see that $\frac{1}{3}x^3 - 2x$ is exactly what computes areas under $x^2 - 2$.

The idea

Start with a rate that does not change. If water runs into a tank at a steady 5 litres per minute for 12 minutes, the total is $5 \times 12 = 60$ litres. Draw the rate against time and that total is the area of a rectangle: height times width.

Now let the rate vary, and the trick is to pretend for a moment that it does not. Chop the 12 minutes into short strips. Across one short strip the rate barely changes, so treat it as constant, and that strip contributes about (rate) $\times$ (width) — a rectangle. Add the rectangles up and you have an estimate of the total. Take narrower strips and the estimate improves.

Here is that estimate for the score model of M3, $s(h) = 45 + 6h - 0.25h^2$, over $0 \leq h \leq 12$, using six strips and then twenty-four:

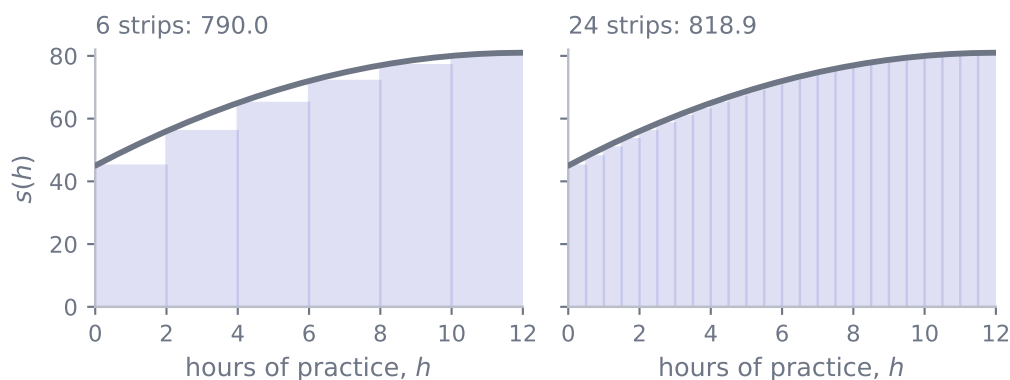


Figure 1: The same area estimated with six strips and with twenty-four. Each rectangle takes the height of the curve at its left edge, so every one of them undershoots on a rising curve; the visible gap between the rectangles and the curve is exactly the error, and it shrinks as the strips narrow.

```
def s(h):
    return 45 + 6*h - 0.25*h**2

import numpy as np
for n in [6, 24, 100, 1000, 100_000]:
    left_edges = np.linspace(0, 12, n + 1)[:n]
    print(f"{n:>7} strips    estimate {s(left_edges).sum() * 12/n:12.6f}")
```

6 strips	estimate	790.000000
24 strips	estimate	818.875000
100 strips	estimate	825.832800
1000 strips	estimate	827.783928
100000 strips	estimate	827.997840

The estimates settle on 828, in the same way that the difference quotients of [M3](#) settled on 4. There the interval shrank towards a point; here the strips shrink towards zero width and there are ever more of them. The value they settle on is the **integral** of s from 0 to 12, written

$$\int_0^{12} s(h) \, dh = 828.$$

Two readings, as with the derivative. **Geometrically** it is the area between the curve and the horizontal axis, between those two limits. **Practically** it is a total accumulated from a rate: here, marks-per-student summed over an interval of hours, so the answer carries units of marks $\times$ hours. An integral always carries the units of the thing being integrated multiplied by the units of the variable you integrate over.

The mechanics

Notation

$$\int_a^b f(x) \, dx$$

The $\int$ is an elongated S, for *sum*. The dx is the width of a strip, and it also tells you which variable you are accumulating over — which matters as soon as there is more than one symbol in play. a and b are the **limits**. This is a *definite* integral: limits attached, and the answer is a number.

Two properties fall straight out of the picture:

- **Area below the axis counts as negative.** A strip where f is negative has negative height, so it subtracts. The integral is a *signed* area, and it can be zero for a function that is nowhere zero.
- **Swapping the limits flips the sign**, so $\int_b^a f = -\int_a^b f$. Strips of negative width.

The fundamental theorem

Adding up ever more rectangles is a terrible way to get an exact answer. The theorem that rescues it is the most useful result in this half of the subject.

Fix the left limit and let the right one move. Define the **accumulation function**

$$A(x) = \int_a^x f(t) dt,$$

the area collected so far when you have got as far as x . (The variable inside had to be renamed: t runs across the strips while x marks where you stopped. They cannot both be x .)

Now nudge x by a small amount h . The extra area is a sliver of width h and height about $f(x)$, so $A(x+h) - A(x) \approx f(x)h$, and therefore

$$A'(x) = \lim_{h \rightarrow 0} \frac{A(x+h) - A(x)}{h} = f(x).$$

The sliver argument needs f to be continuous at x : if f jumps there, the sliver has no single height and A has a corner instead of a tangent. Every f in this lesson is continuous, and the result is stated for that case.

Accumulating and differentiating undo each other. That is the fundamental theorem of calculus, and its practical form is: if you can find any F with $F' = f$, then

$$\int_a^b f(x) dx = F(b) - F(a),$$

usually written $[F(x)]_a^b$. Such an F is called an **antiderivative** of f .

The antiderivative table

Every line of it is a line of M4's table read from right to left.

$f(x)$	an antiderivative $F(x)$	
c	cx	check: $(cx)' = c$
$x^n, n \neq -1$	$\frac{x^{n+1}}{n+1}$	raise the power, divide by the new one
$\frac{1}{x}$	$\ln x $	the case the power rule cannot do
e^x	e^x	its own antiderivative, as it is its own derivative

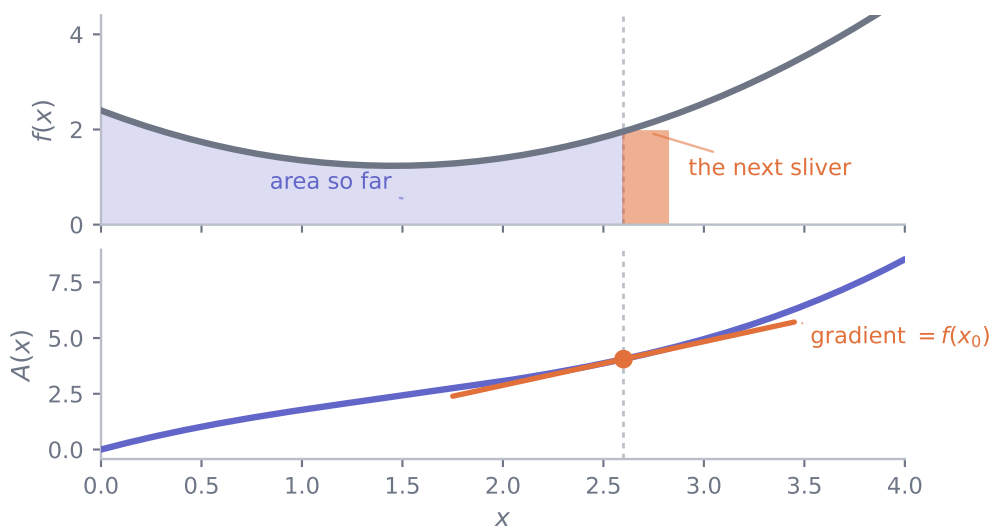


Figure 2: The same function twice. Above: f , with the area accumulated up to x_0 shaded and the next thin sliver picked out — its area is about $f(x_0)$ times its width. Below: the accumulated area $A(x)$ as a function of where you stop. The gradient of A at x_0 is the *height* of f at x_0 , which is what the fundamental theorem says.

Sums and constant multiples pass through unchanged: $\int(f + cg) = \int f + c \int g$. There is nothing so convenient for products or quotients — the chain rule and product rule run backwards into methods (substitution, integration by parts) that these notes do not need.

Antiderivatives are not unique: $F(x) + C$ works for any constant C , because a constant differentiates to zero. Hence the $+C$ you may remember. For a *definite* integral it cancels — $(F(b) + C) - (F(a) + C)$ — so you may drop it, and this lesson does.

Where this stops working

Plenty of ordinary functions have no antiderivative expressible in the usual symbols. The one that matters most is e^{-x^2} , whose area is the whole business of the normal distribution in S6. No amount of cleverness will produce a formula for it. Such integrals are computed numerically, by a better-organised version of the strips above, and the last section of this lesson does exactly that.

Worked example

Part 1: an average, from an integral. Over the first 12 hours of practice, what score does the model of M3 predict *on average*? Not the average of a few sampled points — the average over the whole continuous range, which is the accumulated total divided by the width:

$$\text{average} = \frac{1}{b-a} \int_a^b f(x) dx.$$

An antiderivative of $s(h) = 45 + 6h - 0.25h^2$, term by term from the table:

$$S(h) = 45h + 3h^2 - \frac{h^3}{12}.$$

Evaluate at both limits:

$$\int_0^{12} s(h) dh = \left[45h + 3h^2 - \frac{1}{12}h^3 \right]_0^{12} = (540 + 432 - 144) - 0 = 828,$$

which is the number the strips were converging on. The average is $828/12 = 69.0$ marks — and note the units working: marks $\times$ hours, divided by hours, gives marks.

Part 2: a proportion, from an area. Now the cohort itself. A histogram drawn with `density=True` has bar *heights* in **proportion of the cohort per mark**, chosen so that the total **area** of the bars is exactly 1. An area under it is then a proportion.

```
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
scores = cohort[cohort["final_score"].between(0, 100)]["final_score"].to_numpy()

edges = np.arange(42.5, 100.1, 2.5)
heights, edges = np.histogram(scores, bins=edges, density=True)
widths = np.diff(edges)

within = (edges[:-1] >= 60) & (edges[:-1] < 70)
print(f"{len(scores)} scores, {len(widths)} bars {widths[0]} marks wide")
print(f"total area under the bars    {(heights * widths).sum():.6f}")
print(f"area of the bars from 60-70 {(heights[within] * \
    widths[within]).sum():.6f}")
print(f"proportion scoring 60-70    {((scores >= 60) & (scores < \
    70)).mean():.6f}")
```

```
238 scores, 23 bars 2.5 marks wide
total area under the bars    1.000000
area of the bars from 60-70 0.432773
proportion scoring 60-70    0.432773
```

The last two numbers are equal, and that is the whole point: the area under a density between two values *is* the proportion of the data between them. The tallest bar here is only about 0.054, and reading that as a count or a probability is the standard mistake: it is a proportion *per mark*, and nothing stops such a height rising above 1. Express the same scores as fractions of full marks rather than in marks and the horizontal axis compresses by a factor of 100, so every height multiplies by 100.

Try it yourself

Exercise 1 Evaluate $\int_1^3 (2x + 1) dx$, then check the answer without calculus.

Show the solution

An antiderivative is $F(x) = x^2 + x$, so

$$\int_1^3 (2x + 1) dx = [x^2 + x]_1^3 = (9 + 3) - (1 + 1) = 10.$$

The check: $y = 2x + 1$ is a straight line, so the region under it between $x = 1$ and $x = 3$ is a trapezium with parallel sides $f(1) = 3$ and $f(3) = 7$ and width 2. Its area is $\frac{1}{2}(3 + 7) \times 2 = 10$.

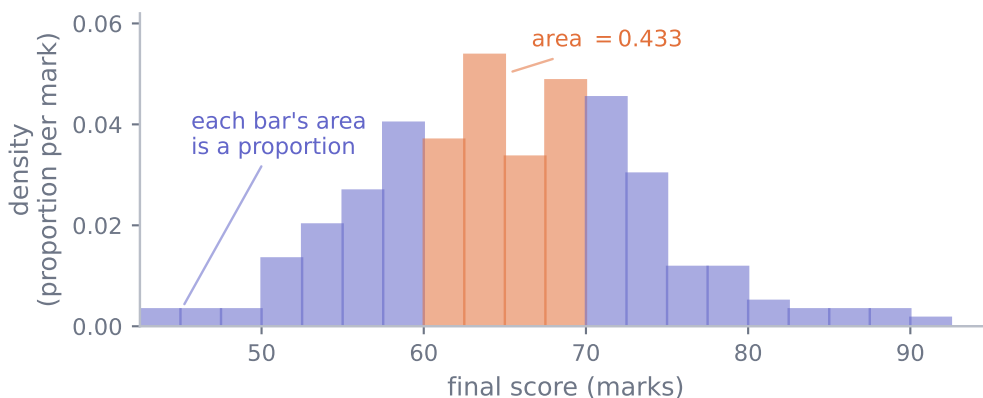


Figure 3: Final scores as a density: bar heights are proportions of the cohort per mark, and every bar's area is the proportion of the cohort in that range. The shaded bars are the range 60 to 70, whose total area is 0.433 — which is exactly the fraction of the 238 students who scored there.

Do this whenever the shape is one you know. Geometry and the antiderivative are computing the same thing, and disagreement means one of them is wrong.

Exercise 2 Evaluate $\int_0^2 (x - 1) dx$. Then explain the answer.

💡 Show the solution

$$\int_0^2 (x - 1) dx = \left[\frac{1}{2}x^2 - x \right]_0^2 = (2 - 2) - 0 = 0.$$

Zero, although $x - 1$ is zero only at the single point $x = 1$. Between 0 and 1 the function is negative and contributes $-\frac{1}{2}$; between 1 and 2 it is positive and contributes $+\frac{1}{2}$; they cancel exactly.

The tempting reading is “the area under the curve is zero”, which is false: the region between the line and the axis has area 1. The integral is a *signed* area. If you want the geometric area, integrate $|x - 1|$, or split at the crossing point and add the pieces without their signs.

Exercise 3 Water flows into a tank at a rate $r(t)$ litres per minute, where t is minutes since you started watching. You are told that $\int_0^{30} r(t) dt = 450$.

Say in one sentence, with units, what that means — and then say what it does **not** tell you.

💡 Show the solution

450 litres flowed in during the first 30 minutes. The units come out of the integral itself: litres per minute, multiplied by minutes.

What it does not tell you:

- **the rate at any particular moment.** The average was 15 litres per minute, but the flow could have been steady, or a torrent for two minutes and a trickle after. Totals do not determine rates, exactly as in M3 a rate did not determine a total.
- **how much is in the tank.** The integral measures what flowed *in* over that window. Without the starting volume — and without knowing whether anything drained out — you cannot get a level from it.

This is the mirror image of Exercise 3 in M3, and worth holding side by side with it: a rate says

nothing about a level, and a total says nothing about an instant.

Check it in Python

The claim to test is the fundamental theorem: that evaluating an antiderivative at the two limits gives the same number as adding up strips. Two ways of adding up strips, in fact — the crude left-edge rectangles from earlier, and the **trapezoid rule**, which joins consecutive points with a straight line instead of a flat one, so each strip follows the slope of the curve rather than ignoring it.

```
def left_rectangles(f, a, b, n):
    edges = np.linspace(a, b, n + 1)
    return f(edges[:-1]).sum() * (b - a)/n

def trapezoid(f, a, b, n):
    y = f(np.linspace(a, b, n + 1))
    return (y[0]/2 + y[1:-1].sum() + y[-1]/2) * (b - a)/n

def S(h):
    # our antiderivative, by hand
    return 45*h + 3*h**2 - h**3/12

exact = S(12) - S(0)
print(f"antiderivative: S(12) - S(0) = {exact}")
for n in (4, 16, 64, 1024):
    print(f"  n={n:>5}    rectangles {left_rectangles(s, 0, 12, n):11.5f}"
          f"    trapezoid {trapezoid(s, 0, 12, n):11.5f}")
```

```
antiderivative: S(12) - S(0) = 828.0
n=    4  rectangles  769.50000  trapezoid  823.50000
n=   16  rectangles  814.21875  trapezoid  827.71875
n=   64  rectangles  824.60742  trapezoid  827.98242
n= 1024  rectangles  827.78899  trapezoid  827.99993
```

Both converge on 828, and the trapezoid rule with 64 strips is already closer than the rectangles with 1024 — a sixteenth of the work for a tenth of the error. The method matters as much as the number of strips, which is the lesson of M3's numerical derivative arriving from the other direction.

Now the integral that has no antiderivative. Over the whole real line, $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$, a fact you cannot get to by any manipulation of the table above:

```
gauss = lambda x: np.exp(-x**2)

for limit, n in [(3, 1_000), (6, 100), (6, 1_000), (6, 100_000)]:
    estimate = trapezoid(gauss, -limit, limit, n)
    print(f"over [-{limit}, {limit}] with n={n:>7}    {estimate:.10f}"
          f"    matches sqrt(pi): {np.isclose(estimate, np.sqrt(np.pi))}")
```

```
over [-3, 3] with n= 1000  1.7724146921  matches sqrt(pi): False
over [-6, 6] with n=  100  1.7724538509  matches sqrt(pi): True
```

```
over [-6, 6] with n= 1000    1.7724538509    matches sqrt(pi): True
over [-6, 6] with n= 100000  1.7724538509    matches sqrt(pi): True
```

The first line is the one to study. It uses a thousand strips — ten times as many as the line below it, which lands on the right answer — and it is still wrong in the fifth decimal place, and adding strips will never fix it. The error is not in the step size but in the *interval*: cutting the curve off at ± 3 throws away real area in the tails. Two different things can go wrong with a numerical integral, and they need opposite responses.

Common wrong turns

Where this usually goes wrong

Confusing the integral with the integrand $r(t)$ is a rate in litres per minute; $\int_0^{30} r(t) dt$ is a total in litres. They are different quantities in different units, exactly as f and f' are in M3.

Assuming an integral cannot be negative It is a signed area. $\int_0^2 (x - 1) dx = 0$ even though the region between the line and the axis plainly has area 1.

Reading a density as a probability A density has units — here, proportion of the cohort per mark — and its height can be any positive number at all, including numbers far above 1. Only its *area* is a probability, which is why this lesson is a prerequisite for S5.

Losing the constant of integration where it matters It cancels in a definite integral and you should drop it there. It does not cancel when you are reconstructing a quantity from its rate: knowing the flow into a tank gives you the change in level, and you still need one measured level to pin down C .

Expecting every function to have an antiderivative e^{-x^2} has none in elementary terms, and it is the single most important function in statistics. Failing to find a formula is often not a failure.

Blaming the strip count for every numerical error Too few strips and too narrow an interval look alike — both give an answer that is a bit wrong. Only the first is cured by more strips. Change one thing at a time and watch which one moves the answer.

Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. What is $\int_0^2 3 dx$?

- a. 3
- b. 6
- c. 0
- d. $3x$

2. $\int_1^e \frac{1}{x} dx$ — evaluate it, and say which line of the antiderivative table you used and why the power rule is no help.

3. A density curve for waiting times, measured in minutes, reaches a height of 1.8 at its peak. Is that a mistake? What would the height become if the same waiting times were measured in hours?

4. F and G are both antiderivatives of the same f , and $F \neq G$. What can you say about $F - G$, and why does the difference not affect $\int_a^b f$?

5. Your trapezoid estimate of $\int_{-3}^3 e^{-x^2} dx$ sticks at 1.77241 no matter how many strips you use, while $\sqrt{\pi} = 1.77245$. What is wrong, and what fixes it?

 Show the answers

1. **(b)** 6. The region is a rectangle of height 3 and width 2. By the table, $[3x]_0^2 = 6 - 0$. Option (a) forgets the width; (d) forgets that a definite integral is a number, not a function — the limits have been substituted in and x is gone.
2. **1.** $\int \frac{1}{x} dx = \ln|x|$, so the answer is $\ln e - \ln 1 = 1 - 0 = 1$. The power rule would want $\frac{1}{x} = x^{-1}$ to become $\frac{x^0}{0}$, which divides by zero. That single excluded case is the reason logarithms appear in the table at all.
3. **Not a mistake.** A density is a *proportion per unit of the measurement* — here, proportion of waiting times per minute — and nothing bounds its height — only its total area is fixed at
 1. Re-expressed in hours the horizontal axis compresses by a factor of 60, so heights multiply by 60 to keep the area at 1: the peak becomes about 108.
4. $F - G$ is a **constant**. Both have derivative f , so $(F - G)' = 0$, and only a constant has zero derivative everywhere on an interval. It does not affect the definite integral because it is added at b and subtracted at a , cancelling.
5. The **interval**, not the strip count. Cutting off at ± 3 discards the tails of e^{-x^2} , and that missing area is a fixed amount which no refinement of the strips can recover — it is why the estimate stops improving rather than converging slowly. Widening to ± 6 fixes it. The diagnostic habit worth taking away: if refining the mesh stops helping, the error is in the problem you set up, not in how finely you chopped it.

Where this shows up next

S5 is the lesson this one exists for: a continuous random variable is described by a density, and every probability it gives you is an area computed by integration. **S6** meets e^{-x^2} properly, as the normal distribution, and lives with the fact that its integral has no formula. **S7** and **S8** read areas in the tails of a distribution to say how far an estimate might be from the truth.

Data as vectors

Unit 7 · Lesson 1 of 4 · Mathematics · M8 Reading: 4 pages With practice: 1.5 hours Assumes: M1

Why this lesson is here

A dataset is a grid of numbers, and so far you have treated it one number at a time. This lesson treats a whole row or a whole column as a **single object** — a vector — with an arithmetic of its own.

That change of view pays for itself immediately. The distance between two students becomes the length of a vector. The correlation between two variables turns out to be the cosine of the angle between two vectors, which is why it can never leave the range -1 to 1 . The gradient of M6 was a vector all along. And every lesson from here to the end of the Mathematics track — matrices, solving $A\mathbf{x} = \mathbf{b}$, least squares, the SVD — is built on the two operations defined below.

Before you start

You should be able to:

- read subscript and $\sum$ notation — M1;
- find a point on a pair of axes — M1;
- make and index a NumPy array — P6;
- use `.sum()`, and recall that `*` and `@` are different — P7.

Quick check. How far is the point (3, 4) from the origin?

💡 Show the answer

$$\sqrt{3^2 + 4^2} = \sqrt{25} = 5.$$

Pythagoras. Everything this lesson says about length in 238 dimensions is that same calculation with more terms under the square root, and nothing else.

The idea

Take two students from the cohort and record two numbers for each — hours of practice and final score:

	hours	score
S0001	12.9	83.6
S0002	22.8	56.6

Each row is a pair of numbers, so each is a point on a pair of axes. Draw an arrow to it from the origin and you have a **vector**: an ordered list of numbers, which you can also think of as a position, or as a step of a certain size in a certain direction.

Now the move that matters. A student's row is a vector; so is a *column*. The 238 usable final scores in the cohort form one vector with 238 components, and the 238 practice hours form another. You cannot draw those, and you do not need to: the arithmetic below is written with $\sum$ and works for any

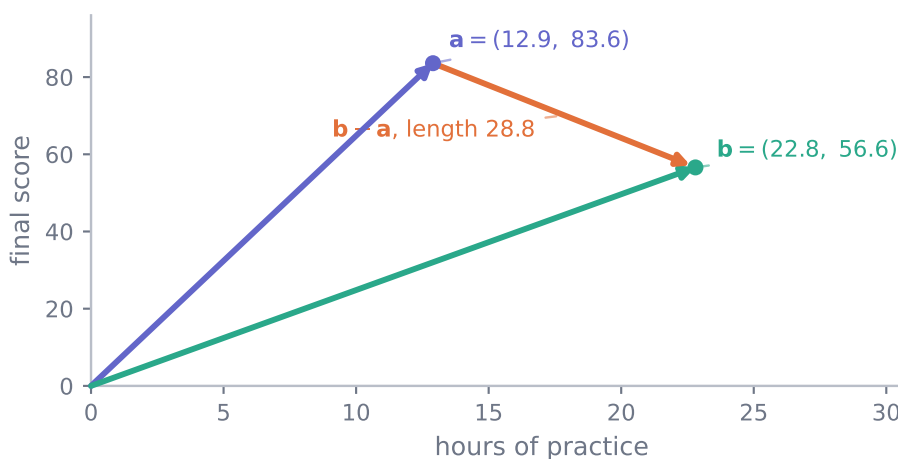


Figure 1: Two students as vectors, and the step from one to the other. The difference vector $\mathbf{b} - \mathbf{a}$ is the arrow joining their tips, and its length, 28.8, is the distance between the two students. The two axes are on different scales — hours span 34, marks span 96 — so the arrows' apparent directions are not the real angles. That distortion is not a drawing error but the thing Exercise 2 is about: geometry on raw data depends on the units each column happens to use.

number of components at all. The picture is a two-dimensional aid to intuition. The formulas are the thing.

The mechanics

Notation

A vector is an ordered list of n real numbers, written in bold: $\mathbf{v} = (v_1, v_2, \dots, v_n)$. The set of all of them is written $\mathbb{R}^n$, and n is the **dimension**. A student's three measurements live in $\mathbb{R}^3$; a column of 238 scores lives in $\mathbb{R}^{238}$.

Note what n counts. It is the number of *components in this vector*, not the number of variables in the dataset. A column vector of 238 scores has dimension 238, however few columns the table has.

Adding and scaling

Both work component by component:

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1, \dots, a_n + b_n), \quad c \mathbf{a} = (c a_1, \dots, c a_n).$$

Both mean something on data. Adding the vector $(5, 5, \dots, 5)$ to a column of marks gives everyone five more marks. Multiplying a column of hours by 60 converts it to a column of minutes — a change of units is a scalar multiple. Vectors of *different* lengths cannot be added at all, which is a feature: it is the arithmetic refusing to combine 238 scores with 12 of something else.

Length and distance

The length of a vector, written $\|\mathbf{v}\|$, is Pythagoras with n terms:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2} = \sqrt{\sum_{i=1}^n v_i^2}.$$

The distance between two vectors is the length of the step between them, $\|\mathbf{a} - \mathbf{b}\|$. That single formula is what lets an algorithm ask “which two students are most alike?” — and it is worth noticing straight away that it takes the columns’ units at face value, so a variable measured in large numbers dominates it. That is the same scaling problem that sank the first attempt in M6, arriving in a new place.

The dot product

Multiply matching components and add up the results:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \sum_{i=1}^n a_i b_i.$$

The answer is a single number, not a vector. Two immediate consequences: taking the dot product of a vector with itself gives $\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|^2$, and in NumPy this is `a @ b` — `a * b` is the componentwise product from P7, a vector, and a different thing entirely.

The reason the dot product is worth having a name is that it is also a statement about *angle*:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta, \quad \text{so} \quad \cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|},$$

where θ is the angle between the two vectors. One formula, two readings — a sum of products on the left, a geometric fact on the right. The identity is checked numerically at the end of the lesson rather than proved here.

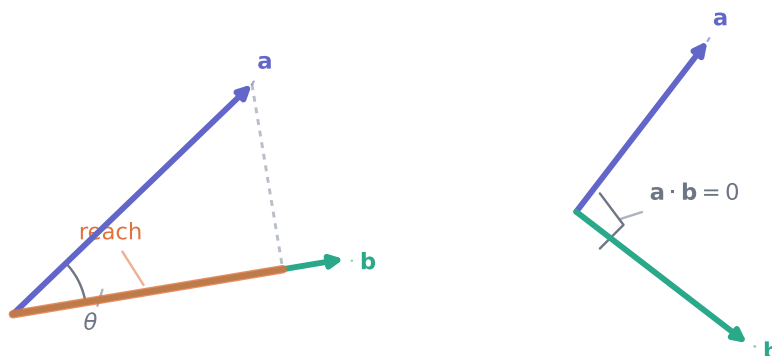


Figure 2: Left: the dot product as reach. Drop $\mathbf{a}$ perpendicularly onto the direction of $\mathbf{b}$; the dot product is the length of $\mathbf{b}$ times how far $\mathbf{a}$ reaches along it. Right: when the angle is a right angle the reach is zero, so the dot product is zero however long the two vectors are.

Since $\cos \theta$ never leaves $[-1, 1]$, the dot product can never exceed $\|\mathbf{a}\| \|\mathbf{b}\|$ in size. And $\theta = 90^\circ$ gives $\cos \theta = 0$: two nonzero vectors are **orthogonal** — at right angles — exactly when their dot product is zero. (The zero vector has zero dot product with everything and no direction at all, so it is the one case where the angle is simply undefined.) That is the definition used in $\mathbb{R}^{238}$, where “at right angles” cannot be checked by looking.

Worked example

Correlation is the cosine of an angle.

Take the two cohort columns as vectors in $\mathbb{R}^{238}$ and **centre** them: subtract each one’s mean, so that each has mean zero. Write $\tilde{h}$ and $\tilde{s}$ for the centred hours and centred scores.

```

import numpy as np
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h, s = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()
hc, sc = h - h.mean(), s - s.mean()

cos_theta = (hc @ sc) / (np.linalg.norm(hc) * np.linalg.norm(sc))
print(f"dimension          {hc.shape[0]}")
print(f"hc . sc              {hc @ sc:.4f}")
print(f"||hc||, ||sc||         {np.linalg.norm(hc):.4f}, {np.linalg.norm(sc):.4f}")
print(f"cos(theta)             {cos_theta:.6f}")
print(f"np.corrcoef             {np.corrcoef(h, s)[0, 1]:.6f}")
print(f"theta                  {np.degrees(np.arccos(cos_theta)):.2f} degrees")

```

```

dimension          238
hc . sc            -3777.9636
||hc||, ||sc||     125.2584, 129.0720
cos(theta)         -0.233679
np.corrcoef        -0.233679
theta              103.51 degrees

```

The correlation coefficient and the cosine are not merely close; they are the same calculation. That is what correlation *is*: the cosine of the angle between two centred columns. Three things follow at once, none of which you have to memorise separately.

- It cannot exceed 1 or fall below -1 , because no cosine does.
- $+1$ means angle zero — the two centred columns point the same way, so one is a positive multiple of the other and the scatter is a perfect rising line.
- 0 means orthogonal.

Here the angle is 103.5° : slightly more than a right angle, which is the geometric version of “a weak negative relationship”.

Why the centring is not optional: without it the same calculation gives 0.876 , which would announce a strong positive relationship in data that has a negative one. Uncentred, both columns are entirely positive, so every term of the sum is positive and the angle between them is forced to be small. Centring removes that artefact — and it has a tidy vector meaning, which is Exercise 3.

The dot product is a sum over students, so you can ask which students it is made of. Each contributes $\tilde{h}_i \times \tilde{s}_i$, which is positive when the student is on the same side of average in both columns and negative when they are on opposite sides:

That picture is worth holding on to, because it is what a correlation coefficient is doing under the arithmetic: counting students for and against, weighted by how far from average they are. It also shows how little separates the two groups here — 160 against 78 is not a landslide, and -0.234 is not a strong number. [S12](#) shows that even this weak negative reverses inside every background group.

Try it yourself

Exercise 1 For $\mathbf{a} = (3, -1, 2)$ and $\mathbf{b} = (1, 4, 0)$, find $\mathbf{a} + \mathbf{b}$, $2\mathbf{a}$, $\mathbf{a} \cdot \mathbf{b}$ and $\|\mathbf{a}\|$.

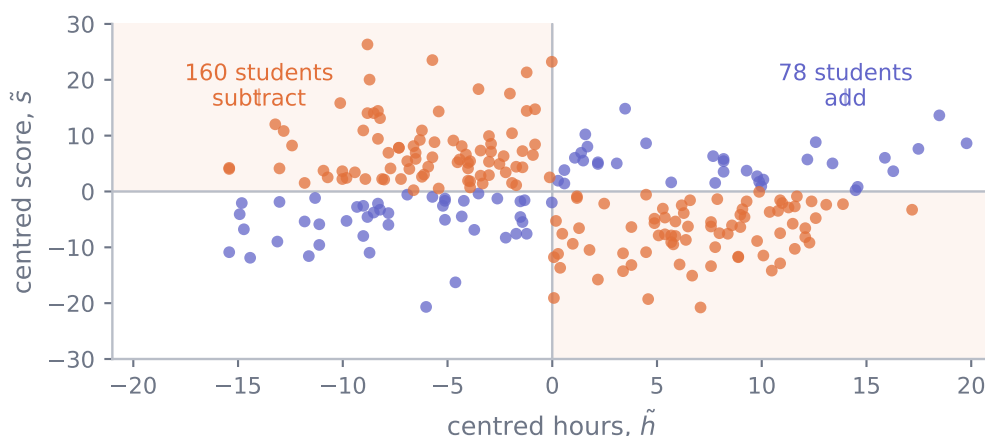


Figure 3: The 238 students, each plotted at its centred hours and centred score. A student in a shaded quadrant is above average in one column and below in the other, so contributes a negative term to the dot product. There are 160 of those against 78 of the others, and they outweigh them: the sum comes to -3778 , and dividing by the two lengths turns it into the correlation -0.234 .

💡 Show the solution

$$\mathbf{a} + \mathbf{b} = (4, 3, 2), \quad 2\mathbf{a} = (6, -2, 4).$$

$$\mathbf{a} \cdot \mathbf{b} = (3)(1) + (-1)(4) + (2)(0) = 3 - 4 + 0 = -1.$$

$$\|\mathbf{a}\| = \sqrt{9 + 1 + 4} = \sqrt{14} \approx 3.742.$$

The one to check yourself on is the dot product: the answer is the number -1 , not the vector $(3, -4, 0)$. That vector is the componentwise product — what $\mathbf{a} * \mathbf{b}$ computes in NumPy — and summing it is the last step you must not forget.

Exercise 2 Two students are recorded as $(89.1, 12.9, 83.6)$ and $(42.4, 22.8, 56.6)$, in the order (diagnostic score, hours, final score). How far apart are they? Which of the three measurements is doing most of the work, and why is that a problem?

💡 Show the solution

The difference is $(89.1 - 42.4, 12.9 - 22.8, 83.6 - 56.6) = (46.7, -9.9, 27.0)$, so

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{46.7^2 + 9.9^2 + 27^2} = \sqrt{2180.89 + 98.01 + 729} = \sqrt{3007.9} \approx 54.84.$$

The diagnostic score contributes 2180.89 of that 3007.9 — about 72% — not because it matters more but because it is measured on a scale that produces bigger gaps. Hours of practice, which run from 1 to 36, can never contribute much against a variable running from 27 to 100.

This is why distances are usually computed on standardised columns. It is the same lesson as M6's diverging descent: a formula that adds squared differences treats “one unit” as meaning the same thing in every column, and it is your job to make that true before you use it.

Exercise 3 Let $\mathbf{1} = (1, 1, \dots, 1)$ and let $\tilde{\mathbf{v}}$ be a centred vector — one whose components have mean zero. Show that $\tilde{\mathbf{v}} \cdot \mathbf{1} = 0$, and say what that means in words.

 Show the solution

$$\tilde{\mathbf{v}} \cdot \mathbf{1} = \sum_{i=1}^n \tilde{v}_i \times 1 = \sum_{i=1}^n \tilde{v}_i = n \times \text{mean}(\tilde{\mathbf{v}}) = 0.$$

In words: **a centred vector is orthogonal to the all-ones vector**. Centring is not just a convenience for making the arithmetic tidy — it is the geometric operation of removing everything the column has in common with “the same value for everybody”, leaving only the variation about that value.

That is the honest reason the worked example had to centre before measuring an angle. Correlation is meant to be about variation, and the uncentred vectors still carry their overall level, which is what dragged the answer to 0.876.

Check it in Python

The claim to test is the identity $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$, which the lesson asserted without proof. Nothing in the definition of the dot product mentions an angle, so this needs checking rather than believing.

In two dimensions the angle can be measured independently, straight off the axes, with `arctan2`:

```
a2, b2 = np.array([2.6, 2.5]), np.array([3.6, 0.6])

angle_a = np.arctan2(a2[1], a2[0])          # each vector's own direction
angle_b = np.arctan2(b2[1], b2[0])
theta_geometry = abs(angle_a - angle_b)

theta_dot = np.arccos((a2 @ b2) / (np.linalg.norm(a2) * np.linalg.norm(b2)))

print(f"angle measured from the axes  {np.degrees(theta_geometry):.6f} degrees")
print(f"angle from the dot product    {np.degrees(theta_dot):.6f} degrees")
print(f"agree: {np.isclose(theta_geometry, theta_dot)}")
```

```
angle measured from the axes  34.414375 degrees
angle from the dot product    34.414375 degrees
agree: True
```

That works only because two dimensions can be drawn. The formula, though, has no such restriction, and the consequence worth testing in high dimensions is that $|\mathbf{a} \cdot \mathbf{b}|$ can never exceed $\|\mathbf{a}\| \|\mathbf{b}\|$ — because a cosine cannot exceed 1. Try to break it:

```
rng = np.random.default_rng(2026)
ratios = []
for _ in range(10_000):
    x, y = rng.normal(size=50), rng.normal(size=50)
    ratios.append(abs(x @ y) / (np.linalg.norm(x) * np.linalg.norm(y)))
ratios = np.array(ratios)

parallel = rng.normal(size=50)
tripled = abs(parallel @ (3*parallel)) / (np.linalg.norm(parallel) *
    - np.linalg.norm(3*parallel))
```

```
print(f"10,000 random pairs in R^50:  largest |cos| {ratios.max():.6f}"
      f"      typical |cos| {np.median(ratios):.6f}")
print(f"a vector against 3x itself:   |cos| {tripled:.6f}")
```

```
10,000 random pairs in R^50:  largest |cos| 0.515970   typical |cos| 0.094704
a vector against 3x itself:   |cos| 1.000000
```

Ten thousand attempts get nowhere near 1, and the bound is reached only when the two vectors are parallel. The typical value is worth a second look too. The code takes absolute values, so what the median says is that half of all random pairs have $|\cos \theta|$ below 0.09 — an angle within about 5° of a right angle, on one side or the other, since positive and negative are equally likely. In fifty dimensions, two directions picked at random are nearly orthogonal — a fact with no analogue in the plane, and one to remember before reading much into a small correlation.

Note also the last line: tripling a vector does not change the angle at all. **Angles are scale-free and dot products are not**, which is exactly why correlation is reported instead of the raw sum of products.

Finally, the trap that costs the most time in practice:

```
x, y = np.array([3.0, -1.0, 2.0]), np.array([1.0, 4.0, 0.0])
print("x * y =", x * y, " -> a vector, the componentwise product")
print("x @ y =", x @ y, " -> a number, the dot product")
print("(x * y).sum() =", (x * y).sum(), " -> the same number, the long way")
```

```
x * y = [ 3. -4.  0.] -> a vector, the componentwise product
x @ y = -1.0 -> a number, the dot product
(x * y).sum() = -1.0 -> the same number, the long way
```

$*$ and $@$ both run without error and return different kinds of thing. Nothing will tell you which one you meant.

Common wrong turns

Where this usually goes wrong

Writing $a * b$ for the dot product In NumPy $*$ multiplies component by component and gives a *vector*; $@$ gives the single number. Both are legal, so the mistake surfaces later as a shape error somewhere unrelated, or as a silently wrong answer.

Reading a large dot product as a strong relationship The dot product grows if you simply rescale a column — measure hours in minutes and it multiplies by 60. Only after dividing by both lengths is the number comparable with anything, and that quotient is the cosine.

Correlating without centring On these two columns it turns -0.234 into $+0.876$, which is not a small error but the opposite conclusion. Uncentred columns of positive numbers all point roughly the same way.

Trusting distance across columns with different scales $\|a - b\|$ adds squared differences as though one unit meant the same thing in every column. A diagnostic score out of 100 will drown a column of hours whatever the two variables actually mean.

Confusing the dimension with the number of variables A column of 238 scores is one vector in $\mathbb{R}^{238}$. It is not “a 238-variable problem”, and n here counts rows.

Expecting the pictures to keep working The formulas hold in any dimension; two- and three-dimensional *intuition* does not always. Two directions drawn at random in $\mathbb{R}^{50}$ are nearly at right angles almost every time, as the check above measures — nothing in the plane prepares you for that.

Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. $\mathbf{a} = (1, 2, 2)$. What is $\|\mathbf{a}\|$?
 - a. 5
 - b. 3
 - c. $\sqrt{5}$
 - d. 9
2. Two centred columns have $\cos \theta = 0$. What does that say about the vectors, and what does it say about the two variables?
3. You double every entry of $\mathbf{b}$. What happens to $\mathbf{a} \cdot \mathbf{b}$, and what happens to the angle between them?
4. Someone reports a correlation of 1.4. Without seeing their data, how do you know it is wrong?
5. $\mathbf{x} * \mathbf{y}$ and $\mathbf{x} @ \mathbf{y}$ both run and neither raises an error. What is different about what they return, and how would you catch the mistake?

Show the answers

1. **(b)** 3. $\sqrt{1 + 4 + 4} = \sqrt{9} = 3$. Option (a) adds the components; (c) forgets to square them; (d) is $\|\mathbf{a}\|^2$, which is $\mathbf{a} \cdot \mathbf{a}$ — a related quantity, and a common slip when the square root gets dropped.
2. The vectors are **orthogonal**, at right angles. For the variables it means the correlation is zero: no *linear* relationship. It does not mean the two are unrelated — a strong curved relationship can have correlation zero, which is why a scatter plot is not optional.
3. The dot product **doubles**; the angle is **unchanged**. Scaling changes a vector's length, not its direction, and the two extra factors of 2 cancel in $\mathbf{a} \cdot \mathbf{b} / (\|\mathbf{a}\| \|\mathbf{b}\|)$. This is the whole reason correlation is scale-free.
4. Correlation is a cosine, and no cosine exceeds 1. So the number cannot be a correlation of anything — the mistake is arithmetic, not data, and no amount of looking at the dataset will explain it.
5. $\mathbf{x} * \mathbf{y}$ returns a **vector** of componentwise products; $\mathbf{x} @ \mathbf{y}$ returns a single **number**. The reliable catch is to check the shape, or to assert it: `assert np.isscalar(result) or result.shape == ()`. Left unchecked it propagates — a vector where a number was expected often broadcasts happily into something with a plausible shape and no meaning.

Where this shows up next

M9 stacks vectors into a matrix, and matrix multiplication turns out to be a grid of dot products. **M11** solves least squares by finding the vector in a subspace closest to the data — “closest” being the length defined here, and the answer being characterised by an orthogonality condition. **M12** looks for the

directions in which a dataset varies most, which is a question about angles. **S3** and **S10** use correlation as a summary and worry about what it does and does not license you to say.

Matrices as operations on data

Unit 7 · Lesson 2 of 4 · Mathematics · M9 Reading: 5 pages With practice: ☒ 2 hours Assumes: M8

Why this lesson is here

Your dataset already is a matrix: one row per student, one column per variable — 240 by 7 as the cohort file sits on disk. So the first thing a matrix can be is a table, and you knew that.

The useful idea is the second one. A matrix is also an **operation** — feed it a vector and it hands back another vector — and almost everything done to data in practice is one of those operations written down. Making a *linear* prediction for every row at once, rotating a cloud of points, re-weighting a set of columns: each of those is a single multiplication. Once you can read $A\mathbf{x}$ two ways, the whole of the rest of this track opens up, because M10, M11 and M12 are all questions about what a particular matrix does.

Before you start

You should be able to:

- take a dot product, and say what it means — M8;
- compute a vector's length, and recognise orthogonality — M8;
- read subscript notation like a_{ij} — M1;
- index a 2-D NumPy array and check its `.shape` — P6.

Quick check. What is $(2, 5) \cdot (3, -1)$?

💡 Show the answer

$$(2)(3) + (5)(-1) = 6 - 5 = 1.$$

Every entry of every matrix product in this lesson is one of these. If the dot product is not automatic yet, reread M8 first — the rest of this lesson is bookkeeping about *which* dot products to take.

The idea

Write the cohort's practice hours and a column of ones side by side:

$$X = \begin{pmatrix} 1 & h_1 \\ 1 & h_2 \\ \vdots & \vdots \\ 1 & h_{238} \end{pmatrix}$$

That is a **matrix**: a rectangular grid of numbers, here with 238 rows and 2 columns, which we call its **shape**, 238×2 . Rows first, always.

Now take the fitted line from M6, whose two parameters were an intercept and a slope, and put them in a vector $\theta = (69.2261, -0.2408)$. The prediction for student i is

$$\hat{y}_i = 69.2261 \times 1 + (-0.2408) \times h_i,$$

which is exactly the dot product of row i of X with θ . Do it for every row and you have all 238 predictions. **That** is what a matrix times a vector means:

$$X\theta = \begin{pmatrix} \text{row}_1 \cdot \theta \\ \text{row}_2 \cdot \theta \\ \vdots \\ \text{row}_{238} \cdot \theta \end{pmatrix}$$

inner shapes must match:
 $(3, 2) @ (2,)$ is legal,
 $(2,) @ (3, 2)$ is not

row 2 of A , dotted with x : $(2, 1) \cdot (5, 3) = 10 + 3 = 13$

Figure 1: A matrix times a vector, with the shapes that make it legal. Each entry of the answer is one dot product: a whole row of the matrix against the whole input vector. The highlighted row gives the highlighted output entry, $2 \times 5 + 1 \times 3 = 13$. The matrix has as many columns as the input has entries — that is the rule — and as many rows as the answer.

Two readings of Ax , and you need both.

Rows. Each output entry is one dot product of a row with the input. This is the reading for “apply a rule to every observation” — one prediction per student.

Columns. Equivalently, Ax is the columns of A added together after being scaled by the entries of x :

$$\begin{pmatrix} 4 & 0 \\ 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 5 \\ 3 \end{pmatrix} = 5 \begin{pmatrix} 4 \\ 2 \\ 1 \end{pmatrix} + 3 \begin{pmatrix} 0 \\ 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 20 \\ 13 \\ 14 \end{pmatrix}$$

Same answer, different story. The column reading says that whatever x you choose, Ax can only ever be a combination of A ’s columns — it cannot escape them. That single observation is what [M10](#) and [M11](#) are about, so it is worth a picture.

The mechanics

Shape, and the one rule

A matrix with m rows and n columns has shape $m \times n$; the entry in row i , column j is a_{ij} , row index first. A vector of n numbers behaves like an $n \times 1$ matrix.

For AB to exist, **the number of columns of A must equal the number of rows of B** . Then

$$(m \times n)(n \times p) = (m \times p) :$$

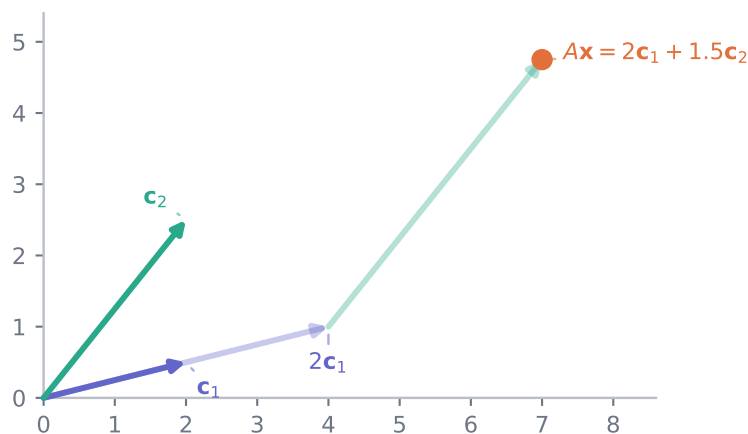


Figure 2: The column reading in the plane. Whatever the two weights are, Ax is built by walking some multiple of the first column and then some multiple of the second. Here $2c_1 + 1.5c_2$ lands on (7, 4.75). With two columns pointing in different directions every point in the plane is reachable; if they pointed the same way, only one line would be.

the inner dimensions must match and they disappear; the outer ones survive. Most of the errors you will make in this subject are this rule being broken, and the error message will say so in exactly these numbers.

The product

Entry (i, j) of AB is the dot product of **row i of A** with **column j of B** :

$$(AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj}.$$

A matrix product is a grid of dot products, one per (row, column) pair, and that is the whole definition. Everything else is consequence.

Order matters. AB and BA are usually different, and often only one of them even has legal shapes:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}, \quad \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}$$

The second matrix swaps columns when it multiplies from the right and swaps rows when it multiplies from the left. So “multiply by B ” is not one instruction — which side you multiply on is part of the instruction.

Matrix multiplication *is* associative, $(AB)C = A(BC)$, which is what lets you combine a chain of operations into one matrix and apply it once.

The identity

I has ones down the diagonal and zeros elsewhere, and $IA = AI = A$. It is the operation that does nothing, and it is the yardstick for M10’s question: given A , is there a matrix that undoes it?

The transpose

A^T is A with rows and columns exchanged: $(A^T)_{ij} = a_{ji}$, so an $m \times n$ matrix becomes $n \times m$. Two facts earn their keep:

- $(AB)^T = B^T A^T$ — the order reverses.
- For a data matrix X , the product $X^T X$ is small, square, and full of the sums that summarise the columns. It is the object every least-squares calculation is built out of, and the worked example below takes it apart.

The transpose is also how a dot product is written in matrix notation: $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b}$, a $(1 \times n)(n \times 1) = (1 \times 1)$ product — a single number, as [M8](#) said.

Worked example

One multiplication predicts the whole cohort.

Build the 238×2 design matrix — a column of ones for the intercept and the practice hours — and take the fit from [M6](#) at full precision.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h, y = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()

X = np.column_stack([np.ones(len(h)), h])
slope, intercept = np.polyfit(h, y, 1)
theta = np.array([intercept, slope])

predictions = X @ theta
print(f"X {X.shape} @ theta {theta.shape} -> predictions {predictions.shape}")
print(f"theta = {theta}")
print(f"first three predictions {predictions[:3]}")
print(f"by hand, student 1:      {theta[0] * 1 + theta[1] * h[0]:.4f}")
```

```
X (238, 2) @ theta (2,) -> predictions (238,)
theta = [69.22611682 -0.24079311]
first three predictions [66.11988566 63.73603384 68.81676853]
by hand, student 1:      66.1199
```

No loop over students appears anywhere. The shapes did the work: $(238, 2)$ against $(2,)$ can only combine one way, and it produces one number per row.

Now the small square matrix. $X^T X$ has shape $(2, 238) \times (238, 2) = (2, 2)$ — four numbers, whatever the number of students:

```
print("X.T @ X =\n", X.T @ X)
print("\nX.T @ y =", X.T @ y)
print(f"\nn = {len(h)}, sum h = {h.sum():.1f}, sum h^2 = {(h**2).sum():.2f}")
print(f"sum y = {y.sum():.1f}, sum h*y = {(h*y).sum():.2f}")
```

```
X.T @ X =
[[ 238.    3908.4 ]
 [ 3908.4  79872.82]]
```

```
X.T @ y = [ 15534.7  251330.53]
```

```
n = 238, sum h = 3908.4, sum h^2 = 79872.82
sum y = 15534.7, sum h*y = 251330.53
```

Read the entries. $X^T X$ is $\begin{pmatrix} n & \sum h \\ \sum h & \sum h^2 \end{pmatrix}$ and $X^T \mathbf{y}$ is $(\sum y, \sum hy)$: the 476 numbers of the design matrix have been compressed into four, and the 238 outcomes into two. Those six numbers are all that a straight-line fit needs, which is why fitting one to a billion rows is not a billion times harder than fitting it to a thousand.

And the property that makes it the *best* line. Take the residuals $\mathbf{r} = \mathbf{y} - X\theta$ and multiply by X^T :

```
r = y - X @ theta
print("X.T @ r      =", X.T @ r)
print(f"sum of residuals      {r.sum():+.3e}")
print(f"sum of residual x hours {(r * h).sum():+.3e}")
print("orthogonal to both columns of X:", np.allclose(X.T @ r, 0))
```

```
X.T @ r      = [7.89412979e-12  1.21630706e-10]
sum of residuals      +7.887e-12
sum of residual x hours +1.216e-10
orthogonal to both columns of X: True
```

Both entries are zero to within floating-point noise — note that this needed `np.allclose` rather than `== 0`, for the reasons in [P2](#). Written out, $X^T \mathbf{r} = \mathbf{0}$ says the residual vector is **orthogonal to every column of X** , in exactly the sense of [M8](#): its dot product with the ones column is zero, and its dot product with the hours column is zero.

That is not a coincidence of this dataset. It is the defining property of the least-squares fit, and [M11](#) turns it round — imposes $X^T \mathbf{r} = \mathbf{0}$ and solves for θ — to get the answer without any descent at all.

Try it yourself

Exercise 1 Let $A = \begin{pmatrix} 1 & 2 \\ 0 & 3 \\ 4 & 1 \end{pmatrix}$ and $\mathbf{x} = (2, -1)$. Compute $A\mathbf{x}$ two ways — by rows and by columns — and confirm they agree.

 Show the solution

By rows, each entry is a dot product:

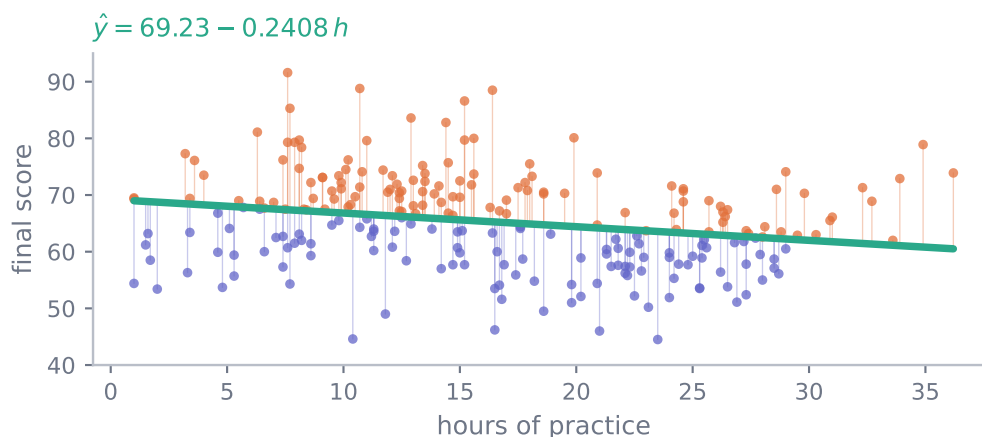


Figure 3: The fitted line, with each student's residual drawn as the vertical gap to it. Orange gaps are above the line, indigo below. $X^T \mathbf{r} = \mathbf{0}$ is the statement that these gaps balance twice over: they sum to zero, and they still sum to zero after each is weighted by that student's hours of practice.

$$A\mathbf{x} = \begin{pmatrix} (1)(2) + (2)(-1) \\ (0)(2) + (3)(-1) \\ (4)(2) + (1)(-1) \end{pmatrix} = \begin{pmatrix} 0 \\ -3 \\ 7 \end{pmatrix}.$$

By columns, scale and add:

$$2 \begin{pmatrix} 1 \\ 0 \\ 4 \end{pmatrix} - 1 \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \\ 8 \end{pmatrix} - \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -3 \\ 7 \end{pmatrix}.$$

Shapes: $(3, 2)$ times $(2,)$ gives $(3,)$. The answer has one entry per *row* of A , and $\mathbf{x}$ needs one entry per *column* — get those the wrong way round and NumPy raises the mismatch rather than guessing.

Exercise 2 Let $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 1 & 0 \\ 5 & 1 \end{pmatrix}$. Compute AB and BA .

💡 Show the solution

$$AB = \begin{pmatrix} 1+10 & 0+2 \\ 3+20 & 0+4 \end{pmatrix} = \begin{pmatrix} 11 & 2 \\ 23 & 4 \end{pmatrix}, \quad BA = \begin{pmatrix} 1 & 2 \\ 5+3 & 10+4 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 8 & 14 \end{pmatrix}.$$

Different — and not in some subtle way. Both products are legal here because both matrices are square and the same size; for non-square matrices one order is usually illegal outright, which at least tells you immediately.

Worth checking a habit: for AB , entry $(2, 1)$ used row 2 of A and column 1 of B , giving $(3)(1) + (4)(5) = 23$. If you found yourself using column 2 of A , you have the convention backwards.

Exercise 3 X is a data matrix with 500 rows and 4 columns, and $\mathbf{y}$ has 500 entries. Which of these are legal, and what shape does each result have?

$X\mathbf{y}$, $X^T\mathbf{y}$, X^TX , XX^T , $\mathbf{y}^TX$.

 Show the solution

expression	shapes	verdict
Xy	(500, 4)(500,)	illegal — 4 columns against 500 entries
$X^T y$	(4, 500)(500,)	legal, gives (4,)
$X^T X$	(4, 500)(500, 4)	legal, gives (4, 4)
XX^T	(500, 4)(4, 500)	legal, gives (500, 500)
$y^T X$	(1, 500)(500, 4)	legal on paper, gives (1, 4)

That last row carries a warning. On paper y is a 500×1 column and y^T is a 1×500 row, so the product is 1×4 . NumPy makes no such distinction for a one-dimensional array: $y.T$ is the same object as y , and $y.T @ x$ comes back with shape (4,). The mathematics and the library agree about the four numbers and disagree about how to wrap them — worth knowing before it surprises you.

The pair worth staring at is the last two of the square ones. $X^T X$ is 4×4 — sixteen numbers, one per pair of columns, and it is what least squares needs. XX^T is 500×500 — a quarter of a million numbers, one per pair of *students*. Both are legal, both are meaningful, and writing one when you meant the other is a mistake whose only symptom may be that your program has become extremely slow.

Check it in Python

The lesson's central claim is that a matrix product is a grid of dot products. Test it by building one from nothing but the dot product of [M8](#) and comparing against @:

```
rng = np.random.default_rng(2026)
A = rng.normal(size=(4, 3))
B = rng.normal(size=(3, 5))

by_hand = np.zeros((A.shape[0], B.shape[1]))
for i in range(A.shape[0]):
    for j in range(B.shape[1]):
        by_hand[i, j] = A[i, :] @ B[:, j]      # row i of A against column j of B

print("shapes:", A.shape, "@", B.shape, "->", (A @ B).shape)
print("built from dot products matches @ :", np.allclose(by_hand, A @ B))
```

```
shapes: (4, 3) @ (3, 5) -> (4, 5)
built from dot products matches @ : True
```

Then the claims about shape and order, including the ones that are supposed to fail:

```
print("(AB)^T == B^T A^T :", np.allclose((A @ B).T, B.T @ A.T))
print("(AB)^T == A^T B^T :", np.allclose((A @ B).T, A.T @ B.T))
    if A.T.shape[1] == B.T.shape[0] else "not even a legal shape")

try:
```

```
B @ A
except ValueError as exc:
    print("B @ A raises:", exc)
```

```
(AB)^T == B^T A^T : True
```

```
(AB)^T == A^T B^T : not even a legal shape
```

```
B @ A raises: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc signature
```

The middle line is the one to look at: reversing the order in a transpose is not a stylistic choice you can get away with — here the shapes do not even permit it. And the error message from the last block names the two dimensions that disagree, which is almost always enough to find the mistake.

One more, because it is the single most common bug in code that uses matrices:

```
M = np.array([[1.0, 2.0], [3.0, 4.0]])
print("M * M =\n", M * M)
print("M @ M =\n", M @ M)
print("same?", np.array_equal(M * M, M @ M))
```

```
M * M =
[[ 1.  4.]
 [ 9. 16.]]
M @ M =
[[ 7. 10.]
 [15. 22.]]
same? False
```

$*$ squares each entry where it stands. $@$ takes the four dot products. Both return a 2×2 array, both run without complaint, and only one of them is matrix multiplication — the point P7 made before you had the definition to attach it to.

Common wrong turns

Where this usually goes wrong

Using $*$ when you mean $@$ On two square matrices of the same size, both are legal and both return the right shape. Nothing will fail. The answer is simply wrong, and it can travel a long way before anything notices.

Assuming $AB = BA$ They differ in general, and frequently only one is even legal. “Multiply by B ” is an incomplete instruction until you say which side.

Getting rows and columns the wrong way round a_{ij} is row i , column j ; shape is (rows, columns); entry (i, j) of a product uses row i of the *left* matrix and column j of the *right* one. Every one of those is row-first. Mix the convention up — column i of the left against row j of the right — and you get $(BA)^T$, which for square matrices has exactly the right shape and is neither AB nor its transpose. The shape check will not save you.

Distributing a transpose without reversing $(AB)^T = B^T A^T$, not $A^T B^T$. In the usual case the second is not even a legal product, which is a mercy.

Confusing $X^T X$ with XX^T For a 500×4 data matrix these are 4×4 and 500×500 . Both compute; one of them will exhaust your memory on a real dataset.

Reading a matrix as *only* a table The table view says what is stored. The operation view says what it does, and that is the view **M10**, **M11** and **M12** need — a matrix takes a vector and returns a vector, and the columns are the only directions it can return.

Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A is 3×5 and B is 5×2 . What is the shape of AB ?
 - a. 3×2
 - b. 5×5
 - c. 2×3
 - d. illegal
2. In one sentence, what is entry (i, j) of AB ?
3. X is your 238×2 design matrix. Give the shapes of $X^T X$ and XX^T , and say which one you want.
4. $A\mathbf{x}$ can never be anything other than a combination of the columns of A . Why, and what does that imply if the two columns of a 3×2 matrix are parallel?
5. Your code runs, the shapes are right, and the numbers are wrong. Name two matrix-specific mistakes that behave exactly like that.

Show the answers

1. **(a)** 3×2 . The inner dimensions (5 and 5) match and vanish; the outer ones survive. Option (b) keeps the inner dimension, which is the one that disappears; (c) has the factors the wrong way round — that would be the shape of $B^T A^T$.
2. The **dot product of row i of A with column j of B** . Row from the left matrix, column from the right one.
3. $X^T X$ is 2×2 ; XX^T is 238×238 . You want $X^T X$ — four numbers summarising the columns, which is what fitting a line needs. The other is one number per pair of students, and it grows with the square of your sample size.
4. Because $A\mathbf{x}$ is the columns scaled by the entries of $\mathbf{x}$ and added — that is the column reading of the product, so no choice of $\mathbf{x}$ can produce a direction the columns do not already span. If the two columns of a 3×2 matrix are parallel, they span only a single line, so every possible $A\mathbf{x}$ lies on that line and almost every target is unreachable. **M10** is about what to do then.
5. Writing $*$ for $@$ on two square matrices of the same size, and a stray transpose — using A^T where A was meant, on a square matrix. Both keep the shape and change the answer, so neither raises anything. (A third: taking entry (i, j) from column i of the left matrix and row j of the right, which yields $(BA)^T$ — right shape, wrong matrix. On the pair in Exercise 2 it returns $\begin{pmatrix} 1 & 8 \\ 2 & 14 \end{pmatrix}$ while $AB = \begin{pmatrix} 11 & 2 \\ 23 & 4 \end{pmatrix}$.)

Where this shows up next

M10 asks the reverse question: given A and a target $\mathbf{b}$, which $\mathbf{x}$ satisfies $A\mathbf{x} = \mathbf{b}$ — and what to do when none does. **M11** imposes the orthogonality condition $X^T \mathbf{r} = \mathbf{0}$ found in the worked example

and solves it, which is least squares in closed form. **M12** asks which directions a matrix stretches most, and finds them in $X^T X$. **S11** fits this exact design matrix to the cohort and asks what the fitted slope licenses you to say.

Solving $Ax = b$

Unit 8 · Lesson 1 of 3 · Mathematics · M10 Reading: 5 pages With practice: 2 hours Assumes: M9

Why this lesson is here

M9 asked what a matrix *does* to a vector. This lesson asks the reverse: given the matrix and the answer you want, which input produces it? That question is what “solving” means, and it is where fitting a model ends up once you stop walking downhill and write the condition for the best fit directly.

The part that matters most is not the arithmetic — a library does that in one call. It is knowing which of three situations you are in: exactly one solution, none at all, or infinitely many. With real data the honest answer is usually **none**, and recognising that rather than fighting it is precisely what M11 is for.

Before you start

You should be able to:

- multiply a matrix by a vector, by rows and by columns — M9;
- check that two shapes can legally be multiplied — M9;
- recognise when two vectors point along the same line — M8;
- call a NumPy function and read what it returns — P6.

Quick check. Solve $2x + y = 7$ and $x - y = 2$.

 Show the answer

Add the two equations: $3x = 9$, so $x = 3$, and then $y = 1$.

You have just done Gaussian elimination on a 2×2 system. Everything below is that, plus a vocabulary for the cases where it does not work.

The idea

A system of linear equations is a matrix equation. These two,

$$\begin{array}{rcl} 2x + y = 7 \\ x - y = 2 \end{array} \quad \text{are} \quad \underbrace{\begin{pmatrix} 2 & 1 \\ 1 & -1 \end{pmatrix}}_A \underbrace{\begin{pmatrix} x \\ y \end{pmatrix}}_{\mathbf{x}} = \underbrace{\begin{pmatrix} 7 \\ 2 \end{pmatrix}}_{\mathbf{b}},$$

and as in M9 there are two ways to look at that, both worth having.

The row picture. Each row is one equation, and in two variables each equation is a line. A solution is a point on every line at once — an intersection. Two lines in a plane can cross once, never (parallel), or everywhere (the same line drawn twice), and those are the only possibilities:

The column picture. By the column reading of M9, $A\mathbf{x}$ is the columns of A scaled by the entries of $\mathbf{x}$ and added. So the question “does $A\mathbf{x} = \mathbf{b}$ have a solution?” becomes

can I build $\mathbf{b}$ out of the columns of A ?

The set of everything you can build that way is called the **span** of the columns — the reachable set. A

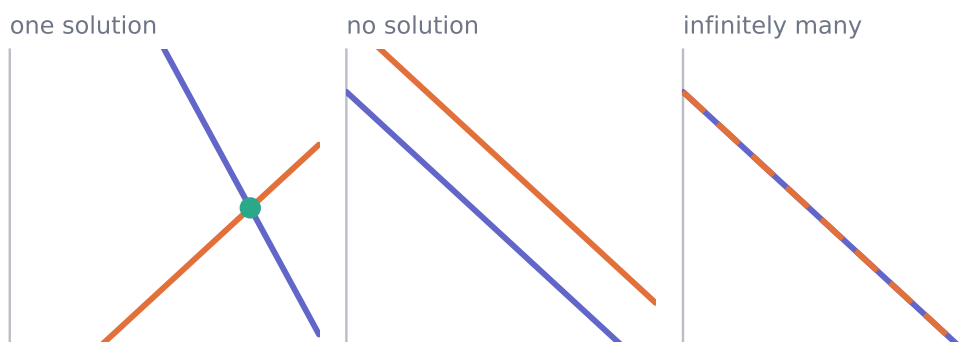


Figure 1: The three things a system can do, as lines. Left: one crossing, one solution. Middle: parallel lines, no point lies on both, and the system is *inconsistent*. Right: the two equations describe the same line, so every point on it solves the system and there are infinitely many solutions. Nothing else can happen.

solution exists exactly when $\mathbf{b}$ lies inside it, and that is a far more useful way to think about it than the lines, because it survives into dimensions you cannot draw.

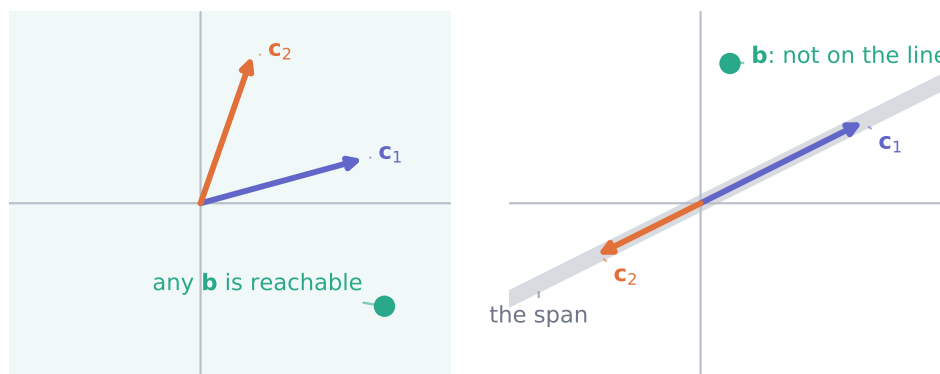


Figure 2: The same three cases seen from the columns. Left: two columns pointing in different directions span the whole plane, so every target is reachable and there is one recipe for each. Right: two columns along the same line span only that line — targets on it are reachable in infinitely many ways, and everything off it, including the $\mathbf{b}$ marked here, is unreachable no matter what $\mathbf{x}$ you choose.

The mechanics

Elimination

The method is the one from the quick check, written down carefully: add multiples of one equation to another until the system is triangular, then substitute back. On

$$\begin{aligned} 2x + y &= 7 \\ x - y &= 2 \end{aligned}$$

subtract $\frac{1}{2} \times (\text{row 1})$ from row 2 to remove x there, leaving $-\frac{3}{2}y = -\frac{3}{2}$, so $y = 1$; substitute into row 1 to get $x = 3$.

Three operations are allowed, because none of them changes the solution set: swap two rows, multiply a row by a nonzero number, and add a multiple of one row to another. That is the whole algorithm, and

`numpy.linalg.solve` is a careful version of it — careful mainly about *which* row it uses to eliminate with, because dividing by a tiny pivot destroys accuracy.

Singular matrices

Elimination breaks down when a row eliminates to nothing. For a square A that happens exactly when the columns are dependent — one is a multiple of the other, or more generally one is a combination of the rest — and such an A is called **singular**. A singular matrix has a span smaller than the space it lives in, so most targets are unreachable, and the few that are reachable are reachable in infinitely many ways. Singular is the boundary between the second and third panels of both figures.

For a 2×2 matrix there is a single number that detects it, the **determinant**:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc, \quad \text{singular} \iff \det A = 0.$$

For the system above, $\det = (2)(-1) - (1)(1) = -3 \neq 0$, so one solution, which is what we found.

The inverse, and why not to use it

If A is square and non-singular there is a matrix A^{-1} with $A^{-1}A = AA^{-1} = I$, and then

$$A\mathbf{x} = \mathbf{b} \implies \mathbf{x} = A^{-1}\mathbf{b}.$$

That is the right way to *think* and the wrong way to *compute*. Forming A^{-1} does more arithmetic than solving the system, and every extra operation is another chance to lose digits. **Write `np.linalg.solve(A, b)`, not `np.linalg.inv(A) @ b`** — the two agree on well-behaved problems and diverge exactly when you most need the answer, as the check at the end of this lesson shows.

When “solvable” is too crude a question

$\det A \neq 0$ says a unique solution exists. It says nothing about whether you can compute it reliably. If the columns are nearly dependent — the lines nearly parallel — the crossing point is exquisitely sensitive to the right-hand side:

The number that measures this is the **condition number**, `np.linalg.cond(A)`. Roughly: a condition number of 10^k means you can expect to lose about k digits of accuracy between the data and the answer. It is the same phenomenon that made gradient descent diverge in M6 — a long narrow valley and a nearly singular matrix are two descriptions of one thing.

More equations than unknowns

A dataset gives you one equation per row and one unknown per parameter, so a model with two parameters fitted to 238 students is 238 equations in 2 unknowns. Such a system is **overdetermined**, and unless the data is exactly linear it has no solution at all. That is not a failure of the data or the method; it is the normal case, and it is the whole reason M11 exists.

Worked example

The cohort system has no solution — and the one you can solve instead.

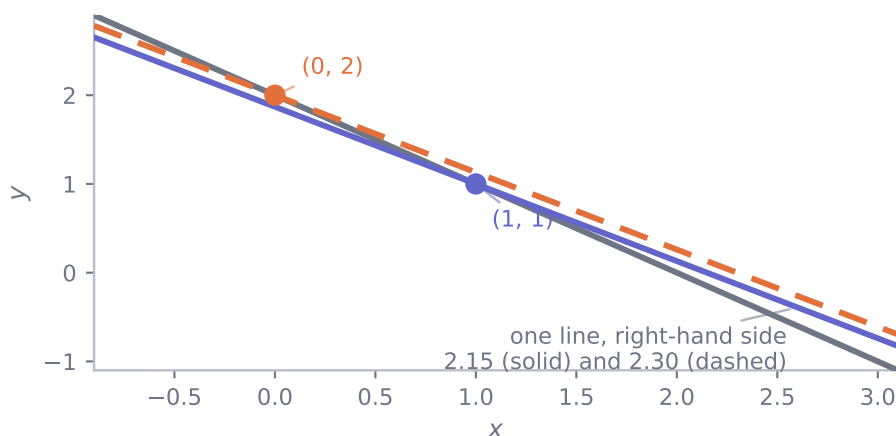


Figure 3: Two nearly parallel lines. Moving the second line's right-hand side from 2.15 to 2.30 — a shift you can barely see — slides the crossing point from (1, 1) to (0, 2). The system is not singular, so a unique solution exists in both cases; it is just that where the solution *is* depends on the data far more strongly than you might expect.

Take the design matrix of M9 and ask for an *exact* fit: values of θ with $X\theta = \mathbf{y}$, every prediction landing on every student's actual score.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h, y = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()
X = np.column_stack([np.ones(len(h)), h])

print(f"X is {X.shape}: {X.shape[0]} equations in {X.shape[1]} unknowns")
print(f"rank of X           {np.linalg.matrix_rank(X)}")
print(f"rank of [X | y]      {np.linalg.matrix_rank(np.column_stack([X, y]))}")
```

```
X is (238, 2): 238 equations in 2 unknowns
rank of X           2
rank of [X | y]      3
```

Those two ranks are the test. The rank of X is the dimension of the span of its columns; here it is 2, so the reachable set is a plane inside a 238-dimensional space. Attaching $\mathbf{y}$ raises the rank to 3, which says $\mathbf{y}$ adds a direction the columns could not reach: $\mathbf{y}$ is **outside the span**, and $X\theta = \mathbf{y}$ has no solution. The right panel of the span figure, in 238 dimensions.

So ask a different question — not *hit $\mathbf{y}$ exactly*, but *get as close as possible* — and the condition for that was already found in M9: the residual must be orthogonal to every column of X , that is $X^T(\mathbf{y} - X\theta) = \mathbf{0}$. Rearranged, that is a system small enough to solve by hand:

$$X^T X \theta = X^T \mathbf{y} \quad \Longleftrightarrow \quad \begin{pmatrix} 238 & 3908.4 \\ 3908.4 & 79872.82 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 15534.7 \\ 251330.53 \end{pmatrix}$$

238 equations have become 2. Check it is solvable, then solve it:

$$\det = (238)(79872.82) - (3908.4)^2 = 19,009,731.16 - 15,275,590.56 = 3,734,140.6,$$

which is not zero, so there is exactly one answer. Eliminate: multiply row 1 by 3908.4/238 and subtract it from row 2, leaving a single equation in b ; then substitute back for a . The arithmetic is unpleasant enough to be worth handing over:

```
A = X.T @ X
rhs = X.T @ y
theta = np.linalg.solve(A, rhs)

print(f"det(X'X)    {A[0,0]*A[1,1] - A[0,1]*A[1,0]:.1f}")
print(f"cond(X'X)   {np.linalg.cond(A):.1f}")
print(f"theta       intercept {theta[0]:.6f}    slope {theta[1]:.6f}")
print(f"residual of the solved system {np.linalg.norm(A @ theta - rhs):.3e}")
print(f"but of the original system    {np.linalg.norm(X @ theta - y):.4f}")
```

```
det(X'X)    3,734,140.6
cond(X'X)   1716.7
theta       intercept 69.226117    slope -0.240793
residual of the solved system  5.846e-11
but of the original system      125.4985
```

Two residuals, and the difference between them is the whole point of this lesson. The 2×2 system is solved essentially exactly — a residual of 6×10^{-11} against right-hand sides of order 10^5 — because that system genuinely has an answer. The original 238-equation system is still missed by 125.5, and no choice of θ does better, because y is not in the span and nothing can put it there.

The answer itself is the one you have seen twice already: intercept 69.226117, slope -0.240793 , the same numbers gradient descent walked to in [M6](#) and `np.polyfit` returned in [M9](#). Three routes — an iterative walk, a library call and a 2×2 system solved by elimination — and one answer, which is the reassurance that all three are computing the same thing.

Try it yourself

Exercise 1 Solve $\begin{matrix} x + 2y = 5 \\ 3x - y = 1 \end{matrix}$ by elimination, and check your answer by substitution.

Show the solution

Subtract $3 \times$ (row 1) from row 2: $(3x - y) - 3(x + 2y) = 1 - 15$, so $-7y = -14$ and $y = 2$. Then row 1 gives $x = 5 - 4 = 1$.

Check both equations, not just the one you substituted into: $1 + 4 = 5$ ✓ and $3 - 2 = 1$ ✓. Checking only the equation you used to back-substitute will confirm an arithmetic slip rather than catch it.

$\det = (1)(-1) - (2)(3) = -7 \neq 0$, consistent with there being exactly one solution.

Exercise 2 For each system, say how many solutions it has and why.

$$(a) \quad \begin{matrix} x + y = 3 \\ 2x + 2y = 6 \end{matrix} \qquad (b) \quad \begin{matrix} x + y = 3 \\ 2x + 2y = 7 \end{matrix}$$

💡 Show the solution

Both have $\det = (1)(2) - (1)(2) = 0$, so neither has a unique solution — but they differ, and the determinant alone cannot tell you how.

(a) infinitely many. The second equation is exactly twice the first, so it adds nothing. Every point on $x + y = 3$ is a solution.

(b) none. The left-hand sides are still proportional but the right-hand sides are not: $x + y$ cannot be both 3 and 3.5. The lines are parallel and distinct.

In column language: the columns $(1, 2)$ and $(1, 2)$ span only the line through $(1, 2)$. In (a), $\mathbf{b} = (3, 6)$ lies on that line and is reachable in infinitely many ways; in (b), $\mathbf{b} = (3, 7)$ does not and is unreachable.

Exercise 3 You fit a model with 5 parameters to 300 rows of data, giving a 300×5 matrix X . Someone asks you to solve $X\theta = \mathbf{y}$ exactly. What do you tell them, and what do you compute instead?

💡 Show the solution

There will almost certainly be **no exact solution**. The columns of X span at most a 5-dimensional set inside a 300-dimensional space, and a real $\mathbf{y}$ — with measurement noise, and a model that is at best approximately right — will not lie in it. Only a suspiciously perfect dataset would.

What you compute instead is the θ minimising $\|\mathbf{y} - X\theta\|$, by solving the 5×5 system $X^T X \theta = X^T \mathbf{y}$. And the honest footnote: if there *were* an exact solution with 5 parameters and 300 rows, that would be a reason for suspicion rather than celebration. It would mean $\mathbf{y}$ landed exactly inside a 5-dimensional slice of a 300-dimensional space, which does not happen by accident. The usual explanation is that one of your columns is secretly a copy of the thing you are predicting. (It does not strictly follow that the data is noise-free — noise that happens to lie along the columns leaves no residual either — but that is a coincidence you should not expect, and looking for a leaked column is the right first move.)

Check it in Python

Solve, and then check the solution rather than trusting it:

```
for A_small, b_small in [(np.array([[2.0, 1.0], [1.0, -1.0]]), np.array([7.0,
- 2.0])),
                        (np.array([[7.0, 2.0], [3.0, 5.0]]), np.array([1.0,
- 1.0]))]:
    x = np.linalg.solve(A_small, b_small)
    print(f"solution {x}")
    print(f"    A @ x == b ? {np.array_equal(A_small @ x, b_small)}"
          f"    allclose ? {np.allclose(A_small @ x, b_small)}"
          f"    residual {np.abs(A_small @ x - b_small).max():.2e}")
```

```
solution [3. 1.]
```

```
    A @ x == b ? True    allclose ? True    residual 0.00e+00
```

```
solution [0.10344828 0.13793103]
```

```
    A @ x == b ? False    allclose ? True    residual 1.11e-16
```

Both solutions are correct. The first system happens to have a whole-number answer that survives the round trip exactly, so `==` says `True`; the second has an answer that does not terminate in binary, and

`==` says False while the residual is 10^{-16} . Which one you get is an accident of the arithmetic, so `==` is never the question to ask. Substitute back and measure the residual.

A singular system refuses, which is the behaviour you want:

```
singular = np.array([[1.0, 1.0], [2.0, 2.0]])
for target in ([3.0, 6.0], [3.0, 7.0]):
    try:
        print(f"b={target} ->", np.linalg.solve(singular, np.array(target)))
    except np.linalg.LinAlgError as exc:
        print(f"b={target} -> LinAlgError: {exc}")
```

```
b=[3.0, 6.0] -> LinAlgError: Singular matrix
```

```
b=[3.0, 7.0] -> LinAlgError: Singular matrix
```

Note that *both* fail with the same error, although mathematically one has infinitely many solutions and the other has none. `solve` promises a unique answer and declines whenever it cannot give one; distinguishing the two cases is `matrix_rank`'s job, as in the worked example.

And the reason not to invert. The Hilbert matrices below are famous for being very nearly singular, which makes them a stress test rather than a realistic case — but the ordering they reveal holds generally:

```
def hilbert(n):
    return np.array([[1.0/(i + j + 1) for j in range(n)] for i in range(n)])

print(f"{'n':>3} {'condition':>10} {'solve error':>12} {'inv @ b error':>14}")
for n in (6, 10, 14):
    H = hilbert(n)
    x_true = np.ones(n)
    b = H @ x_true
    err_solve = np.abs(np.linalg.solve(H, b) - x_true).max()
    err_inv = np.abs(np.linalg.inv(H) @ b - x_true).max()
    print(f"{'n':>3} {np.linalg.cond(H):>10.1e} {err_solve:>12.2e}
        ↳ {err_inv:>14.2e}")
```

n	condition	solve error	inv @ b error
6	1.5e+07	2.39e-10	1.05e-09
10	1.6e+13	1.69e-04	5.55e-03
14	6.2e+17	8.90e+00	5.41e+02

Two lessons in one table. `inv` is consistently the worse of the two — by a factor of four at $n = 6$, thirty at $n = 10$ and sixty at $n = 14$ — so prefer `solve` as a matter of habit. And by $n = 14$ *both* are useless: the condition number has passed 10^{16} and the answer carries no correct digits at all. That last part is a statement about *double precision*, which carries about 16 digits — the condition number says how many of them you lose, so a problem this bad exhausts the supply. Eighty-digit arithmetic would solve the same system perfectly well. What you cannot do is rescue it with a cleverer algorithm at the same precision: the fix is more digits, or a better-posed question.

Common wrong turns

Where this usually goes wrong

Computing the inverse to solve a system `np.linalg.inv(A) @ b` does more work than `np.linalg.solve(A, b)` and is reliably less accurate. There is almost no reason to form an inverse numerically; if you find yourself typing `inv`, ask what you actually want.

Assuming a solution exists because the algebra is legal With more equations than unknowns the normal outcome is no solution at all. That is a fact about the data, not a bug, and it is why least squares exists.

Reading $\det A \neq 0$ as “this will work” A determinant is a yes/no on uniqueness and says nothing about accuracy. A matrix can be comfortably non-singular and still have a condition number of 10^{10} , at which point the answer you get is mostly noise. Check `np.linalg.cond`, not just the determinant.

Using a tiny determinant as a measure of near-singularity Scale every entry of a 2×2 matrix by 0.001 and the determinant falls by a factor of a million while the matrix is exactly as well behaved as before. The determinant has units; the condition number does not.

Checking a solution with `==` `A @ x == b` will be `False` for reasons that have nothing to do with your mathematics. Use `np.allclose`, and look at the size of `A @ x - b`.

Expecting more data to make a system easier More rows means more equations against the same unknowns, which makes an exact solution *less* likely, not more. What more rows buy you is a better approximate answer, which is a different thing.

Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A is 2×2 with $\det A = 0$ and $A\mathbf{x} = \mathbf{b}$ has at least one solution. How many does it have?
 - a. exactly one
 - b. none
 - c. infinitely many
 - d. cannot tell
2. In column language, what exactly is the condition for $A\mathbf{x} = \mathbf{b}$ to have a solution?
3. X is 300×5 . What are the shapes of $X^T X$ and $X^T \mathbf{y}$, and why is turning the problem into that system worth doing?
4. Two colleagues solve the same system. One writes `inv(A) @ b`, the other `solve(A, b)`, and they disagree in the sixth decimal place. Who is right, and what should you check next?
5. `np.linalg.solve` raises `LinAlgError` on your system. Name the two quite different situations that produce that, and how you would tell them apart.

Show the answers

1. **(c) infinitely many.** A zero determinant rules out a unique solution, so the only options are none or infinitely many — and we were told at least one exists. Option (d) is the tempting one, but the extra fact given closes it off.
2. **$\mathbf{b}$ must lie in the span of the columns of A** — it must be expressible as a combination of them. That is the same statement as $\text{rank}(A) = \text{rank}([A \mid \mathbf{b}])$, which is how you check it in code.
3. $X^T X$ is 5×5 and $X^T \mathbf{y}$ has 5 entries. It is worth doing because 300 equations that have no

solution become 5 that do, and the size of the system you actually solve stops depending on how much data you have.

4. First establish whether the disagreement is even large. Sixth *decimal place* is an absolute measure and tells you nothing on its own — if the solution is of size 10^9 , a gap of 10^{-6} is the last bit of double precision and both answers are excellent. What matters is the disagreement *relative* to the size of the answer. If that is around 10^{-6} , roughly ten digits have been lost, and the thing to check next is `np.linalg.cond(A)`. `solve` is the better bet either way, but when conditioning is the problem the response is to reformulate, not to pick a winner.
5. Either the matrix is **singular** — columns dependent, so no unique solution exists — or it is **not square**, so `solve` does not apply at all. The shape distinguishes them immediately. If it is square and singular, compare `matrix_rank(A)` with `matrix_rank([A | b])` to find out whether you are in the “no solutions” case or the “infinitely many” one.

Where this shows up next

M11 takes the unsolvable system of the worked example seriously: it derives $X^T X \theta = X^T \mathbf{y}$ rather than pulling it out of **M9**, and explains why the closest reachable point is the one whose residual is orthogonal to the span. **M12** explains the condition number properly, in terms of how much a matrix stretches in its most and least favoured directions. **S11** fits this system to the cohort and asks what the answer means, which no amount of linear algebra can settle.

When there is no solution: least squares

Unit 9 · Lesson 1 of 3 · Mathematics · M11 Reading: 5 pages With practice: ☐ 2 hours Assumes: M5, M10

Why this lesson is here

M10 ended on an unsolvable system, and said that with real data this is the normal case rather than a mishap. This lesson is the answer to *then what*.

If you cannot hit $\mathbf{y}$, get as close to it as you can. That single sentence, made precise, produces the most-used calculation in applied statistics — every fitted line, every regression coefficient, every “controlling for” in a results table. It arrives here twice over: once from the calculus of M5 and once from the geometry of M8, and the two derivations meet at the same small system of equations. Meeting it from both sides is what makes the formula memorable instead of arbitrary.

Before you start

You should be able to:

- say when $A\mathbf{x} = \mathbf{b}$ has a solution, in terms of the span — M10;
- set a derivative to zero and check you have a minimum — M5;
- take a dot product and recognise orthogonality — M8;
- form $X^T X$ and read its shape — M9.

Quick check. Which value of a makes $(3 - a)^2 + (7 - a)^2$ smallest?

💡 Show the answer

Differentiate: $-2(3 - a) - 2(7 - a) = 0$, so $4a = 20$ and $a = 5$ — the mean of 3 and 7, exactly as M5 found in general.

That is least squares with one parameter and no predictors. Everything in this lesson is that calculation with more columns.

The idea

We want $X\theta = \mathbf{y}$ and cannot have it: by the column reading of M9, $X\theta$ can only ever land inside the span of X ’s columns, and M10 showed that the cohort’s $\mathbf{y}$ is outside it.

So change the question. Among all the points the span *can* reach, which is nearest to $\mathbf{y}$? “Nearest” means smallest distance, and distance is the length of M8, so we are choosing θ to minimise

$$\|\mathbf{y} - X\theta\|^2 = \sum_{i=1}^n (y_i - (X\theta)_i)^2,$$

the sum of squared residuals. Hence **least squares**.

Geometry answers this before any algebra does. The nearest point of a plane to a point off it is the foot of the perpendicular — any other point in the plane is further away, by Pythagoras. So the winning θ is the one whose residual sticks straight out of the span:

That right angle is the whole of least squares. Turning it into equations takes three lines, and the next

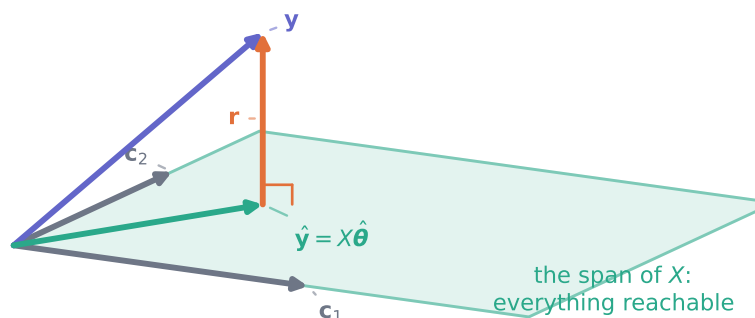


Figure 1: Least squares in one picture — a perspective drawing, in which the shaded parallelogram is a flat span seen edge-on from above rather than a region of the page, so the right angle at the foot of $\mathbf{r}$ is a fact about the vectors and not about the printed picture. The shaded region is everything the columns of X can reach; $\mathbf{y}$ is not in it. The closest reachable point $\hat{\mathbf{y}} = X\hat{\boldsymbol{\theta}}$ is directly below $\mathbf{y}$, and the residual $\mathbf{r}$ joining them meets the span at a right angle. Any other point in the shaded region is further from $\mathbf{y}$, because moving within the span adds a second perpendicular leg to the triangle.

section does it twice.

The mechanics

Route one: geometry

The residual $\mathbf{r} = \mathbf{y} - X\hat{\boldsymbol{\theta}}$ must be perpendicular to the span, and the span is built from the columns of X , so it is enough for $\mathbf{r}$ to be orthogonal to every column. By M8, orthogonal means dot product zero; by M9, all those dot products at once are $X^T \mathbf{r}$. So

$$X^T(\mathbf{y} - X\hat{\boldsymbol{\theta}}) = \mathbf{0} \quad \Longleftrightarrow \quad X^T X \hat{\boldsymbol{\theta}} = X^T \mathbf{y}.$$

These are the **normal equations** — “normal” in its geometric sense of *at right angles*, nothing to do with the normal distribution. M9 found $X^T \mathbf{r} = \mathbf{0}$ by computing it on the cohort and M10 solved the resulting system; this is where it comes from.

Route two: calculus

Take the same quantity and minimise it as a function of the parameters, exactly as in M6. With $r_i = y_i - \sum_j x_{ij}\theta_j$,

$$L(\boldsymbol{\theta}) = \sum_i r_i^2, \quad \frac{\partial L}{\partial \theta_j} = \sum_i 2r_i \frac{\partial r_i}{\partial \theta_j} = -2 \sum_i x_{ij} r_i = -2 (\text{column } j) \cdot \mathbf{r}.$$

Setting every partial derivative to zero says every column has zero dot product with $\mathbf{r}$ — which is $X^T \mathbf{r} = \mathbf{0}$ again. The two routes are the same statement in different languages.

And any stationary point really is a minimum, not just a candidate. L is a sum of squares, so it is never negative and it has no maximum; it is a bowl, and every flat place on a bowl is at the bottom. That is the M5 check, done once for the whole family of problems rather than case by case.

What is *not* guaranteed is that the flat place is a single point. If two columns of X are identical the bowl has a level trough running along it, every point of which is an equally good answer, and L does not grow at all as you slide along it. That is the next section.

When the answer is unique

$X^T X$ is invertible exactly when the columns of X are linearly independent — when no column is a combination of the others. Then

$$\hat{\theta} = (X^T X)^{-1} X^T \mathbf{y},$$

a formula worth recognising and, as M10 insisted, not worth computing that way.

If two columns are identical, the span is unchanged by trading one for the other and there are infinitely many equally good $\hat{\theta}$. The interesting case is the near miss: columns that are nearly dependent leave a long flat valley in the loss, along which the fit barely changes while the coefficients move a great deal.

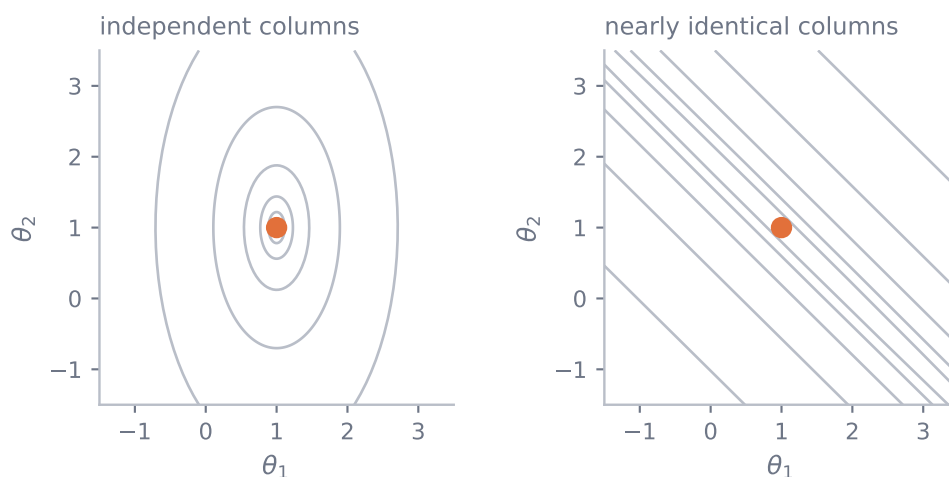


Figure 2: The loss surface for two independent predictors (left) and two nearly identical ones (right), on the same synthetic data. Both have exactly one minimum, marked. But on the right the floor of the valley is almost level for a long way, so the fit can barely tell one point along it from another — the *predictions* are pinned down and the individual *coefficients* are not.

Why libraries do not form $X^T X$

Multiplying X by its own transpose **squares the condition number** of M10: $\text{cond}(X^T X) = \text{cond}(X)^2$. On the cohort's two-column design that is a jump from 41.4 to 1717, which costs nothing; on a badly scaled design it is the difference between an answer and noise. So write

```
theta, *_ = np.linalg.lstsq(X, y, rcond=None)
```

which works on X directly and never forms $X^T X$ at all. The normal equations are how you *understand* least squares; `lstsq` is how you compute it.

Worked example

Adding one column reverses the answer.

Fit the cohort two ways: score on practice hours alone, then score on practice hours *and* the diagnostic score taken on arrival.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h = usable["hours_practice"].to_numpy()
g = usable["diagnostic_score"].to_numpy()
y = usable["final_score"].to_numpy()
ones = np.ones(len(y))

for name, X in [("1 + hours", np.column_stack([ones, h])),
                ("1 + hours + diagnostic", np.column_stack([ones, h, g]))]:
    theta, *_ = np.linalg.lstsq(X, y, rcond=None)
    residual = np.linalg.norm(y - X @ theta)
    print(f"{name:<24} theta = {np.round(theta, 4)}")
    print(f"{'':24} ||r|| = {residual:.4f}    cond(X) = {np.linalg.cond(X):.1f}")
```

1 + hours	theta = [69.2261 -0.2408]
	r = 125.4985 cond(X) = 41.4
1 + hours + diagnostic	theta = [8.9129 0.9156 0.6619]
	r = 70.6309 cond(X) = 614.1

The coefficient on hours of practice goes from -0.2408 to $+0.9156$. Not smaller, not larger — **the opposite sign**. Nothing was done to the data between the two lines, and neither fit is an error; they are answers to different questions.

The explanation is in one number:

```
print(f"corr(hours, diagnostic) {np.corrcoef(h, g)[0, 1]:+.4f}")
print(f"corr(diagnostic, score) {np.corrcoef(g, y)[0, 1]:+.4f}")
```

```
corr(hours, diagnostic) -0.8130
corr(diagnostic, score) +0.6579
```

Students who arrive stronger practise less. So “hours of practice”, on its own, is partly acting as a marker for *arriving weak* — and arriving weak predicts a lower final score. The first fit measures that whole tangle. The second holds the diagnostic score fixed, so its hours coefficient answers a narrower and much more useful question: **among students who arrived equally prepared**, what does an extra hour of practice go with? Just under a mark.

You can see both facts at once by splitting the cohort into three bands of roughly equal size by diagnostic score:

The residual norm also falls, from 125.50 to 70.63, so the second model predicts better. That proves nothing at all. Adding a column can never *increase* the sum of squares, because the old fit remains

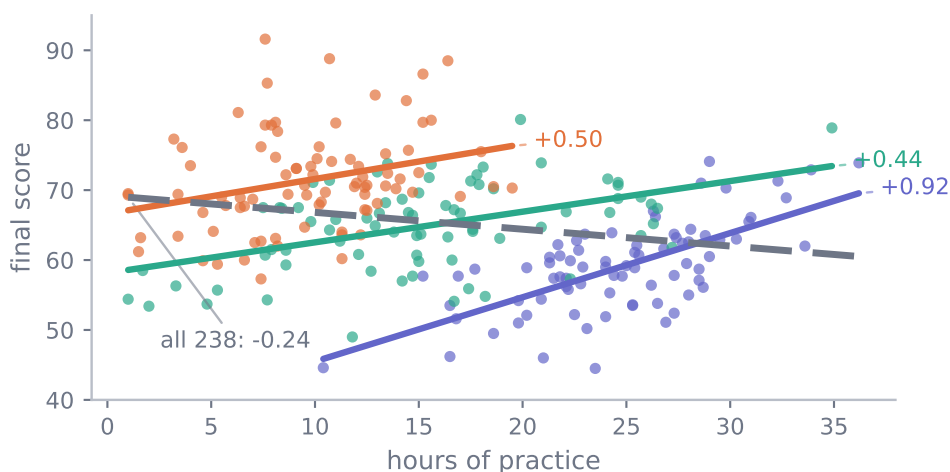


Figure 3: Practice against final score, with the cohort split by diagnostic score into three groups of roughly equal size (78, 76 and 84 — tied scores at the boundaries stop them being exactly equal) — indigo the third who scored lowest on arrival, orange the third who scored highest. Each group’s own fitted line rises. The overall line, in grey, falls — because the groups sit in different places: students who arrived stronger practised less and still scored higher. The three within-group slopes, +0.92, +0.44 and +0.50, are what the multiple-regression coefficient of +0.92 is summarising.

available by giving the new column a coefficient of zero — and for any column that is not already a combination of the existing ones, the fall is strict. A column of pure random noise qualifies, as the check at the end of this lesson demonstrates. A smaller residual is not evidence that a column belongs in the model.

What this example does show is that a regression coefficient has no meaning on its own. “The effect of practice is -0.24 ” and “the effect of practice is $+0.92$ ” are both true statements about the same 238 students, and the thing that distinguishes them is what else was in the matrix. S12 returns to this with the background groups, where it has a name.

Try it yourself

Exercise 1 Take X to be a single column of ones, so the model is “predict the same value for everybody”. Write down $X^T X$ and $X^T \mathbf{y}$, and solve the normal equations.

💡 Show the solution

X is $n \times 1$, so $X^T X$ is 1×1 :

$$X^T X = \sum_{i=1}^n 1 \times 1 = n, \quad X^T \mathbf{y} = \sum_{i=1}^n 1 \times y_i = \sum_i y_i.$$

So $n\hat{\theta} = \sum y_i$ and $\hat{\theta} = \bar{y}$, the mean.

This is M5’s result arriving by a different road, and it is worth noticing what the geometry says about it: the span of a single ones column is the set of vectors with all entries equal, and the closest such vector to $\mathbf{y}$ is the one at $\mathbf{y}$ ’s mean. The residual is then $\mathbf{y} - \bar{y}$, which is orthogonal to the ones column — the fact you proved in M8, Exercise 3.

Exercise 2 A fit gives $X^T X = \begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix}$ and $X^T \mathbf{y} = \begin{pmatrix} 10 \\ 22 \end{pmatrix}$. Find $\hat{\theta}$, and confirm it is the unique answer.

💡 Show the solution

$\det = (4)(14) - (6)(6) = 56 - 36 = 20 \neq 0$, so $X^T X$ is invertible: the columns of X are independent and the answer is unique.

Eliminate: from $4\theta_1 + 6\theta_2 = 10$ and $6\theta_1 + 14\theta_2 = 22$, subtract $1.5\times$ the first from the second to get $5\theta_2 = 7$, so $\theta_2 = 1.4$ and then $\theta_1 = (10 - 8.4)/4 = 0.4$.

Check both equations: $4(0.4) + 6(1.4) = 1.6 + 8.4 = 10$ ✓ and $6(0.4) + 14(1.4) = 2.4 + 19.6 = 22$ ✓.

Exercise 3 Someone builds a design matrix with a column of hours of practice and a second column of the same hours expressed in minutes. What goes wrong, and what happens to the *predictions*?

💡 Show the solution

The second column is $60\times$ the first, so the columns are dependent, the span is no bigger than it was with one of them, and $X^T X$ is singular. There are **infinitely many** $\hat{\theta}$: whatever (θ_1, θ_2) works, so does $(\theta_1 + 60c, \theta_2 - c)$ for any c , since the two changes cancel exactly.

For **this** X the predictions are unaffected. Every one of those parameter vectors produces the same $X\hat{\theta}$, because the closest point in the span is unique even when the recipe for reaching it is not. That is the distinction to carry away: collinearity destroys the interpretation of individual coefficients while the fitted values of a given fit stay put.

Be careful how far you take that. It says the projection is unique for a fixed design matrix; it does not say a nearly collinear fit is robust. Two designs differing in the fourth decimal place can span quite different sets and produce quite different fitted values. In practice you rarely get exact duplication — you get two columns measuring almost the same thing, which is the right-hand panel of the valley figure. `lstsq` returns an answer without complaint, and the coefficients are the first thing to distrust.

Check it in Python

First, that the two derivations and the library agree:

```
X = np.column_stack([ones, h])

normal = np.linalg.solve(X.T @ X, X.T @ y)      # the normal equations
library, *_ = np.linalg.lstsq(X, y, rcond=None)
polyfit = np.polyfit(h, y, 1)[::-1]            # intercept, slope

print(f"normal equations {normal}")
print(f"np.linalg.lstsq {library}")
print(f"np.polyfit      {polyfit}")
print("all three agree:", np.allclose(normal, library) and np.allclose(normal,
    - polyfit))
```

```
normal equations [69.22611682 -0.24079311]
np.linalg.lstsq [69.22611682 -0.24079311]
np.polyfit      [69.22611682 -0.24079311]
all three agree: True
```

Then the claim that gives the method its name — that no other θ has a smaller sum of squares. Nudge the answer in every direction and watch the loss rise:

```
def loss(theta):
    r = y - X @ theta
    return r @ r

rng = np.random.default_rng(2026)
best = loss(library)
beaten = 0
for _ in range(20_000):
    nudge = library + rng.normal(scale=0.05, size=2)
    if loss(nudge) < best:
        beaten = beaten + 1
print(f"loss at the least-squares answer {best:.6f}")
print(f"nudges that beat it, out of 20000: {beaten}")
print(f"and the residual is orthogonal to X: {np.allclose(X.T @ (y - X @
    library), 0)}")
```

```
loss at the least-squares answer 15749.873767
nudges that beat it, out of 20000: 0
and the residual is orthogonal to X: True
```

Twenty thousand attempts and none of them wins, which is what “least” means. The orthogonality check on the last line is the same statement — the two are not independent pieces of evidence, they are the geometry and the arithmetic of one fact.

Next, that a falling residual means nothing. Add a column of pure noise — information that cannot possibly help — and watch the sum of squares drop anyway:

```
X0 = np.column_stack([ones, h])
theta0, *_ = np.linalg.lstsq(X0, y, rcond=None)
rss0 = ((y - X0 @ theta0)**2).sum()
print(f"1 + hours                                RSS {rss0:.4f}")

for trial in range(3):
    noise = rng.normal(size=len(y))
    Xn = np.column_stack([ones, h, noise])
    thetan, *_ = np.linalg.lstsq(Xn, y, rcond=None)
    rssn = ((y - Xn @ thetan)**2).sum()
    print(f" + a column of random noise RSS {rssn:.4f} (fell by {rss0 -
        rssn:.4f})")

Xd = np.column_stack([ones, h, h])
thetad, *_ = np.linalg.lstsq(Xd, y, rcond=None)
print(f" + a duplicate of hours RSS {((y - Xd @ thetad)**2).sum():.4f}
    (unchanged)")
```

```
1 + hours                                RSS 15749.8738
+ a column of random noise RSS 15671.2007 (fell by 78.6731)
+ a column of random noise RSS 15489.4165 (fell by 260.4573)
+ a column of random noise RSS 15734.5969 (fell by 15.2768)
+ a duplicate of hours RSS 15749.8738 (unchanged)
```

Three columns of nonsense, three improvements. The only column that fails to improve the fit is the duplicate — and it fails because it adds nothing to the span, which is exactly the case the guarantee is silent about. Judging a model by whether the residual fell is judging it by something that was going to happen regardless.

Finally, why `lstsq` rather than the normal equations. Fit a polynomial of rising degree, where the columns $1, t, t^2, \dots$ become steadily more alike, and compare the two routes against an answer we know:

```
t = np.linspace(0, 1, 120)

print(f"{'degree':>7} {'cond(X)':>10} {'cond(X.T X)':>12} {'normal eqs':>12}
      {'lstsq':>10}")
for degree in (3, 7, 11):
    Xp = np.column_stack([t**k for k in range(degree + 1)])
    truth = np.ones(degree + 1)
    yp = Xp @ truth                                     # exactly solvable by
    ← construction
    err_normal = np.abs(np.linalg.solve(Xp.T @ Xp, Xp.T @ yp) - truth).max()
    err_lstsq = np.abs(np.linalg.lstsq(Xp, yp, rcond=None)[0] - truth).max()
    print(f"{'degree':>7} {np.linalg.cond(Xp):>10.1e} {np.linalg.cond(Xp.T @
    ← Xp):>12.1e}"
          f" {'err_normal':>12.1e} {'err_lstsq':>10.1e}")
```

degree	cond(X)	cond(X.T X)	normal eqs	lstsq
3	1.2e+02	1.5e+04	1.6e-12	7.5e-15
7	1.2e+05	1.4e+10	2.9e-07	6.0e-12
11	1.2e+08	1.5e+16	6.0e-01	3.3e-09

Two things to read off. $\text{cond}(X.T X)$ really is $\text{cond}(X)$ squared, in every row. And by degree 11 the normal equations return an answer wrong in the *first* decimal place while `lstsq`, working on the same data, is still correct to nine. The mathematics is identical; only the route differs.

Common wrong turns

Where this usually goes wrong

Reading a coefficient without saying what else is in the model The cohort's practice coefficient is -0.24 alone and $+0.92$ alongside the diagnostic score. Neither number means anything until you say which matrix it came out of.

Treating a smaller residual as evidence Adding any column can only reduce the sum of squares, since the previous fit survives by giving the new column a zero coefficient. Improvement is automatic and therefore not informative.

Computing $(X^T X)^{-1} X^T y$ literally It is the right formula and the wrong algorithm — it inverts a matrix (M10) whose condition number has already been squared. `np.linalg.lstsq` does neither.

Confusing “the projection is unique” with “the predictions are stable” For a *fixed* X , the closest point in the span is unique even when the $\hat{\theta}$ reaching it is not — so duplicate columns leave the fitted values of that fit alone while making the coefficients arbitrary. That is a statement about one design matrix, not a promise of robustness: nudge the data slightly and a nearly collinear fit can move its fitted values a long way too. Coefficients are the first casualty, not the only possible one.

Thinking least squares requires a straight-line relationship The model must be linear in the **parameters**, not in the data. Fitting $y = \theta_0 + \theta_1 t + \theta_2 t^2$ is a least-squares problem with three columns, and its curve is a parabola.

Believing the minimum is the truth Least squares finds the closest point in the span of the columns you supplied. If the right column is not in X , the answer is the best available lie, and the arithmetic will not mention it.

Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

- What condition does least squares impose on the residual $\mathbf{r}$?
 - $\mathbf{r} = \mathbf{0}$
 - $\mathbf{r}$ is orthogonal to every column of X
 - $\mathbf{r}$ is as small as the data allows in every entry
 - $\sum r_i = 0$
- Why is $L(\theta) = \|\mathbf{y} - X\theta\|^2$ guaranteed to have a minimum rather than a maximum at its stationary point?
- A colleague adds a column and reports that the residual sum of squares fell. What have they demonstrated?
- $\text{cond}(X) = 10^4$. What is $\text{cond}(X^\top X)$, and roughly how many digits does solving the normal equations cost you?
- Two columns of X are nearly identical. Which of these become unreliable — the fitted values, the individual coefficients, or both?

Show the answers

- (b).** Orthogonality to every column is the defining condition, and it is what $X^\top \mathbf{r} = \mathbf{0}$ says. (a) is what we wanted and could not have. (d) is *implied* by (b) whenever X contains a column of ones — the ones column's dot product with $\mathbf{r}$ is $\sum r_i$ — but it is one consequence, not the condition. (c) is not a coherent requirement: residuals trade off against one another and cannot all be minimised at once.
- Because L is a sum of squares: never negative, and with no maximum. So any flat place has to be a lowest point — a bowl has no summit. Note what this does *not* claim: if the columns of X are dependent the bottom is a level trough rather than a point, and there are infinitely many equally good answers. Guaranteed minimum, not guaranteed unique.
- Nothing.** The residual sum of squares cannot rise when a column is added, because the previous fit is still reachable by setting the new coefficient to zero. They have demonstrated that their arithmetic works.
- $\text{cond}(X^\top X) = 10^8$, so roughly **eight** of the sixteen digits double precision offers, against about four if you work with X directly. That is the entire argument for `lstsq`.
- The coefficients first and worst.** For the design matrix in front of you the closest point in the span is unique, so $X\hat{\theta}$ is pinned down while the division of credit between two near-identical columns is not. But uniqueness is not stability: perturb the data a little and a nearly collinear fit can shift its fitted values as well. The safe summary is that the

coefficients become untrustworthy immediately and the predictions are merely better behaved, not safe.

Where this shows up next

M12 explains the condition number properly — as a ratio of how much a matrix stretches its most and least favoured directions — and gives the decomposition that `lstsq` actually uses. **S11** takes this exact fit and asks the statistical questions: how uncertain is that slope, and what would license calling it an effect. **S12** takes up the sign reversal in the worked example and shows how routinely it happens.

Directions in data: eigenvectors and the SVD

Unit 10 · Lesson 1 of 3 · Mathematics · M12 Reading: 5 pages With practice: ☒ 2.5 hours Assumes: M11

Why this lesson is here

A matrix does not treat all directions alike. It stretches some, shrinks others, and leaves a few pointing exactly where they started. This lesson asks which directions those are and by how much — one question, which turns out to answer four you have already met.

Why $\text{cond}(X^T X)$ is $\text{cond}(X)^2$ (M11). Why a nearly collinear design leaves a long flat valley in the loss instead of a bowl (M11). What `np.linalg.lstsq` is actually doing (M10). And what a *principal component* is, which is where a dataset with forty columns gets turned into a picture you can look at. It is also the last lesson of the Mathematics track.

Before you start

You should be able to:

- read $A\mathbf{x}$ as an operation on a vector — M9;
- form $X^T X$ and say what shape it is — M9;
- say what the condition number of a matrix is *for* — M11;
- take a length and recognise orthogonality — M8.

Quick check. What does $\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$ do to the vectors $(1, 0)$ and $(0, 1)$?

💡 Show the answer

$(1, 0) \mapsto (2, 0)$ and $(0, 1) \mapsto (0, 3)$: each is stretched, neither is turned.

Those two directions are **eigenvectors** of that matrix, with stretch factors 2 and 3. This lesson is about finding such directions when the matrix is not conveniently diagonal — and about what they tell you when the matrix is made out of data.

The idea

Take every unit vector — a whole circle of directions — and apply a matrix to all of them. What comes back is an **ellipse**. Not sometimes: always, for any 2×2 matrix, and the higher-dimensional version is an ellipsoid.

That is a strong statement and it is the whole geometry of the subject. It says a matrix does exactly three things: it picks out a set of perpendicular input directions, stretches each by its own factor, and lays the results down along a (possibly different) set of perpendicular output directions.

Once you see that ellipse, several loose ends from earlier lessons tie themselves off. Its longest and shortest half-widths are the biggest and smallest stretch the matrix can apply; their ratio is the condition number. A nearly singular matrix is one whose ellipse is nearly flat.

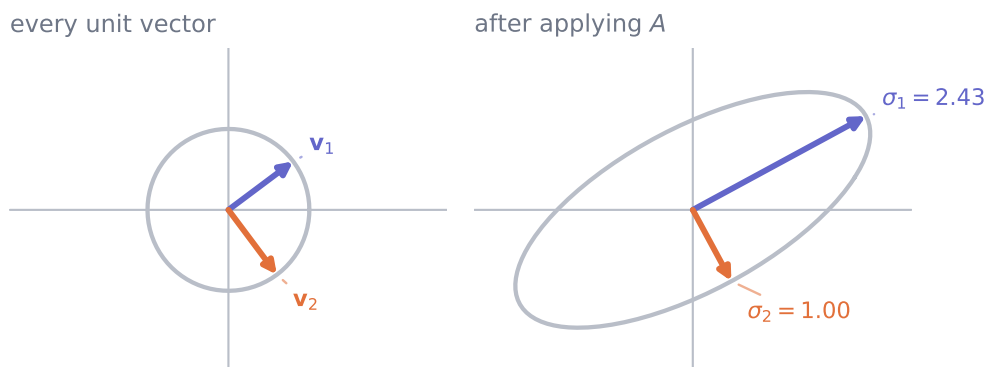


Figure 1: Left: every unit vector, with two particular ones picked out. Right: what the matrix $A = \begin{pmatrix} 2 & 0.9 \\ 0.4 & 1.4 \end{pmatrix}$ does to all of them. The circle becomes an ellipse, and the two marked directions are the pair that is still perpendicular after the journey: they come out along the ellipse's axes, stretched by 2.43 and 1.00. No other perpendicular pair survives as one, and no direction is stretched by more than 2.43 or less than 1.00.

The mechanics

Eigenvectors

For a **square** matrix A , a nonzero $\mathbf{v}$ with

$$A\mathbf{v} = \lambda\mathbf{v}$$

is an **eigenvector**, and the number λ is its **eigenvalue**. In words: the matrix does not turn this direction at all, it only scales it. Note that $\mathbf{v}$ is a *direction* — any nonzero multiple of an eigenvector is one too, with the same eigenvalue, so libraries return a unit-length representative and the sign is arbitrary.

Not every real matrix has real eigenvectors: a rotation by 90° turns every direction, so nothing survives unturned. But the matrices that arise from data are of a much better-behaved kind.

Symmetric matrices, which are the ones you get

A is **symmetric** if $A^T = A$. Both matrices this course cares about are: $X^T X$ from [M11](#), and the covariance and correlation matrices of a dataset. For those, a theorem guarantees the good case:

A real symmetric matrix has real eigenvalues, and its eigenvectors can be chosen to be mutually orthogonal.

So a symmetric matrix has a full set of perpendicular directions it merely scales — its ellipse's axes *are* its eigenvectors. This is why data analysis keeps producing orthogonal directions without anyone asking for them.

The singular value decomposition

Eigenvectors need a square matrix, and a data matrix is 238×3 . The **SVD** removes that restriction. Any matrix at all factorises as

$$X = U \Sigma V^T,$$

where Σ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots$ — the **singular values** — and U and V

have orthogonal unit columns. Read left to right it is exactly the ellipse: V^T names the input directions, Σ stretches each by its own σ , and U says where the results point.

The bridge to eigenvalues is the fact that pays the debts listed at the top:

$$X^T X = V \Sigma^2 V^T,$$

so the **eigenvalues of $X^T X$ are the squares of the singular values of X** , with the same directions.

The condition number, at last

$$\text{cond}(X) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

— the ratio between the ellipse's longest and shortest half-widths, which is the most and the least a matrix stretches anything. (Not its *eccentricity*, which is a different number: $\sqrt{1 - \sigma_2^2/\sigma_1^2}$.) And now the M11 claim is a one-liner: the singular values of $X^T X$ are σ_i^2 , so their ratio is the square of the ratio, and $\text{cond}(X^T X) = \text{cond}(X)^2$ — for a design matrix with independent columns, which is the case that matters here. (If the columns are dependent, $X^T X$ has a zero eigenvalue and its condition number is infinite, which is the same warning said louder.) Forming $X^T X$ squashes the ellipse, which is why `lstsq` — which runs on the SVD of X itself — does not form it.

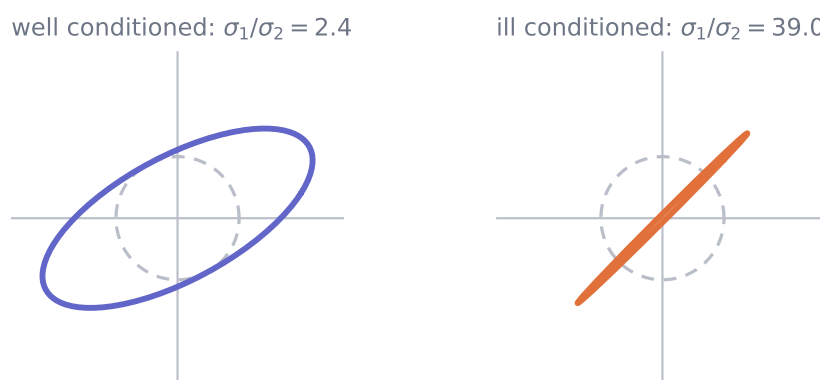


Figure 2: The unit circle under two matrices, drawn to the same scale. Left, $\sigma_1/\sigma_2 = 2.4$: a mild ellipse, and a system you can solve happily. Right, $\sigma_1/\sigma_2 = 39$: nearly flat, so directions that were far apart on the circle arrive almost on top of one another — which is what it means for information to be lost, and what a large condition number is measuring.

Principal components

Now point the same machinery at the data rather than at the design matrix. Centre every column, and take the covariance matrix (or, if the columns are on different scales, standardise first and take the correlation matrix). It is symmetric, so it has orthogonal eigenvectors. Those directions are the **principal components**, and each eigenvalue is the variance of the data along its direction.

The first principal component is the single direction along which the cloud is most spread out. This is a change of *coordinates*, not a change of data: you are looking at the same cloud from a better angle.

Worked example

The cohort has two stories in it, and PCA finds both.

Take the three numeric columns, standardise them so no column dominates through its units alone, and eigen-decompose the correlation matrix.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
names = ["diagnostic_score", "hours_practice", "final_score"]
Z = usable[names].to_numpy()

C = np.corrcoef(Z.T)
values, vectors = np.linalg.eigh(C)
order = np.argsort(values)[::-1]
values, vectors = values[order], vectors[:, order]

print("variance along each principal direction:")
for k in range(3):
    axis = vectors[:, k]
    axis = axis * np.sign(axis[np.argmax(abs(axis))]) # sign is arbitrary; fix
    it
    loads = " ".join(f"{n.split('_')[0]:>10} {c:+.3f}" for n, c in zip(names,
    axis))
    print(f" PC{k+1} eigenvalue {values[k]:.4f} ({100*values[k]/3:4.1f}%)
    {loads}")
print(f"\neigenvalues sum to {values.sum():.4f}, one per standardised column")
```

variance along each principal direction:

PC1	eigenvalue	2.1673	(72.2%)	diagnostic	+0.667	hours	-0.562	final	+0.489
PC2	eigenvalue	0.7720	(25.7%)	diagnostic	-0.051	hours	+0.619	final	+0.783
PC3	eigenvalue	0.0607	(2.0%)	diagnostic	+0.743	hours	+0.548	final	-

0.384

eigenvalues sum to 3.0000, one per standardised column

Read the loadings as recipes. **PC1**, which carries 72% of the variation, is *arrived prepared, practised little, scored well* — the three columns moving together in the pattern that has haunted these notes since [M6](#). It is a single axis along which the cohort mostly lies, and it is a confound rather than an effect.

PC2, with a further 26%, has almost no diagnostic score in it and loads practice and final score in the *same* direction: *practised a lot, scored well, regardless of how you arrived*. That is the real effect, the one [M11](#) recovered as a coefficient of +0.92 once the diagnostic score was held fixed. Nobody told the decomposition to look for it.

PC3 is worth 2% and is nearly nothing, which is itself informative: it says the three columns are close to satisfying one linear relation, so they carry about two dimensions of information rather than three.

Finally, the promised connection to conditioning. The singular values of the two design matrices from [M11](#) give their condition numbers directly:

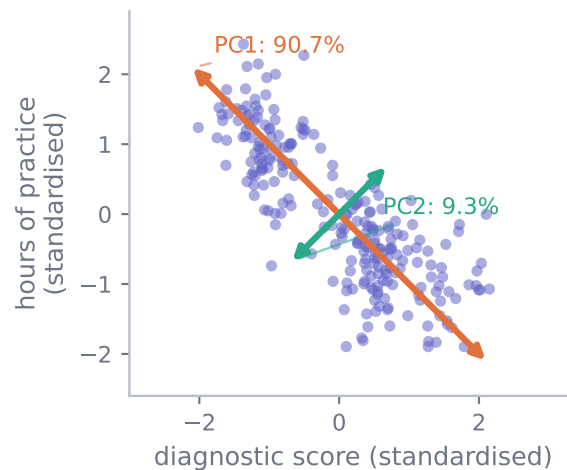


Figure 3: The cohort in two of its standardised columns, with the principal axes of that two-column problem drawn to lengths proportional to $\sqrt{\lambda}$. The cloud is an ellipse and the axes are its principal directions: 90.7% of the variation lies along the long one, which runs down and to the right — students who arrived stronger practised less. The other 9.3% is all that remains once you know where a student sits along it.

```
h = usable["hours_practice"].to_numpy()
g = usable["diagnostic_score"].to_numpy()
ones = np.ones(len(h))

for name, X in [("1 + hours", np.column_stack([ones, h])),
                ("1 + hours + diagnostic", np.column_stack([ones, h, g]))]:
    sigma = np.linalg.svd(X, compute_uv=False)
    print(f"{name:<24} sigma = {np.round(sigma, 3)}")
    print(f"{'':24} sigma_max/sigma_min = {sigma.max()/sigma.min():8.2f}")
    print(f"{'':24} np.linalg.cond = {np.linalg.cond(X):8.2f}")
    print(f"{'':24} cond(X'X) = {np.linalg.cond(X.T @ X):.1f}")
    print(f"{'':24} and cond(X)^2 = {np.linalg.cond(X)**2:.1f}")
```

1 + hours	sigma = [282.956 6.829]	sigma_max/sigma_min = 41.43	np.linalg.cond = 41.43
	cond(X'X) = 1716.7	and cond(X)^2 = 1716.7	
1 + hours + diagnostic	sigma = [1024.104 177.347 1.668]	sigma_max/sigma_min = 614.14	np.linalg.cond = 614.14
	cond(X'X) = 377170.7	and cond(X)^2 = 377170.7	

The third design's smallest singular value is 1.67 against a largest of 1024, and that near-zero direction is the collinearity of [M11](#) measured rather than described: the columns very nearly satisfy a linear relation, so there is a direction in parameter space that the design barely responds to. It is the long flat valley, given a number.

Try it yourself

Exercise 1 Find the eigenvalues and eigenvectors of $A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$.

💡 Show the solution

Eigenvalues solve $\det(A - \lambda I) = 0$:

$$(3 - \lambda)^2 - 1 = 0 \implies 3 - \lambda = \pm 1 \implies \lambda = 4 \text{ or } 2.$$

For $\lambda = 4$: $(A - 4I)\mathbf{v} = 0$ gives $-v_1 + v_2 = 0$, so $\mathbf{v} \propto (1, 1)$. For $\lambda = 2$: $v_1 + v_2 = 0$, so $\mathbf{v} \propto (1, -1)$. Two things to notice. The eigenvectors are **orthogonal** — $(1, 1) \cdot (1, -1) = 0$ — as the theorem promises for a symmetric matrix. And they are directions: $(2, 2)$ and $(-1, -1)$ are the same eigenvector as $(1, 1)$, so a library returning $(-0.707, -0.707)$ has not given you a different answer.

Exercise 2 A 500×8 design matrix has singular values $\sigma = (40, 12, 9, 5, 3, 2, 1, 0.004)$. What is $\text{cond}(X)$, what is $\text{cond}(X^T X)$, and what should you do?

💡 Show the solution

$$\text{cond}(X) = \frac{40}{0.004} = 10,000, \quad \text{cond}(X^T X) = 10,000^2 = 10^8.$$

Working through the normal equations would cost about eight of the sixteen digits double precision offers; working from the SVD of X , about four. So first: use `lstsq` and not $(X^T X)^{-1}$. But the deeper answer is that $\sigma_8 = 0.004$ against $\sigma_1 = 40$ is a message about the *data*, not the arithmetic. It says the eight columns very nearly satisfy a linear relation — that they carry about seven columns' worth of information. The coefficient attached to that near-null direction is whatever noise happens to say. Look for the relation, and consider dropping a column or combining two.

Exercise 3 You run PCA on a dataset with heights in metres and incomes in pounds, without standardising. PC1 comes out as almost exactly the income axis. Is that a discovery?

💡 Show the solution

No. Incomes vary over tens of thousands and heights over fractions of a metre, so income's variance dwarfs height's by a factor of billions before anything interesting has been considered. PCA maximises variance, and variance has units.

Re-express the incomes in thousands of pounds and heights in millimetres and PC1 will change, from data that is identical in every respect that matters. That is the test: a result that moves when you change a unit is a result about the units.

Standardise each column first — subtract the mean and divide by the standard deviation — which is the same as working from the correlation matrix rather than the covariance matrix, and is what the worked example above does. The exception is when the columns are already in the same units and their relative sizes are genuinely meaningful, in which case leaving them alone is the informed choice.

Check it in Python

First, the definition:

```
A2 = np.array([[3.0, 1.0], [1.0, 3.0]])
values2, vectors2 = np.linalg.eigh(A2)
```

```

for k in range(2):
    v = vectors2[:, k]
    print(f"lambda = {values2[k]:.4f}    A @ v = {np.round(A2 @ v, 4)}"
          f"    lambda * v = {np.round(values2[k]*v, 4)}"
          f"    equal: {np.allclose(A2 @ v, values2[k]*v)}")
print("eigenvectors orthogonal:", np.isclose(vectors2[:, 0] @ vectors2[:, 1], 0))

```

```

lambda = 2.0000    A @ v = [-1.4142  1.4142]    lambda * v = [-1.4142  1.4142]    equal: True
lambda = 4.0000    A @ v = [2.8284 2.8284]    lambda * v = [2.8284 2.8284]    equal: True
eigenvectors orthogonal: True

```

Then the bridge between the SVD and the eigenvalues, which is the claim the condition-number result rests on:

```

X = np.column_stack([ones, h, g])
U, sigma, Vt = np.linalg.svd(X, full_matrices=False)

eig_of_XtX = np.sort(np.linalg.eigvalsh(X.T @ X))[:-1]
print("singular values of X      ", np.round(sigma, 4))
print("their squares             ", np.round(sigma**2, 4))
print("eigenvalues of X.T @ X    ", np.round(eig_of_XtX, 4))
print("match:", np.allclose(sigma**2, eig_of_XtX))
print("U S V.T rebuilds X:", np.allclose(U @ np.diag(sigma) @ Vt, X))

```

```

singular values of X      [1024.1041  177.3473   1.6675]
their squares             [1.04878930e+06  3.14520663e+04  2.78070000e+00]
eigenvalues of X.T @ X    [1.04878930e+06  3.14520663e+04  2.78070000e+00]
match: True
U S V.T rebuilds X: True

```

And a check that fails, on purpose. It is tempting to compare eigenvectors entry by entry; the sign is arbitrary, so that comparison is unreliable even when nothing is wrong:

```

mine = vectors2[:, 0]
theirs = -vectors2[:, 0]          # an equally valid answer to the same problem

print("mine   ", np.round(mine, 4))
print("theirs", np.round(theirs, 4))
print("allclose on the entries:      ", np.allclose(mine, theirs))
print("same direction (|dot| == 1):  ", np.isclose(abs(mine @ theirs), 1.0))

```

```

mine   [-0.7071  0.7071]
theirs [ 0.7071 -0.7071]
allclose on the entries:      False
same direction (|dot| == 1):  True

```

The first test says the two disagree and the second says they do not. The second is the one to trust: compare eigenvectors by the absolute value of their dot product *after* scaling both to unit length — which is what `eigh` returns, so the check above is safe as written — or by fixing a sign convention yourself, as the worked example did.

Common wrong turns

Where this usually goes wrong

Treating an eigenvector's sign or length as meaningful $\mathbf{v}$ and $-2\mathbf{v}$ are the same eigenvector. Two libraries, or the same library on two days, may hand you opposite signs, and neither is wrong. Never compare them with `==` or `allclose` on the raw entries.

Running PCA on unstandardised columns Variance carries units, so the column with the largest *variance* wins — which is not the same as the column with the largest numbers, since PCA centres first. A column of values near a billion that barely moves contributes almost nothing. Unless every column is already in the same units for a good reason, standardise first — that is, decompose the correlation matrix rather than the covariance matrix.

Calling PC1 “the most important variable” It is not a variable at all; it is a weighted combination of all of them. In the cohort, PC1 is *arrived prepared, practised little, scored well* — three columns at once, which is precisely why it was worth computing.

Reading a small eigenvalue as a useless direction It says the data barely varies along there, which usually means the columns nearly satisfy a linear relation. That is a finding about your variables, and often the most actionable thing in the output.

Expecting eigenvectors from a non-square matrix $A\mathbf{v} = \lambda\mathbf{v}$ needs input and output in the same space. A 238×3 matrix has no eigenvectors; it has singular values and vectors, which is what the SVD exists to provide.

Assuming a large condition number means you made a mistake It usually means your columns overlap. The remedy is in the data — drop a column, combine two, collect a design that separates them — not in the solver.

Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. X is a design matrix with two independent columns and singular values 10 and 2. What is $\text{cond}(X^T X)$?
 - a. 5
 - b. 25
 - c. 10
 - d. 100
2. Why does a correlation matrix always have orthogonal eigenvectors, while a general square matrix need not?
3. The eigenvalues of a 4×4 correlation matrix are 2.9, 0.9, 0.2, 0.0. What does the last one tell you, and what do the four sum to?
4. Your PCA gives PC1 loadings (0.71, -0.71); a colleague's gives (-0.71, 0.71) on the same data. Who is wrong?
5. In one sentence each: what do U , Σ and V contribute in $X = U\Sigma V^T$?

 Show the answers

1. **(b)** 25. $\text{cond}(X) = 10/2 = 5$, and the eigenvalues of $X^T X$ are the squares of the singular values, so the ratio squares: 25. Option (a) is $\text{cond}(X)$, the most tempting wrong answer; (d) squares the wrong thing.
2. Because a correlation matrix is **symmetric**, and the spectral theorem guarantees real eigenvalues and orthogonal eigenvectors for real symmetric matrices. A general square matrix has no such guarantee — a 90° rotation of the plane has no real eigenvectors at all, because it turns every direction. (Not every rotation does: a 180° rotation is $-I$ and leaves every direction unturned, and a rotation in three dimensions always fixes its axis.)
3. A zero eigenvalue means there is a direction along which the data does not vary at all: the four columns satisfy an exact linear relation, so one is redundant and the correlation matrix is singular. The four sum to 4 — the eigenvalues of a correlation matrix sum to the number of columns, because each standardised column contributes variance 1.
4. **Neither**. The sign of an eigenvector is arbitrary, and $(0.71, -0.71)$ and $(-0.71, 0.71)$ are the same direction. If they disagreed by anything other than an overall sign there would be something to investigate.
5. V names the perpendicular input directions; Σ says how much each is stretched; U names the perpendicular output directions those stretched pieces are laid along.

Where this shows up next

That completes the Mathematics track. What it was for becomes visible in Statistics.

S3 uses correlation as a summary of two columns; this lesson showed what a whole matrix of them contains. **S11** fits the design matrix of **M11** and asks how uncertain its coefficients are — a question whose answer is governed by the singular values above. **S12** returns to PC1, the *arrived prepared, practised little, scored well* direction, and gives it its proper name.